

Mathematical Pathways to Machine Learning:

Calculus, Linear Algebra, and Optimization

Xiantao Li

Pennsylvania State University

August 2, 2026

Copyright © 2026 Xiantao Li.

Except where otherwise noted, this work is licensed under the Creative Commons Attribution–NonCommercial 4.0 International License (CC BY-NC 4.0):

<https://creativecommons.org/licenses/by-nc/4.0/>.

For commercial licensing and other permissions, contact Xiantao.Li@psu.edu.

First edition, Version 1.0, August 2026.

ISBN: [insert ISBN if obtained]

DOI: <https://doi.org/10.5281/zenodo.21752763>

The source files and supplementary materials are available at:

https://xxl12.github.io/from_math_to_ml

Preface

In the fall of 2025, Pennsylvania State University launched a Bachelor of Science degree in Artificial Intelligence Engineering (AIE), housed in the School of Electrical Engineering and Computer Science within the College of Engineering. The author was involved in the development of the core curriculum beginning in the fall of 2023. It became clear early in that process that students entering the program would need a solid mathematical foundation—one that draws elements from calculus, linear algebra, and optimization, together with their computer implementations—before they could engage meaningfully with machine learning (ML) courses. One option was simply to wait for students to complete the traditional courses in each of these areas. But doing so would create a long chain of prerequisites that delays the students’ progression through the program. More importantly, the traditional courses are designed and structured with a much broader audience in mind, not with ML-oriented applications as their focus. As a result, they lack the kinds of examples that allow students to quickly connect the mathematical elements to the machine learning methods they will soon encounter.

Another difficulty was the scarcity of suitable textbooks. Although several texts on the mathematical foundations of machine learning exist, they tend to be theory-heavy, written at a level more appropriate for advanced undergraduate or graduate students, and therefore not well suited for a course of this kind, aimed at first-year and early sophomore students. This gap motivated the creation of **MATH 225: Mathematical Foundations for Machine Learning**. The course number was deliberately chosen between MATH 220 (Matrices) and MATH 230 (Calculus and Vector Analysis), reflecting its role as a bridge that integrates linear algebra and multivariable calculus, extended toward optimization and computation.

In the first offering of MATH 225, the course relied primarily on the author’s handwritten notes, in which examples were collected and compiled so that students could work through them by hand while, at the same time, seeing a direct connection to machine learning. These notes form the basis of the present text. The overall hope is that, by the end of the course, students will have these mathematical concepts and operations “built in,” and will have familiarized themselves with implementing the basic operations in a programming language, so that they are well prepared for further study of machine learning and artificial intelligence.

The author thanks his colleagues Mehrdad Mahdavi and Sharon Huang for their input during the initial course design, and the students in the first offering of MATH 225, whose comments and feedback allowed the author to adjust and refine the topics.

How to Read This Book

This book grew out of the weekly lectures of MATH 225 at Penn State, a course designed to give first-year and early sophomore students the mathematical foundation behind modern machine learning. At Penn State, MATH 225 assumes one semester of calculus and introductory programming,

but no prior course in linear algebra, multivariable calculus, or optimization. The mathematical development begins from first principles, and the computational exercises require only elementary NumPy.

The book is organized into three parts, mirroring the three big questions of machine learning:

1. **How is data represented?** (Part I: Linear Algebra, Weeks 1–6.) Data—images, text, medical records—becomes vectors and matrices, and we learn the geometry of these objects: lengths, angles, projections, eigenvalues, and the singular value decomposition.
2. **How does a machine measure and reduce its error?** (Part II: Vector Calculus and Unconstrained Optimization, Weeks 7–9.) We build the gradient and the Hessian, and use them to find the lowest point of a “loss landscape.”
3. **How do algorithms actually get there—and what if there are rules to obey?** (Part III: Iterative and Constrained Optimization, Weeks 10–15.) We study gradient descent, momentum, Newton’s method, Lagrange multipliers, the KKT conditions, and duality, ending with real machine learning applications such as support vector machines.

Throughout the book you will meet six kinds of colored boxes:

Definition boxes (yellow)

These give the precise mathematical meaning of a new term. Read them slowly; every word matters.

Key Idea boxes (purple)

These summarize the single most important takeaway of a section—the thing to remember on an exam five weeks later.

Worked Example boxes (gray)

These items walk through a calculation in complete detail, with the numbers being substituted shown in **red**. Work through them with a pencil; do not just read them.

Common Pitfall boxes (red)

These flag the mistakes students make most often, so you can avoid them.

ML Link boxes (blue)

These connect the mathematics you just learned to a real machine learning system—neural networks, search engines, recommender systems, and more. They are the “why are we learning this?” boxes. You do not need to understand every detail on the first reading. Their purpose is to give you a glimpse of where the mathematics is heading and help you begin seeing these connections.

Exam Practice boxes (indigo)

These contain, verbatim, problems from the two midterm exams of the Spring 2026 offering of MATH 225, placed in the week whose material they test. Treat them as practice exams: attempt each one closed-book, under time pressure, *before* looking back at the worked examples.

Three ways through the book

Different readers need different depths, so each chapter supports three tracks:

- **Minimal track:** read the definitions, the Key Idea boxes, and one worked example per section. This is the fastest route to a working vocabulary.
- **Standard course track:** everything above, plus all worked examples, the end-of-chapter exercises, and the Exam Practice boxes. This is the path the MATH 225 lectures follow.
- **ML-preparation track:** everything above, plus the ML Link boxes and the capstone material of Weeks 13–15, ideally reproducing the by-hand computations in NumPy as you go.

Notation Guide

Symbol	Meaning
a, b, λ, α	scalars (single numbers), written in plain italics
$\mathbf{x}, \mathbf{y}, \mathbf{u}$	vectors, written in bold lowercase; always <i>columns</i> unless stated
A, B, H	matrices, written in capital letters
$\mathbb{R}^n$	the set of all vectors with n real entries
$\mathbb{R}^{m \times n}$	the set of all matrices with m rows and n columns
$\mathbf{x}^T, A^T$	the transpose (rows become columns)
$\ \mathbf{x}\ $	the length (norm) of the vector $\mathbf{x}$
$\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u} \cdot \mathbf{v}$ or $\mathbf{u}^T \mathbf{v}$	the inner (dot) product
∇f	the gradient of f (vector of first partial derivatives)
H or $\nabla^2 f$	the Hessian of f (matrix of second partial derivatives)
λ	an eigenvalue, or (in Part III) a Lagrange multiplier
σ_i	a singular value

One convention to watch. The symbol λ does double duty—an eigenvalue in Parts I–II and a Lagrange multiplier in Part III—and the context always makes clear which is meant. Also note that data can be stored either as the *columns* of a matrix (as in the PCA of Week 6, where each data point is a column $\mathbf{y}_n$) or as the *rows* (as in the regression of Week 9, where each row of A is one training example). Both conventions are standard; we flag the choice whenever it matters.

Contents

Preface	iii
I The Geometry of Data (Linear Algebra)	1
1 Week 1: Vectors, Vector Spaces, and Matrices	2
1.1 The Intuition: How AI Sees the World	2
1.2 Vectors and Their Two Basic Operations	3
1.3 Vector Spaces: The Rules of the Game	3
1.3.1 Subspaces	4
1.3.2 Span: Building Subspaces from Ingredients	5
1.4 Matrices and the Matrix–Vector Product	6
1.5 Matrix–Matrix Multiplication	7
1.5.1 Special Matrices	7
1.6 Machine Learning Application	7
1.7 End of Chapter Exercises	8
2 Week 2: Inner and Outer Products	10
2.1 The Intuition: Measuring Similarity	10
2.2 The Inner Product, Norms, and Angles	10
2.3 The Outer Product: The Opposite Move	11
2.4 Rules for Transposes and Quadratic Forms	12
2.5 Linear Independence, Basis, and Coordinates	12
2.6 The Matrix Inverse and the Determinant	13
2.7 Machine Learning Application	13
2.8 End of Chapter Exercises	14
3 Week 3: Orthogonality and Projections	15
3.1 The Intuition: Casting Shadows to Remove Noise	15
3.2 Orthogonal Sets and Why They Are Wonderful	15
3.3 Orthogonal Projection onto a Line	17
3.4 Machine Learning Application	18
3.5 End of Chapter Exercises	18
4 Week 4: Gram–Schmidt and Eigen-Decomposition	20
4.1 The Intuition: Manufacturing Good Axes, Finding Natural Ones	20
4.2 The Gram–Schmidt Process	20
4.2.1 The QR Factorization	21

4.3	Eigenvalues and Eigenvectors: A Matrix's Natural Axes	24
4.3.1	Diagonalization	25
4.3.2	Symmetric Matrices: The Spectral Theorem	26
4.4	Machine Learning Application	26
4.5	End of Chapter Exercises	27
5	Week 5: Eigenvalues in Action and the SVD	28
5.1	The Intuition: Networks and Equilibrium	28
5.2	Matrices from Graphs	28
5.3	Markov Chains and PageRank	29
5.4	The Singular Value Decomposition (SVD)	30
5.4.1	Deriving How to Compute the SVD	31
5.5	Machine Learning Application	32
5.6	End of Chapter Exercises	33
6	Week 6: Principal Component Analysis	36
6.1	The Intuition: Finding the Directions That Matter	36
6.2	Setting Up PCA	36
6.3	Deriving the PCA Solution	37
6.4	The SVD Connection	39
6.5	Machine Learning Application	39
6.6	End of Chapter Exercises	40
II	Navigating the Loss Landscape (Unconstrained Optimization)	41
7	Week 7: Gradients and Steepest Descent	42
7.1	The Intuition: Blindfolded on a Mountain	42
7.2	Warm-up: Derivatives and Taylor Polynomials in One Variable	42
7.3	Partial Derivatives and the Gradient	44
7.4	The Multivariable Chain Rule and the Jacobian	44
7.4.1	A Toolbox of Gradient Identities	49
7.5	Level Curves and the Geometry of the Gradient	49
7.6	The Directional Derivative	50
7.7	Machine Learning Application	51
7.8	End of Chapter Exercises	51
8	Week 8: The Hessian and Taylor Approximations	53
8.1	The Intuition: Feeling the Curvature	53
8.2	The Hessian Matrix	53
8.2.1	Classifying via Eigenvalues	54
8.2.2	The 2×2 Shortcut: The Determinant Test	54
8.3	Linear and Quadratic Approximations in Several Variables	55
8.4	Machine Learning Application	56
8.5	End of Chapter Exercises	56

9	Week 9: Critical Points and Least Squares	58
9.1	The Intuition: Finding the Flat Spots	58
9.2	Convex Functions: When Local Means Global	59
9.3	The Grand Crossover: Least Squares as Optimization	60
9.3.1	Polynomial Regression: Same Machinery, Richer Models	62
9.3.2	How to Solve the Normal Equations Well	62
9.4	Machine Learning Application	63
9.5	End of Chapter Exercises	64
III	The Algorithmic Journey (Iterative and Constrained Optimization)	66
10	Week 10: Norms, Iterations, Convergence	67
10.1	The Intuition: Why We Must Walk, Not Teleport	67
10.2	Vector Norms and Matrix Norms	67
10.3	Linear Iterations and the Convergence Theorem	68
10.4	The Vocabulary of Optimization Problems	69
10.5	Machine Learning Application	70
10.6	End of Chapter Exercises	70
11	Week 11: Gradient Descent and Conditioning	71
11.1	The Intuition: How Big a Step Dare We Take?	71
11.2	Gradient Descent on a Quadratic Bowl: The Full Analysis	71
11.3	The Optimal Rate and the Condition Number	72
11.4	Convergence of the Loss Values	73
11.5	Machine Learning Application	73
11.6	End of Chapter Exercises	74
12	Week 12: Newton's Method and Momentum	76
12.1	The Intuition: Beyond Plain Gradient Descent	76
12.2	Root Finding: Bisection versus Newton	76
12.3	Fixed Points, Contractions, and Rates of Convergence	78
12.4	Newton's Method for Optimization	79
12.5	The Momentum Method: A Heavy Ball Rolling Downhill	80
12.6	Machine Learning Application	81
12.7	End of Chapter Exercises	82
13	Week 13: Equality Constraints and Lagrange Multipliers	83
13.1	The Intuition: Optimizing While Obeying Rules	83
13.2	Two Constrained Problems That Run Machine Learning	85
13.2.1	Quadratic Forms on the Unit Sphere: PCA, Properly This Time	85
13.2.2	Maximum Entropy: Why Uniform Is the Fairest Guess	85
13.3	Algorithms for Constraints: Penalties and Projections	85
13.4	Machine Learning Application	86
13.5	End of Chapter Exercises	87

14 Week 14: Inequality Constraints and the KKT Conditions	88
14.1 The Intuition: The Mathematical Fence	88
14.2 The KKT Conditions	89
14.3 Machine Learning Application	90
14.4 End of Chapter Exercises	91
15 Week 15: Duality and ML Capstones	92
15.1 The Intuition: Bypassing the Binary Guesswork	92
15.2 Soft Fences: The Exterior Penalty Method	93
15.3 Capstone 1: Allocating a Data Budget Optimally	94
15.4 Capstone 2: Support Vector Machines	94
15.4.1 Soft Margins: When the Data Refuse to Separate	96
15.5 Machine Learning Application	96
15.6 End of Chapter Exercises	97
References and Further Reading	99

Part I

The Geometry of Data (Linear Algebra)

Chapter 1

Week 1: Vectors, Vector Spaces, and Matrices

1.1 The Intuition: How AI Sees the World

In high school mathematics we mostly work with *scalars*: single numbers that describe a single quantity, like a speed of 60 mph or a temperature of 72°F. But the data that machine learning systems consume is never a single number. A patient’s health record might contain cholesterol level, blood pressure, and heart rate—three numbers that belong together. A tiny grayscale photo of 28×28 pixels is a package of 784 numbers. A single sentence fed to a language model becomes thousands of numbers.

To handle this multidimensional reality, we bundle numbers into ordered lists called **vectors**. A vector is more than a list, though: it is a *point* (or equivalently, an *arrow*) positioned in a space whose dimension equals the number of entries. Each entry is a *feature* of the data. Once data lives in a geometric space, we can ask geometric questions: How far apart are two patients? What angle do two word-vectors make? Which direction does the data vary the most? How do we move the points around? Those questions, and their answers, are the subject of Part I.

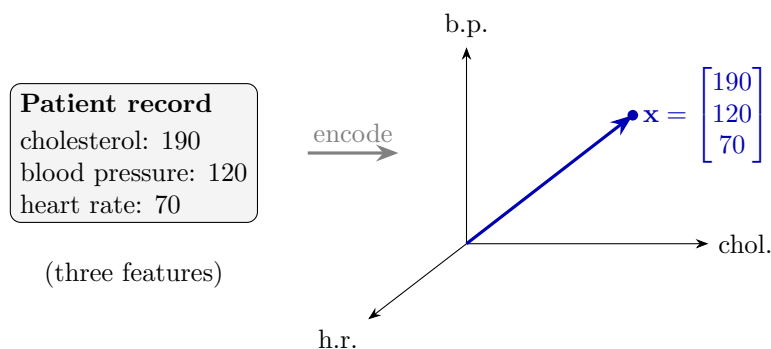


Figure 1.1: A data record becomes a vector: a single point (arrow) in a space with one axis per feature. With 784 pixels, the same picture holds—we just cannot draw 784 axes.

1.2 Vectors and Their Two Basic Operations

Definition: Vector in $\mathbb{R}^n$

An n -dimensional vector, written mathematically as $\mathbf{x} \in \mathbb{R}^n$, is an ordered list of n real numbers, written as a column:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

The number x_i is the i -th *entry* (or *component*) of $\mathbf{x}$.

Only two operations are currently needed: we can **add** two vectors (entry by entry) and we can **scale** a vector by a number (multiply every entry). For example, in $\mathbb{R}^2$,

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}, \quad 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

Geometrically, addition places the arrows tip-to-tail, and scaling stretches (or flips, if the scalar is negative) the arrow.

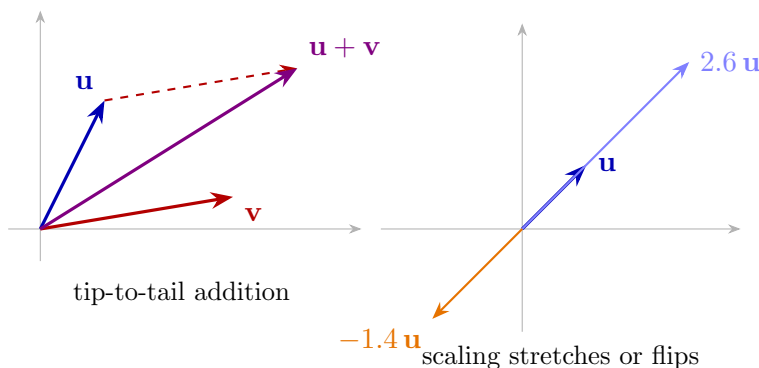


Figure 1.2: The two fundamental vector operations.

1.3 Vector Spaces: The Rules of the Game

Where do vectors live? The answer is in a **vector space**: any collection of objects that can be added and scaled while obeying the familiar rules of arithmetic. The formal definition lists eight axioms; the spirit of all eight is that *addition and scaling behave the way you expect*.

Definition: Vector Space (informal statement of the axioms)

A set V is a vector space if for all $\mathbf{x}, \mathbf{y}, \mathbf{z} \in V$ and all scalars $a, b \in \mathbb{R}$:

1. $\mathbf{x} + \mathbf{y} \in V$ and $a\mathbf{x} \in V$ (closure: you cannot leave the space)
2. $\mathbf{x} + \mathbf{y} = \mathbf{y} + \mathbf{x}$ (order of addition doesn't matter)
3. $(\mathbf{x} + \mathbf{y}) + \mathbf{z} = \mathbf{x} + (\mathbf{y} + \mathbf{z})$ (grouping doesn't matter)

4. there is a zero vector $\mathbf{0}$ with $\mathbf{x} + \mathbf{0} = \mathbf{x}$
5. every $\mathbf{x}$ has a negative $-\mathbf{x}$ with $\mathbf{x} + (-\mathbf{x}) = \mathbf{0}$
6. $a(b\mathbf{x}) = (ab)\mathbf{x}$
7. $a(\mathbf{x} + \mathbf{y}) = a\mathbf{x} + a\mathbf{y}$ and $(a + b)\mathbf{x} = a\mathbf{x} + b\mathbf{x}$
8. $1 \cdot \mathbf{x} = \mathbf{x}$

The surprise—and the power—of this definition is that vector spaces are *everywhere*, not just lists of numbers:

- **Example 1 (data records).** $V = \mathbb{R}^3$, where $\mathbf{x} = [\text{cholesterol}, \text{blood pressure}, \text{heart rate}]^T$. Adding records or averaging them keeps us inside $\mathbb{R}^3$.
- **Example 2 (polynomials).** $V = \mathbb{P}_n$, the set of all polynomials $a_0 + a_1x + \cdots + a_nx^n$ with real coefficients. The sum of two such polynomials is again one, and scaling a polynomial is still a polynomial. A polynomial behaves exactly like the coefficient vector $[a_0, a_1, \dots, a_n]^T \in \mathbb{R}^{n+1}$.
- **Example 3 (trigonometric polynomials).** The set of functions $a_0 + a_1 \cos x + b_1 \sin x + \cdots + a_n \cos nx + b_n \sin nx$. These are the building blocks of audio signals—and of the Fourier analysis used in speech recognition.
- **Example 4 (piecewise linear functions).** Functions built from straight-line segments. ReLU neural networks produce continuous piecewise-linear functions. This is one reason piecewise-linear function spaces are useful mathematical models for understanding neural networks.

Key Idea

“Vector” does not mean “list of numbers.” It means “anything you can add and scale.” Data records, polynomials, sound waves, and even functions computed by neural networks may generate vector spaces.

1.3.1 Subspaces

Often we care about a smaller space sitting inside a bigger one.

Definition: Subspace

A subset $H \subseteq V$ is a **subspace** of V if it is itself a vector space using the same operations. In practice you only need to check three things:

1. $\mathbf{0} \in H$,
2. if $\mathbf{x}, \mathbf{y} \in H$ then $\mathbf{x} + \mathbf{y} \in H$ (closed under addition),
3. if $\mathbf{x} \in H$ and $a \in \mathbb{R}$ then $a\mathbf{x} \in H$ (closed under scaling).

Example 1.1: Which sets are subspaces of $\mathbb{R}^2$?

(a) $H = \{[x_1, 0]^T : x_1 \in \mathbb{R}\}$, the horizontal axis. The zero vector $[0, 0]^T$ is in H ; adding two vectors with second entry 0 keeps the second entry 0; scaling does too. **H is a subspace.**

(b) $H = \{[x_1, x_2]^T : x_2 = ax_1 + b\}$ with $b \neq 0$, a line that misses the origin. The zero vector is *not* on this line (plug in $x_1 = 0$: we get $x_2 = b \neq 0$). **Not a subspace.** Geometrically: in three dimensions, subspaces are lines and planes *through the origin*.

1.3.2 Span: Building Subspaces from Ingredients**Definition: Span**

Given vectors $\mathbf{x}_1, \dots, \mathbf{x}_k \in V$, their **span** is the set of all *linear combinations*:

$$\text{span}\{\mathbf{x}_1, \dots, \mathbf{x}_k\} = \{c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \dots + c_k\mathbf{x}_k : c_1, \dots, c_k \in \mathbb{R}\}.$$

A span is always a subspace: it contains $\mathbf{0}$ (take all $c_i = 0$) and is closed under addition and scaling by construction.

Notice that the span requires exhausting all possible coefficients. The span of one nonzero vector is a line; the span of two non-parallel vectors in $\mathbb{R}^3$ is a plane. Asking “is the data point $\mathbf{d}$ in the span of $\mathbf{x}_1, \mathbf{x}_2$?” is the prototype of every data-fitting question in machine learning: can my model’s building blocks reproduce this observation?

Example 1.2: Testing membership in a span

Is $\mathbf{d} = \begin{bmatrix} 11 \\ 8 \end{bmatrix}$ in $\text{span}\left\{\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ -1 \end{bmatrix}\right\}$?

We seek scalars a, b such that $a \begin{bmatrix} 1 \\ 2 \end{bmatrix} + b \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 11 \\ 8 \end{bmatrix}$. Reading off each row gives a system of two equations:

$$a + 3b = 11$$

$$2a - b = 8$$

From the second equation, $b = 2a - 8$. Substituting into the first: $a + 3(2a - 8) = 11 \implies 7a = 35 \implies a = \mathbf{5}$, and then $b = 2(5) - 8 = \mathbf{2}$. Yes:

$$\mathbf{d} = 5 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

The key observation. We can rewrite the system in compact form by lining the two ingredient vectors up as the *columns of a matrix*:

$$\begin{bmatrix} 1 & 3 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 11 \\ 8 \end{bmatrix}.$$

This is our first encounter of a system of equations expressed with a 2 by 2 matrix, and it motivates everything that follows.

1.4 Matrices and the Matrix–Vector Product

Definition: Matrix

A matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular array of numbers with m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

The entry a_{ij} sits in row i , column j (“row first, column second”). A vector in $\mathbb{R}^m$ is just an $m \times 1$ matrix; a $1 \times n$ matrix is called a *row vector*.

The most important operation in this entire book is the **matrix–vector product** $\mathbf{y} = A\mathbf{x}$, where $A \in \mathbb{R}^{m \times n}$ and $\mathbf{x} \in \mathbb{R}^n$ produce $\mathbf{y} \in \mathbb{R}^m$. There are two equivalent ways to compute and understand it, and you should be fluent in both.

Key Idea: Two views of $A\mathbf{x}$

Row view (how to compute): the i -th entry of $\mathbf{y}$ is the (dot) product of row i of A with $\mathbf{x}$:

$$y_i = a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n.$$

Column view (what it means): $A\mathbf{x}$ is a *linear combination of the columns of A* , with the entries of $\mathbf{x}$ as the weights:

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n, \quad A = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n].$$

The column view says: *the set of all possible outputs $A\mathbf{x}$ is exactly the span of the columns of A .*

Example 1.3: Tracing the matrix–vector product both ways

Let

$$A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Row view. Take the dot product of each row with $\mathbf{x}$:

$$\begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} \color{red}{5} \\ \color{red}{2} \end{bmatrix} \xrightarrow{\text{Row 1}} 2(\color{red}{5}) + (-1)(\color{red}{2}) = 10 - 2 = \color{red}{8}$$

$$\xrightarrow{\text{Row 2}} 3(\color{red}{5}) + 4(\color{red}{2}) = 15 + 8 = \color{red}{23}$$

so $A\mathbf{x} = [8, 23]^T$.

Column view. Weight the columns by the entries of $\mathbf{x}$:

$$A\mathbf{x} = \color{red}{5} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \color{red}{2} \begin{bmatrix} -1 \\ 4 \end{bmatrix} = \begin{bmatrix} 10 \\ 15 \end{bmatrix} + \begin{bmatrix} -2 \\ 8 \end{bmatrix} = \begin{bmatrix} 8 \\ 23 \end{bmatrix}. \checkmark$$

Both views give the same answer—always. This exact operation occurs billions of times per second inside a GPU during AI training.

1.5 Matrix–Matrix Multiplication

If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times k}$, the product $C = AB \in \mathbb{R}^{m \times k}$ is defined entry by entry:

$$c_{ij} = \sum_{\ell=1}^n a_{i\ell} b_{\ell j} = (\text{row } i \text{ of } A) \cdot (\text{column } j \text{ of } B).$$

A second, often more useful view: AB applies A to each column of B ,

$$AB = A[\mathbf{b}_1 \ \mathbf{b}_2 \ \cdots \ \mathbf{b}_k] = [A\mathbf{b}_1 \ A\mathbf{b}_2 \ \cdots \ A\mathbf{b}_k].$$

This makes sense of the chaining rule $(AB)\mathbf{x} = A(B\mathbf{x})$: doing transformation B , then transformation A , is the same as the single transformation AB . This is precisely how a multi-layer neural network is one giant chain of matrix products (though intertwined with nonlinear activation functions).

Common Pitfall: Order matters!

Matrix multiplication is **not commutative**: in general $AB \neq BA$. For instance

$$\begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 2 & 3 \end{bmatrix}, \quad \text{but} \quad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}.$$

Sometimes AB exists while BA is not even defined (the inner dimensions must match: $(m \times n)(n \times k)$ works; $(n \times k)(m \times n)$ does not unless $k = m$).

1.5.1 Special Matrices

- **Diagonal matrices.** $D = \text{diag}(d_1, \dots, d_n)$ has the d_i on the main diagonal and zeros elsewhere. Multiplying on the right, AD , scales the i -th column of A by d_i ; multiplying on the left, DA , scales the i -th row by d_i .
- **Identity matrix.** $I_n = \text{diag}(1, 1, \dots, 1)$. It is the “do nothing” transformation: $AI = IA = A$.
- **Transpose.** A^T swaps rows and columns: $(A^T)_{ij} = a_{ji}$. If A is $m \times n$, then A^T is $n \times m$. For example,

$$\begin{bmatrix} 1 & 1 \\ 2 & 3 \\ 3 & 2 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \end{bmatrix}.$$

1.6 Machine Learning Application

ML Link: Forward Propagation in Neural Networks

A *linear layer* of a neural network takes an input vector $\mathbf{x}$ (e.g. the pixels of an image) and computes $\mathbf{z} = A\mathbf{x} + \mathbf{b}$, where A is a *weight matrix* and $\mathbf{b}$ a *bias vector*. A full neural-network layer then applies a simple nonlinear *activation function* ϕ to each entry:

$$\mathbf{y} = \phi(A\mathbf{x} + \mathbf{b}).$$

The entries of A , together with the biases, are learned parameters that encode how input features are combined. The nonlinearity is essential: without it, a stack of layers $A_3(A_2(A_1\mathbf{x})) = (A_3A_2A_1)\mathbf{x}$ would collapse into a single matrix product—one layer in disguise. When a computer runs a language model or recognizes your face, the dominant cost is exactly these matrix–vector products, executed trillions of times. Figure 1.3 shows the dictionary between the picture and the algebra (for clarity the figure shows only the linear part $\mathbf{y} = A\mathbf{x}$).

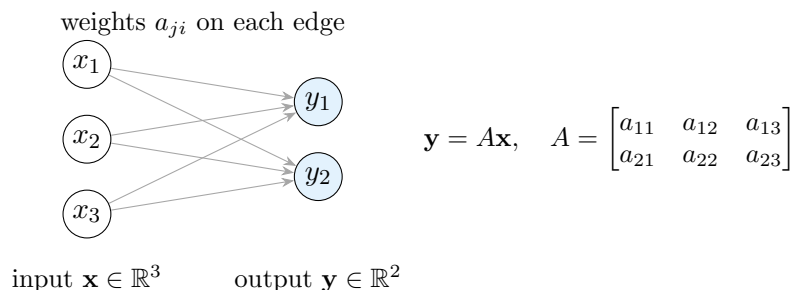


Figure 1.3: One layer of a neural network *is* a matrix–vector product: $y_j = a_{j1}x_1 + a_{j2}x_2 + a_{j3}x_3$ is exactly the row view of $A\mathbf{x}$.

1.7 End of Chapter Exercises

Week 1 Exercises

1.1 Compute the product of the matrix A and vector $\mathbf{x}$: $A = \begin{bmatrix} 1 & -2 & 3 \\ 0 & 4 & -1 \\ 2 & 1 & 0 \end{bmatrix}$, $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$. Do it twice: once using the row view and once using the column view, and check the answers agree.

1.2 Compute the matrix products AB and BA for $A = \begin{bmatrix} 2 & 1 \\ 0 & -1 \\ 3 & 2 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 0 & -2 \\ 4 & 1 & 0 \end{bmatrix}$. What are the dimensions of each product? Does the order of multiplication matter?

1.3 Let $M = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 4 \\ 0 & 1 & 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix}$. Compute $M\mathbf{v}$ and verify explicitly that the result equals $2(\text{col } 1) + 1(\text{col } 2) - 1(\text{col } 3)$.

1.4 Consider the vector space $\mathbb{P}_2$ of polynomials of degree at most 2. Let $u(t) = 1 + t$ and $v(t) = t + t^2$. Find the value of k such that $p(t) = 3 + 4t + kt^2$ lies in $\text{span}\{u, v\}$. (*Hint: write $p = au + bv$ and match the coefficients of 1 , t , and t^2 .*)

1.5 Decide whether each set is a subspace of $\mathbb{R}^2$. Justify using the three subspace

conditions. (a) all vectors with $x_1 = x_2$; (b) all vectors with $x_1 \geq 0$; (c) all vectors with $x_1 x_2 = 0$.

1.6 (Dimension check) If $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, and $\mathbf{b} \in \mathbb{R}^m$, what are the dimensions of $A\mathbf{x}$, $A^T(A\mathbf{x} - \mathbf{b})$, and $A^T A$? (Get in the habit of checking shapes *before* computing—it catches most errors for free.)

1.7 (Debug the statement) A student says: “A neural network with ReLU activations forms a vector space, because ReLU networks compute piecewise-linear functions.” Is this correct? Explain, using the closure axioms and the discussion of Example 4 in Section 1.3.

Exam Practice

Compute the matrix product AB , where

$$A = \begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 1 & 0 \\ 2 & 0 & 1 \\ 1 & 2 & -1 \end{bmatrix}.$$

Show the calculation for each entry in the resulting matrix. (Before computing, predict the dimensions of AB , and explain why BA is not defined here.)

Chapter 2

Week 2: Inner Products, Outer Products, and Independence

2.1 The Intuition: Measuring Similarity

Vectors are powerful, but to extract meaning from them we need ways to *compare* them. If one vector represents the word “cat” and another the word “dog,” how does an AI know they are related, while “cat” and “carburetor” are not? The answer is geometry: related vectors point in similar directions. To make “similar directions” precise we need lengths and angles, and both come from a single tool: the **inner product**.

2.2 The Inner Product, Norms, and Angles

Definition: Inner (dot) product, norm, distance

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{u}^T \mathbf{v} = \mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n \quad (\text{a single number}).$$

The **norm** (length) of a vector is $\|\mathbf{u}\| = \sqrt{\mathbf{u}^T \mathbf{u}} = \sqrt{u_1^2 + \cdots + u_n^2}$, which is just the Pythagorean theorem in n dimensions. A vector with $\|\mathbf{u}\| = 1$ is a **unit vector**. The **distance** between $\mathbf{x}$ and $\mathbf{y}$ is $\|\mathbf{x} - \mathbf{y}\|$.

The fundamental geometric fact—which we will use constantly—is:

Key Idea: The angle formula

$$\mathbf{u}^T \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta, \quad \text{so} \quad \cos \theta = \frac{\mathbf{u}^T \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|},$$

where θ is the angle between $\mathbf{u}$ and $\mathbf{v}$. In particular:

- $\mathbf{u}^T \mathbf{v} > 0$: the vectors point in broadly the *same* direction ($\theta < 90^\circ$);
- $\mathbf{u}^T \mathbf{v} = 0$: the vectors are **orthogonal** (perpendicular, $\theta = 90^\circ$)—they share no common direction;

- $\mathbf{u}^T \mathbf{v} < 0$: the vectors point in broadly *opposite* directions.

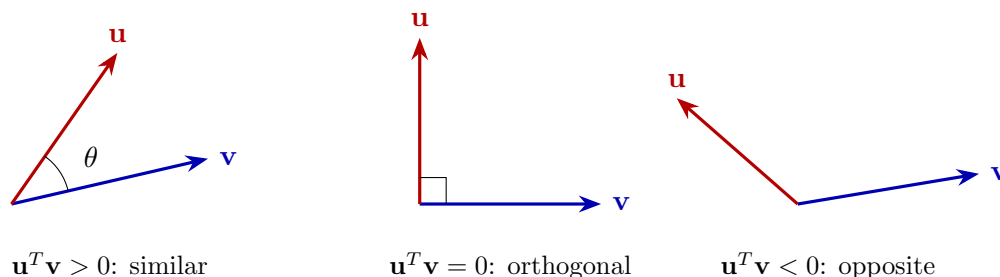


Figure 2.1: The sign of the inner product reads off the geometric relationship between two vectors.

Example 2.1: Lengths, angles, and orthogonality

Let $\mathbf{u} = \begin{bmatrix} 3 \\ 2 \\ 4 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 4 \\ -2 \\ -1 \end{bmatrix}$.

Norms: $\|\mathbf{u}\| = \sqrt{3^2 + 2^2 + 4^2} = \sqrt{9 + 4 + 16} = \sqrt{29}$, and $\|\mathbf{v}\| = \sqrt{16 + 4 + 1} = \sqrt{21}$.

Inner product: $\mathbf{u}^T \mathbf{v} = (3)(4) + (2)(-2) + (4)(-1) = 12 - 4 - 4 = 4$.

Angle: $\cos \theta = \frac{4}{\sqrt{29}\sqrt{21}} \approx 0.162$, so $\theta \approx 80.7^\circ$. Since $\mathbf{u}^T \mathbf{v} = 4 \neq 0$, the vectors are *not* orthogonal, but they are nearly so.

A unit vector in the direction of $\mathbf{u}$: $\hat{\mathbf{u}} = \mathbf{u}/\|\mathbf{u}\| = \frac{1}{\sqrt{29}}[3, 2, 4]^T$. Check: $\|\hat{\mathbf{u}}\| = \frac{1}{\sqrt{29}}\sqrt{29} = 1$. ✓

2.3 The Outer Product: The Opposite Move

The inner product takes two vectors and *collapses* them into a single number ($\mathbf{u}^T \mathbf{v}$: a $1 \times n$ times an $n \times 1$ gives 1×1). Flip the order of the transpose and the dimensions explode instead: the **outer product** $\mathbf{u}\mathbf{v}^T$ ($n \times 1$ times $1 \times m$) is a full $n \times m$ *matrix*.

Example 2.2: Expanding an outer product

$$\mathbf{u} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \quad \Rightarrow \quad \mathbf{u}\mathbf{v}^T = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \begin{bmatrix} 2 & 4 \end{bmatrix} = \begin{bmatrix} 3(2) & 3(4) \\ 1(2) & 1(4) \end{bmatrix} = \begin{bmatrix} 6 & 12 \\ 2 & 4 \end{bmatrix}.$$

Notice that every *column* of the result is a multiple of $\mathbf{u}$ (the columns are $2\mathbf{u}$ and $4\mathbf{u}$), and every *row* is a multiple of $\mathbf{v}^T$. An outer product is the simplest possible matrix—it carries only one direction of information. We will call such matrices *rank one*, and they are the atoms from which the SVD (Week 5) rebuilds every matrix.

2.4 Rules for Transposes and Quadratic Forms

Two algebra rules get used on nearly every page from here on:

$$(AB)^T = B^T A^T \quad (\text{note the reversal of order!}), \quad (A^T)^T = A.$$

A matrix with $A^T = A$ is called **symmetric**; its entries mirror across the diagonal ($a_{ij} = a_{ji}$). Symmetric matrices will turn out to be the best-behaved matrices in this book.

A **quadratic form** pairs a symmetric matrix A with a vector to produce a number:

$$q(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j.$$

For example, with $A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$,

$$\mathbf{x}^T A \mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} x_1 + 2x_2 \\ 2x_1 + 3x_2 \end{bmatrix} = x_1^2 + 4x_1x_2 + 3x_2^2.$$

Quadratic forms are how matrices encode *curvature*; they will reappear as the Hessian in Part II.

A quadratic form can be written with any square matrix, but only its symmetric part contributes: $\mathbf{x}^T A \mathbf{x} = \mathbf{x}^T ((A + A^T)/2) \mathbf{x}$.

2.5 Linear Independence, Basis, and Coordinates

Definition: Linear independence

Vectors $\mathbf{a}_1, \dots, \mathbf{a}_n$ are **linearly independent** if the only way to combine them into the zero vector is with all-zero weights:

$$c_1 \mathbf{a}_1 + c_2 \mathbf{a}_2 + \dots + c_n \mathbf{a}_n = \mathbf{0} \implies c_1 = c_2 = \dots = c_n = 0.$$

Equivalently: no vector in the list is a linear combination of the others. In matrix language, the columns of $A = [\mathbf{a}_1 \dots \mathbf{a}_n]$ are independent exactly when $A\mathbf{x} = \mathbf{0}$ forces $\mathbf{x} = \mathbf{0}$.

Example 2.3: Checking independence

Are $\mathbf{a}_1 = \begin{bmatrix} 3 \\ 5 \\ 0 \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} 4 \\ 2 \\ 6 \end{bmatrix}$ independent?

Suppose $c_1 \mathbf{a}_1 + c_2 \mathbf{a}_2 = \mathbf{0}$. Reading the three rows:

$$3c_1 + 4c_2 = 0$$

$$5c_1 + 2c_2 = 0$$

$$0c_1 + 6c_2 = 0$$

The third equation forces $c_2 = \mathbf{0}$; substituting into the first gives $3c_1 = 0$, so $c_1 = \mathbf{0}$. Only the trivial combination works: **independent**. (By contrast, $\mathbf{u} = [1, 2, -3]^T$ and $\mathbf{v} = [-2, -4, 6]^T$ satisfy $\mathbf{v} = -2\mathbf{u}$, so they are *dependent*.)

Definition: Basis and coordinates

If $\mathbf{a}_1, \dots, \mathbf{a}_n$ are linearly independent, they form a **basis** of $V = \text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_n\}$. Every $\mathbf{b} \in V$ can then be written

$$\mathbf{b} = x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n$$

for *exactly one* choice of weights $x_1, \dots, x_n$, called the **coordinates** of $\mathbf{b}$ in this basis.

Why unique? Suppose two coordinate lists both worked: $\mathbf{b} = \sum x_i\mathbf{a}_i = \sum y_i\mathbf{a}_i$. Subtracting, $\sum (x_i - y_i)\mathbf{a}_i = \mathbf{0}$. Independence then forces every $x_i - y_i = 0$, i.e. $x_i = y_i$. This three-line argument is your first real proof of the course—and it is the reason “solve $A\mathbf{x} = \mathbf{b}$ ” has a single answer when the columns of A are independent.

2.6 The Matrix Inverse and the Determinant

When the columns of a square matrix $A \in \mathbb{R}^{n \times n}$ are independent, the equation $A\mathbf{x} = \mathbf{y}$ has exactly one solution for every $\mathbf{y}$, and we can write it as $\mathbf{x} = A^{-1}\mathbf{y}$, where A^{-1} is the **inverse matrix**: the unique matrix satisfying

$$A^{-1}A = AA^{-1} = I.$$

For 2×2 matrices there is a closed formula. If $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, then

$$\det A = ad - bc, \quad A^{-1} = \frac{1}{\det A} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad (\text{exists} \iff \det A \neq 0).$$

Geometrically, $|\det A|$ is the *area* of the parallelogram formed by the columns of A (volume of the parallelepiped in higher dimensions). $\det A = 0$ means the columns are squashed onto a line—area zero—which is exactly the dependent case where no inverse can exist.

2.7 Machine Learning Application

ML Link: Cosine Similarity and Word Embeddings

Large language models convert every word (or word-piece) into a vector called an **embedding**, typically with hundreds or thousands of entries. Words are placed so that geometry mirrors meaning. To measure how related two words are, the model computes the **cosine similarity**

$$\cos \theta = \frac{\mathbf{u}^T \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \in [-1, 1].$$

“Cat” and “dog” might score 0.8 (small angle); “cat” and “carburetor” near 0 (nearly orthogonal—no shared meaning). The same trick powers semantic search: your query is embedded as a vector and the search engine returns documents whose vectors make the smallest angle with it. Everything rests on the Week 2 angle formula.

2.8 End of Chapter Exercises

Week 2 Exercises

2.1 Let $\mathbf{u} = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$. Compute the inner product $\mathbf{u}^T \mathbf{v}$, the outer product $\mathbf{u} \mathbf{v}^T$, and state the dimensions of each result.

2.2 Calculate the cosine of the angle between $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}$. Are the vectors closer to parallel or to orthogonal?

2.3 Let $M = \begin{bmatrix} 1 & 0 & 5 \\ -2 & 4 & 3 \end{bmatrix}$. Find M^T , state its dimensions, and compute MM^T . Verify that MM^T is symmetric.

2.4 Determine whether the vectors $\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ -3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} -2 \\ -4 \\ 5 \end{bmatrix}$ are linearly independent. Justify your answer clearly.

2.5 Write out the quadratic form $q(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$ for $A = \begin{bmatrix} 2 & 1 \\ 1 & 5 \end{bmatrix}$ as a polynomial in x_1, x_2 .

2.6 Compute $\det A$ and A^{-1} for $A = \begin{bmatrix} 2 & 5 \\ 1 & 3 \end{bmatrix}$, and verify $AA^{-1} = I$.

Exam Practice

Determine if the following set of vectors is linearly independent. Justify your answer clearly.

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ -3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} -2 \\ -4 \\ 5 \end{bmatrix}.$$

Hint for self-checking: compare with Exercise 2.4—this exam problem is solved by exactly the strategy of Example 2.3. Watch the third entries: they are *not* in the ratio -2 .

Chapter 3

Week 3: Orthogonality and Projections

3.1 The Intuition: Casting Shadows to Remove Noise

Real datasets are messy and redundant. Imagine tracking a car driving down a straight road using a slightly noisy GPS: the recorded positions scatter around the road instead of lying on it. The “signal” is the position *along* the road; the “noise” is the wobble *across* it. If we could split every measurement into (signal along the road) + (noise perpendicular to it), we could keep the first part and discard the second.

That splitting is exactly what an **orthogonal projection** does: it casts the shadow of a vector onto a chosen direction (or subspace), and the leftover part is guaranteed to be perpendicular to it. Projections are the single most-used geometric operation in data science: least squares (Week 9), PCA (Week 6), and noise filtering are all projections.

3.2 Orthogonal Sets and Why They Are Wonderful

Definition: Orthogonal and orthonormal sets

A set of nonzero vectors $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ is **orthogonal** if every pair is perpendicular: $\mathbf{u}_i^T \mathbf{u}_j = 0$ whenever $i \neq j$. If additionally each vector has unit length ($\|\mathbf{u}_i\| = 1$), the set is **orthonormal**. A square matrix U whose columns are orthonormal is called an **orthogonal matrix** and satisfies the beautiful identity

$$U^T U = I \quad \Longleftrightarrow \quad U^{-1} = U^T.$$

(Inverting such a matrix is free: just transpose it.)

Fact 1: Orthogonal sets are automatically independent. Suppose $c_1 \mathbf{u}_1 + \dots + c_r \mathbf{u}_r = \mathbf{0}$. Take the inner product of both sides with $\mathbf{u}_1$:

$$c_1 \underbrace{\mathbf{u}_1^T \mathbf{u}_1}_{\|\mathbf{u}_1\|^2 \neq 0} + c_2 \underbrace{\mathbf{u}_1^T \mathbf{u}_2}_0 + \dots + c_r \underbrace{\mathbf{u}_1^T \mathbf{u}_r}_0 = 0 \implies c_1 = 0.$$

Repeating with $\mathbf{u}_2, \dots, \mathbf{u}_r$ kills every coefficient. Done.

Fact 2: Coordinates in an orthogonal basis are almost free. Ordinarily, writing $\mathbf{y} = x_1\mathbf{u}_1 + \cdots + x_r\mathbf{u}_r$ requires solving a linear system. But if the basis is orthogonal, hit both sides with $\mathbf{u}_j^T$; all cross terms vanish, leaving

$$\mathbf{u}_j^T \mathbf{y} = x_j \mathbf{u}_j^T \mathbf{u}_j \quad \implies \quad x_j = \frac{\mathbf{u}_j^T \mathbf{y}}{\mathbf{u}_j^T \mathbf{u}_j}$$

Each coordinate is a single division—no system to solve. This formula is the workhorse of the rest of the course.

Tall matrices with orthonormal columns. An orthogonal matrix is usually defined to be square, but the idea of orthogonality also applies to rectangular matrices. Suppose that $U \in \mathbb{R}^{m \times n}$ with $m \geq n$, and that the columns of U are orthonormal. Then

$$U^T U = I_n.$$

Such a matrix is sometimes called *semi-orthogonal*. When $m > n$, however, we do not have $UU^T = I_m$. Indeed, UU^T is the orthogonal projection onto the column space of U . It acts as the identity on vectors in that subspace, but removes components perpendicular to it.

Multiplication by U embeds $\mathbb{R}^n$ into the higher-dimensional space $\mathbb{R}^m$ without changing lengths. For any $\mathbf{x} \in \mathbb{R}^n$,

$$\|U\mathbf{x}\|^2 = (U\mathbf{x})^T (U\mathbf{x}) = \mathbf{x}^T U^T U \mathbf{x} = \mathbf{x}^T \mathbf{x} = \|\mathbf{x}\|^2.$$

Thus, although U may be tall, it still preserves the norm of every vector in its input space.

It also preserves inner products:

$$(U\mathbf{x})^T (U\mathbf{y}) = \mathbf{x}^T U^T U \mathbf{y} = \mathbf{x}^T \mathbf{y}.$$

Consequently, the angle between any two nonzero vectors is unchanged when both are multiplied by U . Geometrically, U places a copy of $\mathbb{R}^n$ inside $\mathbb{R}^m$ without stretching or distorting it.

Example 3.1: Coordinates in an orthogonal basis

The vectors $\mathbf{u}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{u}_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$ are orthogonal (check: $\mathbf{u}_1^T \mathbf{u}_2 = -2 + 2 = 0$). Write $\mathbf{y} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}$ in this basis.

$$x_1 = \frac{\mathbf{u}_1^T \mathbf{y}}{\mathbf{u}_1^T \mathbf{u}_1} = \frac{(2)(9) + (1)(2)}{2^2 + 1^2} = \frac{20}{5} = 4,$$

$$x_2 = \frac{\mathbf{u}_2^T \mathbf{y}}{\mathbf{u}_2^T \mathbf{u}_2} = \frac{(-1)(9) + (2)(2)}{(-1)^2 + 2^2} = \frac{-5}{5} = -1.$$

Verification: $4 \begin{bmatrix} 2 \\ 1 \end{bmatrix} - 1 \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 8 + 1 \\ 4 - 2 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}. \checkmark$

3.3 Orthogonal Projection onto a Line

Take a single nonzero direction $\mathbf{u}$ and any vector $\mathbf{y}$. We want to split

$$\mathbf{y} = \underbrace{\hat{\mathbf{y}}}_{\text{shadow, parallel to } \mathbf{u}} + \underbrace{\mathbf{e}}_{\text{leftover, perpendicular to } \mathbf{u}}.$$

Derivation. Since $\hat{\mathbf{y}}$ lies along $\mathbf{u}$, write $\hat{\mathbf{y}} = c\mathbf{u}$ for an unknown scalar c . Perpendicularity of the leftover means

$$0 = \mathbf{u}^T \mathbf{e} = \mathbf{u}^T (\mathbf{y} - c\mathbf{u}) = \mathbf{u}^T \mathbf{y} - c \mathbf{u}^T \mathbf{u} \implies c = \frac{\mathbf{u}^T \mathbf{y}}{\mathbf{u}^T \mathbf{u}}.$$

That single line of algebra gives the projection formula:

Key Idea: The projection formula

$$\text{proj}_{\mathbf{u}} \mathbf{y} = \frac{\mathbf{u}^T \mathbf{y}}{\mathbf{u}^T \mathbf{u}} \mathbf{u}.$$

In matrix form, $\text{proj}_{\mathbf{u}} \mathbf{y} = P\mathbf{y}$ where $P = \frac{\mathbf{u}\mathbf{u}^T}{\mathbf{u}^T \mathbf{u}}$ is the (rank-one) **projection matrix**. It satisfies two signature properties:

$$P^2 = P \text{ (projecting twice changes nothing),} \quad P^T = P \text{ (symmetric).}$$

The complementary matrix $Q = I - P$ projects onto everything perpendicular to $\mathbf{u}$, and $PQ = 0$: the shadow and the leftover truly carry disjoint information.

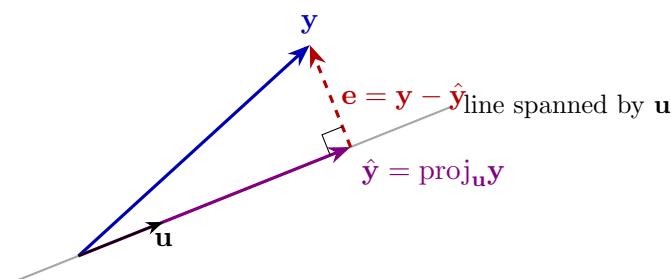


Figure 3.1: Orthogonal projection: $\mathbf{y}$ splits into its shadow $\hat{\mathbf{y}}$ on the line and a leftover $\mathbf{e}$ perpendicular to it. The shadow is the *closest* point on the line to $\mathbf{y}$.

Example 3.2: Projecting data onto a clean direction

Project the noisy measurement $\mathbf{y} = \begin{bmatrix} 7 \\ 6 \end{bmatrix}$ onto the road direction $\mathbf{u} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$.

$$\text{Numerator: } \mathbf{u}^T \mathbf{y} = (4)(\textcolor{red}{7}) + (2)(\textcolor{red}{6}) = 28 + 12 = 40,$$

$$\text{Denominator: } \mathbf{u}^T \mathbf{u} = 4^2 + 2^2 = 20,$$

$$\text{proj}_{\mathbf{u}} \mathbf{y} = \frac{40}{20} \begin{bmatrix} 4 \\ 2 \end{bmatrix} = 2 \begin{bmatrix} 4 \\ 2 \end{bmatrix} = \begin{bmatrix} 8 \\ 4 \end{bmatrix}.$$

The leftover is $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = [-1, 2]^T$, and indeed $\mathbf{u}^T \mathbf{e} = (4)(-1) + (2)(2) = 0$: the noise we removed is exactly perpendicular to the direction we kept.

If we have an orthogonal basis $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ for a whole subspace W , the projection onto W is simply the sum of the one-line projections:

$$\text{proj}_W \mathbf{y} = \frac{\mathbf{u}_1^T \mathbf{y}}{\mathbf{u}_1^T \mathbf{u}_1} \mathbf{u}_1 + \dots + \frac{\mathbf{u}_r^T \mathbf{y}}{\mathbf{u}_r^T \mathbf{u}_r} \mathbf{u}_r.$$

This is why we want orthogonal bases: projection onto a subspace decouples into r independent, one-dimensional problems.

3.4 Machine Learning Application

ML Link: Denoising and Feature Extraction

Projections appear throughout machine learning whenever we want to “keep the part of the data that matters”:

- **Denoising.** If the true signal is known to live in a subspace W (e.g. smooth audio frequencies), projecting a noisy recording onto W removes the perpendicular noise component in one matrix multiplication.
- **Removing a bias direction.** Suppose word embeddings have learned an undesirable direction $\mathbf{u}$ (researchers famously found a “gender direction” in early embeddings). Applying $Q = I - P$ to each embedding subtracts exactly the component along $\mathbf{u}$.
- **Least squares** (Week 9) will literally *be* a projection: the best-fit prediction is the projection of the data onto the span of the model’s columns.

3.5 End of Chapter Exercises

Week 3 Exercises

3.1 Let $\{\mathbf{u}_1 = [-2, 1]^T, \mathbf{u}_2 = [1, 2]^T\}$ be an orthogonal basis. Find both coefficients c_1, c_2 such that $\mathbf{y} = [9, 2]^T$ equals $c_1 \mathbf{u}_1 + c_2 \mathbf{u}_2$, using the coordinate formula (no system solving!).

3.2 Check whether the set $\{[1, 0, -1]^T, [1, \sqrt{2}, 1]^T, [1, -\sqrt{2}, 1]^T\}$ is mutually orthogonal by computing all three pairwise dot products. Then normalize each vector to produce an orthonormal set.

3.3 Let $\mathbf{u}_1 = [1, 1, 0]^T$ and $\mathbf{u}_2 = [1, -1, 0]^T$. (a) Compute the projection of $\mathbf{y} = [3, 1, 5]^T$ onto $\mathbf{u}_1$. (b) Compute the projection of the same $\mathbf{y}$ onto $\mathbf{u}_2$. (c) Add the two projections and explain geometrically why the sum is missing the third entry of $\mathbf{y}$.

3.4 For $\mathbf{u} = [4, 2]^T$, write out the 2×2 projection matrix $P = \mathbf{u}\mathbf{u}^T/(\mathbf{u}^T \mathbf{u})$ explicitly, and verify by direct multiplication that $P^2 = P$.

3.5 Show that the vectors $\mathbf{v}_1 = [1, 2, 3, 4]^T$, $\mathbf{v}_2 = [0, 1, 5, 6]^T$, $\mathbf{v}_3 = [0, 0, 2, 1]^T$ are linearly independent. (*Hint: look at the pattern of leading zeros.*)

Exam Practice

Let $\mathbf{u}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$ and $\mathbf{u}_2 = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}$.

(a) Compute the projection of the vector $\mathbf{y} = \begin{bmatrix} 3 \\ 1 \\ 5 \end{bmatrix}$ onto $\mathbf{u}_1$.

(b) Compute the projection of the same vector $\mathbf{y}$ onto $\mathbf{u}_2$.

(Exercise 3.3 extends this exam problem with a part (c) asking what the sum of the two projections misses—try that too.)

Chapter 4

Week 4: Gram–Schmidt, QR, and Eigen-Decomposition

4.1 The Intuition: Manufacturing Good Axes, Finding Natural Ones

Week 3 showed that orthogonal bases make every computation easy. Two questions remain. First: if our data hands us a messy, non-orthogonal basis, can we *manufacture* an orthogonal one from it? Yes—that is the **Gram–Schmidt process**, and it works by repeatedly subtracting projections. Second: does a matrix come with its own *natural* axes built in? Again yes—those are the **eigenvectors**, the special directions a matrix does not rotate but merely stretches.

4.2 The Gram–Schmidt Process

Given independent vectors $\mathbf{a}_1, \dots, \mathbf{a}_n$, we build orthogonal vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ spanning the same space, one at a time. The rule for each new vector: *take the next ingredient and subtract off its shadow on everything built so far.*

$$\begin{aligned}\mathbf{v}_1 &= \mathbf{a}_1, \\ \mathbf{v}_2 &= \mathbf{a}_2 - \frac{\mathbf{v}_1^T \mathbf{a}_2}{\mathbf{v}_1^T \mathbf{v}_1} \mathbf{v}_1, \\ \mathbf{v}_3 &= \mathbf{a}_3 - \frac{\mathbf{v}_1^T \mathbf{a}_3}{\mathbf{v}_1^T \mathbf{v}_1} \mathbf{v}_1 - \frac{\mathbf{v}_2^T \mathbf{a}_3}{\mathbf{v}_2^T \mathbf{v}_2} \mathbf{v}_2, \\ &\vdots\end{aligned}$$

Why does it work? After the subtraction, the new $\mathbf{v}_k$ is exactly the “leftover” $\mathbf{e}$ from Week 3’s projection picture—and we proved the leftover is perpendicular to everything we projected onto. Normalizing each $\mathbf{v}_i$ to $\mathbf{q}_i = \mathbf{v}_i / \|\mathbf{v}_i\|$ yields an *orthonormal* basis.

Example 4.1: Gram–Schmidt in $\mathbb{R}^3$

Orthogonalize $\mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix}$, then normalize.

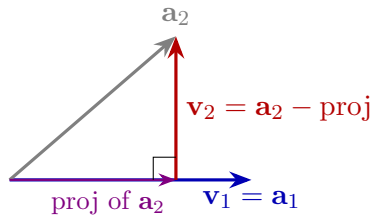


Figure 4.1: One step of Gram–Schmidt: subtracting the shadow of $\mathbf{a}_2$ on $\mathbf{v}_1$ leaves a piece $\mathbf{v}_2$ that is exactly perpendicular to $\mathbf{v}_1$.

Step 1. $\mathbf{v}_1 = \mathbf{a}_1 = [1, 2, 2]^T$, with $\mathbf{v}_1^T \mathbf{v}_1 = 1 + 4 + 4 = \mathbf{9}$.

Step 2. Compute the shadow coefficient:

$$\frac{\mathbf{v}_1^T \mathbf{a}_2}{\mathbf{v}_1^T \mathbf{v}_1} = \frac{(1)(1) + (2)(3) + (2)(1)}{9} = \frac{\mathbf{9}}{\mathbf{9}} = 1.$$

Subtract the shadow:

$$\mathbf{v}_2 = \mathbf{a}_2 - 1 \cdot \mathbf{v}_1 = \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}.$$

Check orthogonality: $\mathbf{v}_1^T \mathbf{v}_2 = 0 + 2 - 2 = 0$. ✓

Step 3 (normalize). $\mathbf{q}_1 = \frac{1}{3}[1, 2, 2]^T$, and since $\|\mathbf{v}_2\| = \sqrt{2}$, $\mathbf{q}_2 = \frac{1}{\sqrt{2}}[0, 1, -1]^T$.

4.2.1 The QR Factorization

The Gram–Schmidt process does more than produce orthonormal vectors. If we keep track of every projection coefficient and normalization factor, those numbers assemble into a matrix.

Suppose

$$A = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n]$$

has independent columns. Gram–Schmidt can be written in the normalized form

$$r_{ij} = \mathbf{q}_i^T \mathbf{a}_j \quad (i < j),$$

followed by

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} r_{ij} \mathbf{q}_i, \quad r_{jj} = \|\mathbf{v}_j\|, \quad \mathbf{q}_j = \frac{\mathbf{v}_j}{r_{jj}}.$$

For $j = 1$, the sum is empty, so $\mathbf{v}_1 = \mathbf{a}_1$.

Rearranging the formula for $\mathbf{v}_j$ gives

$$\mathbf{a}_j = r_{1j} \mathbf{q}_1 + r_{2j} \mathbf{q}_2 + \cdots + r_{jj} \mathbf{q}_j.$$

Thus $\mathbf{a}_1$ uses only $\mathbf{q}_1$; $\mathbf{a}_2$ uses only $\mathbf{q}_1, \mathbf{q}_2$; and so on. Stacking these relations gives

$$\underbrace{[\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n]}_A = \underbrace{[\mathbf{q}_1 \ \mathbf{q}_2 \ \cdots \ \mathbf{q}_n]}_{Q, Q^T Q = I} \underbrace{\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & r_{nn} \end{bmatrix}}_{R \text{ upper triangular}}.$$

This is the **QR factorization**

$$\boxed{A = QR}.$$

The columns of Q are the orthonormal vectors produced by Gram-Schmidt, while R records the coefficients needed to reconstruct the original columns.

Returning to Example 4.1. There we began with

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix},$$

and obtained

$$\mathbf{q}_1 = \frac{1}{3} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}.$$

The coefficients are

$$r_{11} = \|\mathbf{a}_1\| = 3, \quad r_{12} = \mathbf{q}_1^T \mathbf{a}_2 = 3, \quad r_{22} = \sqrt{2}.$$

Therefore

$$\mathbf{a}_1 = 3\mathbf{q}_1, \quad \mathbf{a}_2 = 3\mathbf{q}_1 + \sqrt{2}\mathbf{q}_2.$$

In matrix form,

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 2 & 3 \\ 2 & 1 \end{bmatrix}}_A = \underbrace{\begin{bmatrix} \frac{1}{3} & 0 \\ \frac{2}{3} & \frac{1}{\sqrt{2}} \\ \frac{2}{3} & -\frac{1}{\sqrt{2}} \end{bmatrix}}_Q \underbrace{\begin{bmatrix} 3 & 3 \\ 0 & \sqrt{2} \end{bmatrix}}_R.$$

Thus the Gram-Schmidt calculation in Example 4.1 was already computing a QR factorization. Since A is a tall 3×2 matrix, Q is also tall: it satisfies $Q^T Q = I_2$, although $Q Q^T \neq I_3$. This is called the *thin* or *reduced* QR factorization.

Example 4.1 continued: A three-column QR factorization

Keep the first two vectors from Example 4.1 and add

$$\mathbf{a}_3 = \begin{bmatrix} -3 \\ 4 \\ 2 \end{bmatrix}.$$

We already know $\mathbf{q}_1$ and $\mathbf{q}_2$. To orthogonalize the third vector, first compute its projection coefficients:

$$r_{13} = \mathbf{q}_1^T \mathbf{a}_3 = \frac{-3 + 8 + 4}{3} = 3,$$

and

$$r_{23} = \mathbf{q}_2^T \mathbf{a}_3 = \frac{4 - 2}{\sqrt{2}} = \sqrt{2}.$$

Subtract both projections:

$$\begin{aligned} \mathbf{v}_3 &= \mathbf{a}_3 - r_{13}\mathbf{q}_1 - r_{23}\mathbf{q}_2 \\ &= \begin{bmatrix} -3 \\ 4 \\ 2 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -4 \\ 1 \\ 1 \end{bmatrix}. \end{aligned}$$

Therefore

$$r_{33} = \|\mathbf{v}_3\| = \sqrt{18} = 3\sqrt{2}, \quad \mathbf{q}_3 = \frac{1}{3\sqrt{2}} \begin{bmatrix} -4 \\ 1 \\ 1 \end{bmatrix}.$$

The three orthonormal vectors form

$$Q = \begin{bmatrix} \frac{1}{3} & 0 & -\frac{4}{3\sqrt{2}} \\ \frac{2}{3} & \frac{1}{\sqrt{2}} & \frac{1}{3\sqrt{2}} \\ \frac{2}{3} & -\frac{1}{\sqrt{2}} & \frac{1}{3\sqrt{2}} \end{bmatrix},$$

and all the projection and normalization coefficients form

$$R = \begin{bmatrix} 3 & 3 & 3 \\ 0 & \sqrt{2} & \sqrt{2} \\ 0 & 0 & 3\sqrt{2} \end{bmatrix}.$$

Consequently,

$$A = \begin{bmatrix} 1 & 1 & -3 \\ 2 & 3 & 4 \\ 2 & 1 & 2 \end{bmatrix} = QR.$$

To see exactly what this matrix multiplication means, read it one column at a time:

$$\mathbf{a}_1 = 3\mathbf{q}_1,$$

$$\mathbf{a}_2 = 3\mathbf{q}_1 + \sqrt{2}\mathbf{q}_2,$$

$$\mathbf{a}_3 = 3\mathbf{q}_1 + \sqrt{2}\mathbf{q}_2 + 3\sqrt{2}\mathbf{q}_3.$$

These are precisely the relations produced during Gram-Schmidt. The zeros below the diagonal of R appear because $\mathbf{a}_j$ is constructed using only the first j orthonormal vectors.

In Week 9, QR will let us solve least-squares problems without forming the potentially ill-conditioned matrix $A^T A$.

4.3 Eigenvalues and Eigenvectors: A Matrix's Natural Axes

Definition: Eigenvalue and eigenvector

For a square matrix A , a nonzero vector $\mathbf{x}$ is an **eigenvector** with **eigenvalue** λ if

$$A\mathbf{x} = \lambda\mathbf{x}.$$

In words: A does not rotate $\mathbf{x}$; it only stretches it by the factor λ (flipping it if $\lambda < 0$). Any nonzero multiple $c\mathbf{x}$ is then also an eigenvector for the same λ .

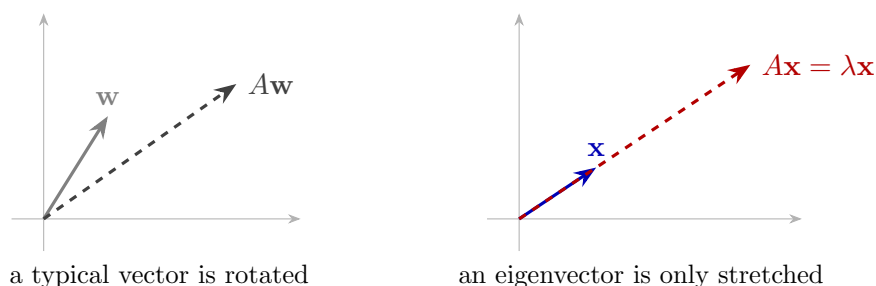


Figure 4.2: Eigenvectors are the directions a matrix leaves unrotated.

How to find them. Rewrite $A\mathbf{x} = \lambda\mathbf{x}$ as $(A - \lambda I)\mathbf{x} = \mathbf{0}$. We need a *nonzero* solution $\mathbf{x}$, which (by Week 2) is possible only when the columns of $A - \lambda I$ are dependent, i.e. when

$$\boxed{\det(A - \lambda I) = 0} \quad (\text{the characteristic equation}).$$

For an $n \times n$ matrix this is a degree- n polynomial in λ ; its roots are the eigenvalues. For each root, solve $(A - \lambda I)\mathbf{x} = \mathbf{0}$ to get the eigenvector(s). In general, the eigenvalues can be complex even if all the entries are real numbers.

Example 4.2: Eigenvalues and eigenvectors, start to finish

Find the eigen-pairs of $B = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$.

Step 1: Characteristic equation.

$$\begin{aligned} \det(B - \lambda I) &= \det \begin{bmatrix} 4 - \lambda & 1 \\ 2 & 3 - \lambda \end{bmatrix} = (4 - \lambda)(3 - \lambda) - (1)(2) \\ &= \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2) = 0, \end{aligned}$$

so $\lambda_1 = 5$ and $\lambda_2 = 2$.

Step 2: Eigenvector for $\lambda_1 = 5$. Solve $(B - 5I)\mathbf{x} = \mathbf{0}$:

$$\begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies x_2 = x_1 \implies \mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

(Both rows give the same equation—the rows become redundant, and it is your sign that the eigenvalue is correct.)

Step 3: Eigenvector for $\lambda_2 = 2$. Solve $(B - 2I)\mathbf{x} = \mathbf{0}$:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{0} \implies x_2 = -2x_1 \implies \mathbf{x}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Sanity check: $B\mathbf{x}_1 = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5\mathbf{x}_1$. ✓

Left eigenvectors. The eigenvectors above multiply A from the right, $A\mathbf{x} = \lambda\mathbf{x}$, and are sometimes called *right* eigenvectors. A **left eigenvector** is a row vector that reproduces itself when multiplied from the *left*:

$$\mathbf{w}^T A = \lambda \mathbf{w}^T.$$

Transposing both sides gives $A^T \mathbf{w} = \lambda \mathbf{w}$: a left eigenvector of A is nothing more than an ordinary eigenvector of A^T . Since A and A^T have the same characteristic polynomial ($\det(A - \lambda I) = \det(A^T - \lambda I)$), the *eigenvalues are identical*; only the eigenvectors change. For a symmetric matrix ($A = A^T$) left and right eigenvectors coincide. Left eigenvectors will reappear in Week 5, where the row vector $[1, 1, \dots, 1]$ is a left eigenvector of every column-stochastic matrix.

4.3.1 Diagonalization

Collect the eigenvectors as the columns of $P = [\mathbf{x}_1 \ \mathbf{x}_2 \ \dots]$ and the eigenvalues into a diagonal matrix $D = \text{diag}(\lambda_1, \lambda_2, \dots)$. The n separate equations $A\mathbf{x}_i = \lambda_i \mathbf{x}_i$ pack into one:

$$AP = PD \implies A = PDP^{-1} \quad (\text{the eigendecomposition}),$$

provided that the columns of P (the eigenvectors) are linearly independent.

This says: in the eigenvector coordinate system, A is merely a diagonal matrix—independent stretching along each natural axis. One immediate payoff: powers become trivial,

$$A^k = (PDP^{-1})(PDP^{-1}) \dots (PDP^{-1}) = PD^k P^{-1},$$

and D^k just raises each diagonal entry to the k -th power. Computing A^{100} costs roughly the same effort as computing A^2 , since the eigenvectors can be reused. (This is the engine behind the Markov chain analysis of Week 5.)

There is another useful way to write the eigendecomposition. Let the rows of P^{-1} be

$$P^{-1} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_n^T \end{bmatrix}.$$

Since $P^{-1}P = I$, these vectors satisfy

$$\mathbf{w}_i^T \mathbf{x}_j = \delta_{ij},$$

where $\delta_{ij} = 1$ when $i = j$ and 0 otherwise. Thus $\mathbf{w}_i$ extracts the coefficient associated with the eigenvector $\mathbf{x}_i$. Expanding PDP^{-1} gives the **rank-one form**

$$A = \sum_{i=1}^n \lambda_i \mathbf{x}_i \mathbf{w}_i^T.$$

Each matrix $\mathbf{x}_i \mathbf{w}_i^T$ has rank one. Consequently, for any vector $\mathbf{b}$,

$$A\mathbf{b} = \sum_{i=1}^n \lambda_i \mathbf{x}_i (\mathbf{w}_i^T \mathbf{b}).$$

In words, $\mathbf{w}_i^T \mathbf{b}$ measures how much of the i th eigenvector is present, λ_i scales that component, and $\mathbf{x}_i$ places it back in the corresponding direction.

Common Pitfall: Not every matrix is diagonalizable

The matrix $A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$ has the repeated eigenvalue $\lambda = 3$ but only *one* independent eigenvector direction, $[1, 0]^T$. With too few eigenvectors to form P , no diagonalization exists. Fortunately, the matrices that matter most in data science avoid this problem entirely, as we now explain.

4.3.2 Symmetric Matrices: The Spectral Theorem

Key Idea: The Spectral Theorem

If A is **symmetric** ($A^T = A$), then:

1. all eigenvalues are *real*,
2. eigenvectors for distinct eigenvalues are *orthogonal*,
3. A is always diagonalizable, with an *orthonormal* eigenvector basis: $A = PDP^T$ where $P^T P = I$.

Expanding column by column gives the elegant rank-one (outer-product!) sum

$$A = \sum_{i=1}^n \lambda_i \mathbf{v}_i \mathbf{v}_i^T.$$

Covariance matrices, Hessians, Gram matrices $A^T A$ —every star of the remaining chapters is symmetric, so the spectral theorem will always be available to us.

4.4 Machine Learning Application

ML Link: Why Eigenvectors Run Data Science

Principal Component Analysis (preview of Week 6). A dataset of images with 10,000 pixels lives in $\mathbb{R}^{10000}$, but real photos do not fill that space—they hug a much lower-dimensional region. The eigenvectors of the dataset’s covariance matrix point along the directions of greatest variation, and the eigenvalues measure how much variation each direction carries. Keeping the top few eigenvectors compresses the data enormously while losing almost nothing.

Stability of training. In Part III we will prove that whether gradient descent converges—and how fast—is decided entirely by the eigenvalues of a Hessian matrix. Eigenvalues are

not a side topic: the central quantities of Part III will be built from them.

4.5 End of Chapter Exercises

Week 4 Exercises

4.1 Apply Gram–Schmidt to $\mathbf{x}_1 = [1, 1, 0, 0]^T$, $\mathbf{x}_2 = [2, 0, 2, 0]^T$, $\mathbf{x}_3 = [0, 0, 3, 3]^T$ to produce an orthogonal set $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$. Verify all three pairwise dot products are zero.

4.2 Apply Gram–Schmidt to $\mathbf{a}_1 = [1, 2, 2]^T$, $\mathbf{a}_2 = [1, 3, 1]^T$ and then normalize to obtain an orthonormal basis $\{\mathbf{q}_1, \mathbf{q}_2\}$.

4.3 Find the eigenvalues of $A = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}$, then find one eigenvector for each eigenvalue.

4.4 Using the eigenvalues $\lambda_1 = 5$, $\lambda_2 = 2$ from Example 4.2, write down P and D for $B = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$ and verify $BP = PD$ by direct multiplication.

4.5 The matrix $S = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ is symmetric. Find its eigenvalues and eigenvectors, check that the eigenvectors are orthogonal, and write $S = \lambda_1 \mathbf{v}_1 \mathbf{v}_1^T + \lambda_2 \mathbf{v}_2 \mathbf{v}_2^T$ with unit eigenvectors.

4.6 Compute the inverse of $A = \begin{bmatrix} 2 & 5 \\ -1 & 3 \end{bmatrix}$ using the 2×2 formula.

Exam Practice

Problem 3 (25 points). Consider the matrix $A = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}$.

- (a) Find the eigenvalues of A .
- (b) Find one eigenvector corresponding to each eigenvalue.
- (c) Find the two left eigenvectors.

(Parts (a)–(b) are Exercise 4.3; for part (c), recall that the left eigenvectors of A are the eigenvectors of A^T , with the same eigenvalues.)

Problem 5 (25 points). Apply the Gram–Schmidt process to the basis $\{\mathbf{a}_1, \mathbf{a}_2\}$ to produce an orthogonal basis $\{\mathbf{v}_1, \mathbf{v}_2\}$, where

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix}.$$

After finding $\mathbf{v}_1$ and $\mathbf{v}_2$, normalize them to find an orthonormal basis $\{\mathbf{q}_1, \mathbf{q}_2\}$. *(Worked in full as Example 4.1—attempt it from a blank page first.)*

Chapter 5

Week 5: Eigenvalues in Action—Graphs, Markov Chains, and the SVD

5.1 The Intuition: Networks and Equilibrium

Real-world systems are often depicted as networks: web pages linked by hyperlinks, people linked by friendships, neurons linked by synapses. This week we represent networks with matrices, watch dynamics unfold on them through repeated matrix multiplication, and discover that the long-run behavior of the whole system is controlled by a single eigenvector. We then meet the singular value decomposition, which extends eigen-analysis to *any* rectangular data matrix and will carry most of the applied weight of this course.

5.2 Matrices from Graphs

A **graph** consists of nodes and edges. Three matrices summarize it:

- the **adjacency matrix** A : $a_{ij} = 1$ if node i is connected to node j , else 0;
- the **degree matrix** D : diagonal, with d_{ii} equal to the number of connections of node i ;
- the **graph Laplacian** $L = D - A$. For an *undirected* graph, L is symmetric, so the spectral theorem of Week 4 applies. Its smallest eigenvalue is always 0, with eigenvector $[1, 1, \dots, 1]^T$; the eigenvector associated with the *second*-smallest eigenvalue, called the **Fiedler vector**, is often used in practice to split the graph into two clusters (nodes with entries of the same sign are grouped together).

This description assumes that the graph is connected. If the graph has $r > 1$ connected components, then 0 is an eigenvalue of multiplicity r , and the nullspace of L is spanned by the indicator vectors of the components. Thus $\lambda_2 = 0$ when the graph is disconnected, and the corresponding “second-smallest eigenvector” is not unique; the usual Fiedler-vector interpretation applies only after restricting attention to a connected component.

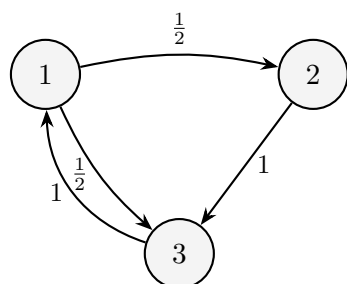
5.3 Markov Chains and PageRank

Now make the network dynamic: imagine a “random surfer” on a tiny web of 3 pages, who at every step clicks one of the outgoing links of the current page, uniformly at random.

Definition: Column-stochastic transition matrix

The matrix P with $P_{ij} = \Pr(\text{move to page } i \mid \text{currently at page } j)$. Every column sums to 1 (the surfer must go *somewhere*), and all entries are ≥ 0 . If $\mathbf{x}_k$ is the vector of probabilities of being at each page after k clicks, then

$$\mathbf{x}_{k+1} = P\mathbf{x}_k \quad \implies \quad \mathbf{x}_k = P^k \mathbf{x}_0.$$



$$P = \begin{bmatrix} 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 1 & 0 \end{bmatrix}$$

column j = where you go *from* page j

Figure 5.1: A three-page web. Page 1 links to pages 2 and 3 (probability $\frac{1}{2}$ each); page 2 links only to 3; page 3 links only to 1. Each column of P sums to 1.

The equilibrium question. As the surfer clicks forever, do the probabilities settle down? If they do, the limiting vector $\mathbf{x}_{\text{eq}}$ must satisfy

$$P\mathbf{x}_{\text{eq}} = \mathbf{x}_{\text{eq}}.$$

Look closely—this is an eigenvalue equation $P\mathbf{x} = \lambda\mathbf{x}$ with $\lambda = 1$! A column-stochastic matrix always has spectral radius 1, and 1 is an eigenvalue (here is a one-line reason: every column of P sums to 1, so the row vector $[1, \dots, 1]$ is a *left* eigenvector with eigenvalue 1—Week 4’s notion earning its keep). A fundamental theorem (Perron–Frobenius) adds that the corresponding eigenvector can be chosen with nonnegative entries. Two more conditions are needed before we may conclude that the iteration $\mathbf{x}_k = P^k \mathbf{x}_0$ actually *converges* to it: the chain must be **irreducible** (every page reachable from every other by following links) and **aperiodic** (the surfer must not be locked into a deterministic cycle). When both hold, the stationary vector is unique and $P^k \mathbf{x}_0$ converges to it from any starting distribution. That eigenvector *is* the ranking: pages with larger equilibrium probability are more “important.” This is Google’s original **PageRank** [8]. The real web graph badly fails both conditions (dead-end pages, disconnected clusters), so PageRank adds a *damping* (teleportation) step—with small probability the surfer jumps to a uniformly random page—precisely to restore them; see Exercise 5.7.

Example 5.1: Computing a PageRank vector by hand

For the web of Figure 5.1, find the steady state. We solve $(P - I)\mathbf{x} = \mathbf{0}$:

$$\begin{bmatrix} -1 & 0 & 1 \\ \frac{1}{2} & -1 & 0 \\ \frac{1}{2} & 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \mathbf{0} \implies \begin{aligned} x_3 &= x_1 \\ x_2 &= \frac{1}{2}x_1 \end{aligned}$$

(the third equation is automatic). Taking $x_1 = 2$ gives $\mathbf{x} = [2, 1, 2]^T$; normalizing so the probabilities sum to 1:

$$\mathbf{x}_{\text{eq}} = \begin{bmatrix} 0.4 \\ 0.2 \\ 0.4 \end{bmatrix}.$$

Pages 1 and 3 are tied for most important; page 2, with only one incoming link, ranks lowest. Google does this on a matrix with billions of rows. (In class, this three-page web proved to be the right size: every number can be computed by hand, yet the example contains the entire PageRank idea.)

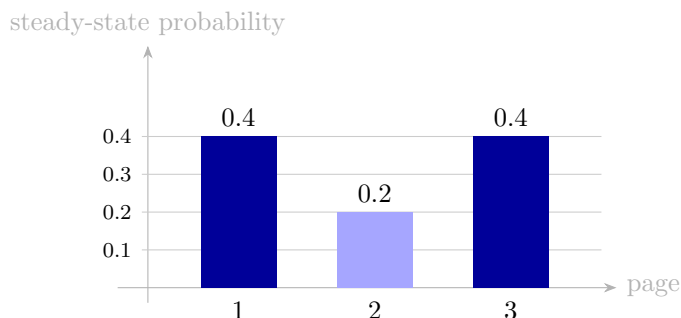


Figure 5.2: The PageRank vector of Example 5.1 as a bar chart. Pages 1 and 3 are tied for the highest rank. Page 2 ranks lower because it receives only half of page 1's probability, while page 1 receives all of page 3's probability.

5.4 The Singular Value Decomposition (SVD)

Eigen-decomposition is beautiful but has a fatal restriction: it needs a *square* matrix. Real datasets are almost never square—think 10,000 users $\times$ 500 movies. The fix is the most important theorem of applied linear algebra.

Key Idea: The SVD (one of the fundamental theorems of applied linear algebra)

Every matrix $A \in \mathbb{R}^{m \times n}$ —square or not, invertible or not—factors as

$$A = U\Sigma V^T,$$

where

- $U \in \mathbb{R}^{m \times m}$ is orthogonal; its columns $\mathbf{u}_1, \dots, \mathbf{u}_m$ are the **left singular vectors** (or-

thonormal axes for the *output* space);

- $V \in \mathbb{R}^{n \times n}$ is orthogonal; its columns $\mathbf{v}_1, \dots, \mathbf{v}_n$ are the **right singular vectors** (orthonormal axes for the *input* space);
- $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with nonnegative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$, the **singular values** (the stretch factors).

Geometrically: any matrix is (rotation/reflection V^T) then (axis-aligned stretch Σ) then (rotation/reflection U). Algebraically, the rank-one expansion

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T, \quad r = \text{rank}(A) \text{ (the number of nonzero singular values),}$$

writes the matrix as a sum of outer products of decreasing importance. (One may equally sum to $\min(m, n)$ and let the terms with $\sigma_i = 0$ contribute nothing.)

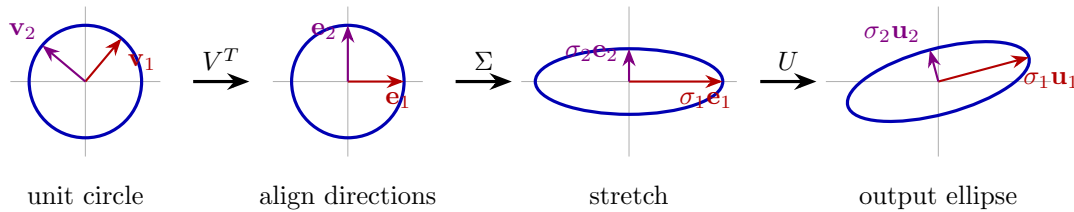


Figure 5.3: For $A = U\Sigma V^T$, the transformation V^T aligns the right singular-vector directions $\mathbf{v}_1, \mathbf{v}_2$ with the coordinate axes; the unit circle remains a circle. Next, Σ stretches these axes by σ_1 and σ_2 , producing an axis-aligned ellipse. Finally, U sends the coordinate directions to the left singular vectors $\mathbf{u}_1, \mathbf{u}_2$. Thus the image ellipse has semi-axis lengths σ_1, σ_2 and directions $\mathbf{u}_1, \mathbf{u}_2$.

5.4.1 Deriving How to Compute the SVD

How do we find singular vectors of a *rectangular* matrix? With a trick: force the matrix to become square and symmetric, then use everything from Week 4. Compute $A^T A$ and substitute the factorization:

$$\begin{aligned} A^T A &= (U\Sigma V^T)^T (U\Sigma V^T) \\ &= V\Sigma^T \underbrace{U^T U}_{=I} \Sigma V^T \\ &= V(\Sigma^T \Sigma) V^T. \end{aligned}$$

The right side is exactly an eigendecomposition of the symmetric matrix $A^T A$! Reading it off:

- the columns of V are the **eigenvectors** of $A^T A$;
- the eigenvalues of $A^T A$ are σ_i^2 , so $\sigma_i = \sqrt{\lambda_i(A^T A)}$ (these are automatically ≥ 0 —another gift of symmetry).

By the identical argument, $AA^T = U(\Sigma\Sigma^T)U^T$, so the columns of U are the eigenvectors of AA^T . For calculations by hand, we compute whichever of A^TA , AA^T is *smaller*, and then transfer to the other side using the linking relation (obtained from $AV = U\Sigma$ column by column) when $\sigma_i > 0$:

$$\boxed{\mathbf{u}_i = \frac{1}{\sigma_i}A\mathbf{v}_i, \quad \mathbf{v}_i = \frac{1}{\sigma_i}A^T\mathbf{u}_i.}$$

Numerical software generally avoids forming these normal matrices explicitly.

Example 5.2: A complete SVD from the lectures

Let $A = \begin{bmatrix} 1 & 0 & 1 \\ -2 & 1 & 0 \end{bmatrix}$ (a 2×3 matrix). Since AA^T is 2×2 while A^TA is 3×3 , we start with the smaller one.

Step 1: Form AA^T and find U and Σ .

$$AA^T = \begin{bmatrix} 1 & 0 & 1 \\ -2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ -2 & 5 \end{bmatrix}.$$

Characteristic equation: $\lambda^2 - 7\lambda + 6 = 0 \implies (\lambda - 6)(\lambda - 1) = 0$, so $\lambda_1 = \mathbf{6}$, $\lambda_2 = \mathbf{1}$ (always order largest first). The singular values are $\sigma_1 = \sqrt{6}$, $\sigma_2 = 1$.

Eigenvector for $\lambda_1 = 6$: solve $(AA^T - 6I)\mathbf{u} = \mathbf{0}$, i.e. $\begin{bmatrix} -4 & -2 \\ -2 & -1 \end{bmatrix} \mathbf{u} = \mathbf{0} \implies u_2 = -2u_1$,

normalized: $\mathbf{u}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ -2 \end{bmatrix}$. Similarly $\lambda_2 = 1$ gives $\mathbf{u}_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. (Check: $\mathbf{u}_1^T \mathbf{u}_2 = 0$, as the spectral theorem promised.)

Step 2: Transfer to the right singular vectors. Using $\mathbf{v}_i = \frac{1}{\sigma_i}A^T\mathbf{u}_i$:

$$\mathbf{v}_1 = \frac{1}{\sqrt{6}}A^T\mathbf{u}_1 = \frac{1}{\sqrt{6}} \begin{bmatrix} 1 & -2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} \mathbf{1} \\ -\mathbf{2} \end{bmatrix} = \frac{1}{\sqrt{30}} \begin{bmatrix} 5 \\ -2 \\ 1 \end{bmatrix},$$

$$\mathbf{v}_2 = \frac{1}{1}A^T\mathbf{u}_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 & -2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{2} \\ \mathbf{1} \end{bmatrix} = \frac{1}{\sqrt{5}} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}.$$

With $\Sigma = \begin{bmatrix} \sqrt{6} & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$, we have assembled $A = U\Sigma V^T$. (A third right singular vector $\mathbf{v}_3$, orthogonal to $\mathbf{v}_1, \mathbf{v}_2$, completes V ; it pairs with the singular value 0. The current decomposition is often known as the thin or reduced SVD.)

5.5 Machine Learning Application

ML Link: PageRank, Spectral Clustering, and Recommenders

The algorithms from this week run some of the largest computations on Earth.

1. PageRank. Ranking search results means computing the $\lambda = 1$ eigenvector of the web's transition matrix—exactly Example 5.1 at the scale of billions of pages.

2. Spectral clustering. The eigenvectors of a social network’s graph Laplacian $L = D - A$ reveal hidden communities with no labels or supervision at all: nodes whose entries in the second eigenvector share a sign belong together.

3. SVD recommender systems. A streaming service’s data is a huge Users $\times$ Movies ratings matrix. Computing its SVD and keeping only the largest few singular values compresses the matrix into a handful of “latent taste” directions (one $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ term might encode “affinity for 90s action movies”). Your recommendations are reconstructed from these few terms—which is why the system can guess movies you never rated.

5.6 End of Chapter Exercises

Week 5 Exercises

5.1 A 3-page internet: page 1 links to pages 2 and 3; page 2 links only to 3; page 3 links only to 1. Construct the 3×3 column-stochastic transition matrix P , and verify each column sums to 1.

5.2 For the network in 5.1, solve $P\mathbf{x} = \mathbf{x}$ to find the steady state, and rank the three pages by importance.

5.3 For $A = \begin{bmatrix} 0 & 2 \\ 0 & 0 \\ -1 & 0 \end{bmatrix}$, compute the symmetric matrix $A^T A$ (note its small size compared with AA^T).

5.4 Find the eigenvalues of the $A^T A$ from 5.3, determine the singular values σ_1, σ_2 of A , and write the 3×2 matrix Σ .

5.5 (Theoretical) A Markov chain starts at state $\mathbf{x}_0$ and evolves by $\mathbf{x}_k = P^k \mathbf{x}_0$. Using the eigendecomposition $P = QDQ^{-1}$, explain why the eigenvector with $\lambda = 1$ dominates as $k \rightarrow \infty$ when all other eigenvalues satisfy $|\lambda_i| < 1$ —and hence why solving $P\mathbf{x} = \mathbf{x}$ is a massive shortcut compared to actually multiplying P a million times.

5.6 (Debug the statement) A student says: “An irreducible column-stochastic matrix always converges to its stationary distribution.” What condition is missing? Construct a 2×2 counterexample by considering two pages that link only to each other, and compute $P^k \mathbf{x}_0$ for $\mathbf{x}_0 = [1, 0]^T$.

5.7 (PageRank damping) Google replaces the raw transition matrix P by the *damped* matrix

$$G = \alpha P + (1 - \alpha) \frac{1}{n} \mathbf{1}\mathbf{1}^T, \quad 0 < \alpha < 1 \text{ (typically } \alpha \approx 0.85),$$

where $\mathbf{1} = [1, \dots, 1]^T$. (a) Verify that G is again column-stochastic. (b) Explain in one or two sentences why every entry of G being strictly positive rules out both dead ends and periodic cycling, so that the convergence of Exercise 5.5 is guaranteed.

Exam Practice

Consider the matrix $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ -1 & 1 \end{bmatrix}$. You are given that the singular values of A are $\sigma_1 = \sqrt{3}$ and $\sigma_2 = \sqrt{2}$, with corresponding right singular vectors (the columns of V)

$$\mathbf{v}_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Using the fundamental relationship of the singular value decomposition ($AV = U\Sigma$, i.e. $\mathbf{u}_i = \frac{1}{\sigma_i} A\mathbf{v}_i$), compute the first two left singular vectors $\mathbf{u}_1$ and $\mathbf{u}_2$. (This problem reappears

as Exercise 6.5; the linking relation is boxed in Section 5.4.)

Chapter 6

Week 6: SVD at Work—Principal Component Analysis

6.1 The Intuition: Finding the Directions That Matter

Suppose you measure $M = 100$ medical features for each of $N = 10,000$ patients. One hundred dimensions is impossible to visualize and expensive to process—but the features are highly redundant (height and weight correlate; blood pressures correlate). The data cloud, viewed in $\mathbb{R}^{100}$, is a thin pancake: it varies a lot along a few directions and almost not at all along the rest.

Principal Component Analysis (PCA) finds those few important directions. It is the most widely used dimensionality-reduction method in science, and—as we will derive—it is nothing more than the SVD of the data matrix.

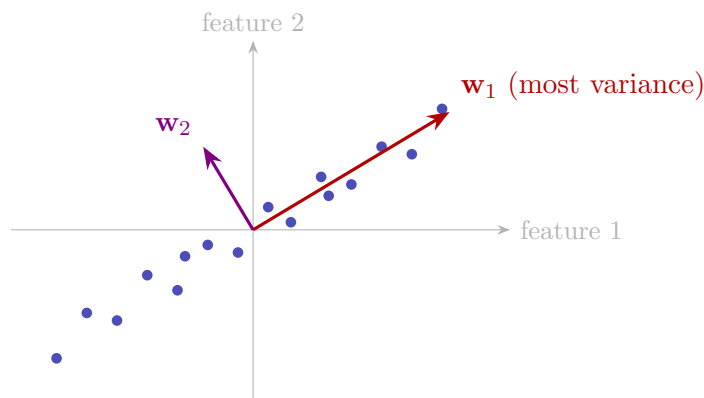


Figure 6.1: A 2D data cloud. The first principal direction $\mathbf{w}_1$ points along the cloud’s long axis; $\mathbf{w}_2$ is perpendicular and carries little variance. PCA keeps coordinates along $\mathbf{w}_1$ and may discard $\mathbf{w}_2$ entirely—compressing 2D data to 1D with minimal loss.

6.2 Setting Up PCA

Suppose the original data points are $\mathbf{y}_1^{\text{raw}}, \dots, \mathbf{y}_N^{\text{raw}} \in \mathbb{R}^M$. Their sample mean is

$$\bar{\mathbf{y}} = \frac{1}{N} \sum_{n=1}^N \mathbf{y}_n^{\text{raw}}.$$

PCA is applied to the centered data

$$\mathbf{y}_n = \mathbf{y}_n^{\text{raw}} - \bar{\mathbf{y}}, \quad Y = [\mathbf{y}_1 \ \mathbf{y}_2 \ \cdots \ \mathbf{y}_N] \in \mathbb{R}^{M \times N}.$$

Thus the columns of Y have mean zero.

Goal. Find $D \ll M$ orthonormal directions $\mathbf{w}_1, \dots, \mathbf{w}_D$, collected as

$$W = [\mathbf{w}_1 \ \cdots \ \mathbf{w}_D] \in \mathbb{R}^{M \times D}, \quad W^T W = I_D.$$

The compressed representation of the n -th centered data point is

$$\mathbf{x}_n = W^T \mathbf{y}_n \in \mathbb{R}^D, \quad x_{dn} = \mathbf{w}_d^T \mathbf{y}_n.$$

Collecting these coordinate vectors as columns gives

$$X = [\mathbf{x}_1 \ \cdots \ \mathbf{x}_N] = W^T Y \in \mathbb{R}^{D \times N}.$$

Here x_{dn} is the entry in row d and column n of X : the coordinate of data point n along principal direction d .

Which directions should we pick? The ones along which the centered data *varies the most*—directions with no variation carry no information.

6.3 Deriving the PCA Solution

The variance of the data when projected onto a unit direction $\mathbf{w}$ is

$$\frac{1}{N} \sum_{n=1}^N (\mathbf{w}^T \mathbf{y}_n)^2 = \frac{1}{N} \sum_{n=1}^N \mathbf{w}^T \mathbf{y}_n \mathbf{y}_n^T \mathbf{w} = \mathbf{w}^T \left(\underbrace{\frac{1}{N} \sum_{n=1}^N \mathbf{y}_n \mathbf{y}_n^T}_C \right) \mathbf{w} = \mathbf{w}^T C \mathbf{w}.$$

$C = \frac{1}{N} Y Y^T$

The symmetric $M \times M$ matrix C is the **empirical covariance matrix** (note that it is built from *outer products* of the data—Week 2’s operation finding its purpose). A small remark on conventions: many statistics texts define the covariance with $1/(N-1)$ instead of $1/N$. This rescales every eigenvalue by the same factor but does not change the eigenvectors, so the principal *directions*—the objects PCA actually delivers—are identical under either convention. So PCA is the constrained optimization problem

$$\max_{\mathbf{w}} \mathbf{w}^T C \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^T \mathbf{w} = 1.$$

We will develop the full machinery for such problems in Week 13 (Lagrange multipliers); here is the punchline, which we can verify directly. Because C is symmetric, the spectral theorem gives $C = \sum_i \lambda_i \mathbf{v}_i \mathbf{v}_i^T$ with orthonormal $\mathbf{v}_i$ and $\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$. Expanding any unit vector as $\mathbf{w} = \sum_i c_i \mathbf{v}_i$ with $\sum_i c_i^2 = 1$:

$$\mathbf{w}^T C \mathbf{w} = \sum_i \lambda_i c_i^2 \leq \lambda_1 \sum_i c_i^2 = \lambda_1,$$

with equality exactly when $\mathbf{w}$ is a unit eigenvector associated with the largest eigenvalue. Conclusion:

Key Idea: PCA = eigenvectors of the covariance

The direction of maximal variance is the top eigenvector of $C = \frac{1}{N}YY^T$, and the captured variance is the top eigenvalue λ_1 . The best D directions are the top D eigenvectors.

The PCA algorithm.

1. Compute the mean $\bar{\mathbf{y}}$ and center the data: $\mathbf{y}_n = \mathbf{y}_n^{\text{raw}} - \bar{\mathbf{y}}$.

2. Form

$$Y = [\mathbf{y}_1 \cdots \mathbf{y}_N], \quad C = \frac{1}{N}YY^T.$$

3. Compute the eigenvectors of C and keep the D corresponding to its largest eigenvalues as the columns of W .

4. Compress each data point:

$$\mathbf{x}_n = W^T \mathbf{y}_n, \quad X = W^T Y.$$

5. Reconstruct the centered and original data, respectively, by

$$\mathbf{y}_n \approx W \mathbf{x}_n, \quad \mathbf{y}_n^{\text{raw}} \approx \bar{\mathbf{y}} + W \mathbf{x}_n.$$

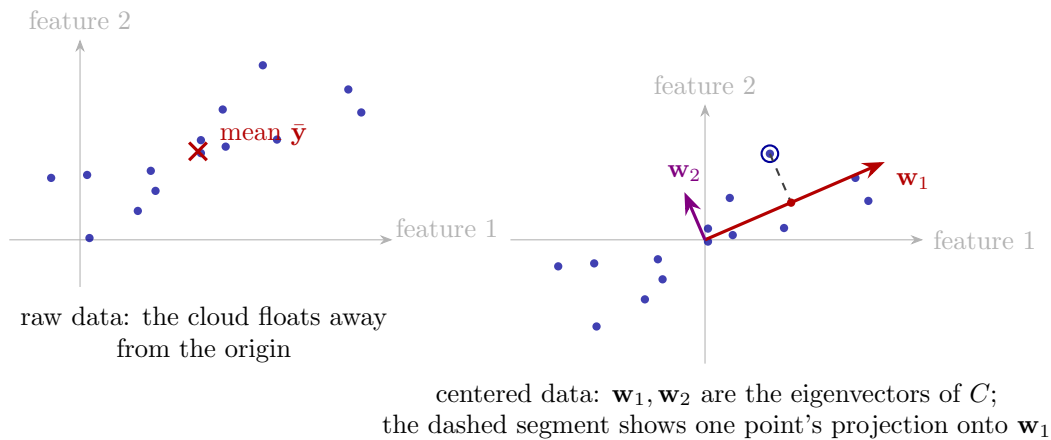


Figure 6.2: PCA on a real (synthetic) point cloud of $N = 14$ points. Left: the raw data; Step 1 of the algorithm subtracts the mean $\bar{\mathbf{y}}$ from every point. Right: the same cloud after centering. The arrows are the two eigenvectors of the covariance matrix C , drawn with lengths $2\sqrt{\lambda_i}$ (two standard deviations); here $\lambda_1 \approx 2.75$ and $\lambda_2 \approx 0.19$, so $\mathbf{w}_1$ captures about 93% of the variance. The compressed 1D code of the circled point is the position of its projection (red dot) along $\mathbf{w}_1$. Skipping the centering step would make the “top direction” point from the origin toward the mean instead (compare Exercise 6.6).

6.4 The SVD Connection

Here is the elegant finale. Take the SVD of the centered data matrix, $Y = U\Sigma V^T$. Then

$$C = \frac{1}{N}YY^T = \frac{1}{N}U\Sigma V^T V\Sigma^T U^T = U\left(\frac{\Sigma\Sigma^T}{N}\right)U^T,$$

which is an eigendecomposition of C with eigenvectors U and eigenvalues σ_i^2/N . Therefore:

- the principal directions $\mathbf{w}_i$ are exactly the **left singular vectors** $\mathbf{u}_i$ of the data matrix;
- the variance along each is σ_i^2/N ;
- the compressed coordinates come directly from the SVD. If $W = U_D = [\mathbf{u}_1 \cdots \mathbf{u}_D]$, then

$$X = W^T Y = \Sigma_D V_D^T.$$

Therefore, the entry in row d and column n is

$$x_{dn} = X_{dn} = \mathbf{u}_d^T \mathbf{y}_n = \sigma_d (\mathbf{v}_d)_n,$$

where $(\mathbf{v}_d)_n$ is the n -th entry of the d -th right singular vector.

In practice nobody forms C at all—software computes the SVD of Y directly (it is numerically safer). *PCA is the SVD.*

Moreover, truncating the rank-one expansion after D terms,

$$Y \approx \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \cdots + \sigma_D \mathbf{u}_D \mathbf{v}_D^T,$$

gives the *best possible* rank- D approximation of the data matrix (the Eckart–Young theorem). The discarded error is governed by the singular values you threw away—so a quick look at how fast the σ_i decay tells you how compressible your dataset is.

Example 6.1: A tiny PCA by hand

Four centered data points in $\mathbb{R}^2$ (columns of Y):

$$Y = \begin{bmatrix} 2 & 1 & -1 & -2 \\ 1 & 0.5 & -0.5 & -1 \end{bmatrix}.$$

(Notice every point satisfies $y_2 = \frac{1}{2}y_1$: the cloud is perfectly flat, lying on a line.)

Step 1: Covariance.

$$C = \frac{1}{4}YY^T = \frac{1}{4} \begin{bmatrix} 4+1+1+4 & 2+0.5+0.5+2 \\ 2+0.5+0.5+2 & 1+0.25+0.25+1 \end{bmatrix} = \begin{bmatrix} 2.5 & 1.25 \\ 1.25 & 0.625 \end{bmatrix}.$$

Step 2: Eigenvalues. $\det(C - \lambda I) = \lambda^2 - 3.125\lambda + (2.5)(0.625) - 1.25^2 = \lambda^2 - 3.125\lambda + 0$, so $\lambda_1 = \mathbf{3.125}$ and $\lambda_2 = \mathbf{0}$.

Step 3: Top eigenvector. $(C - 3.125I)\mathbf{w} = \mathbf{0}$ gives $-0.625w_1 + 1.25w_2 = 0 \implies w_1 = 2w_2$, normalized: $\mathbf{w}_1 = \frac{1}{\sqrt{5}}[2, 1]^T$.

Interpretation. $\lambda_2 = 0$ says there is *zero* variance perpendicular to $\mathbf{w}_1$: one coordinate per point, $x_n = \mathbf{w}_1^T \mathbf{y}_n$, captures the dataset with no loss at all. We compressed 2D data to 1D for free. Real data is never perfectly flat, but the same machinery discards directions whose λ is merely *small*.

6.5 Machine Learning Application

ML Link: Eigenfaces and Image Compression

A face image with 100×100 pixels is a vector in $\mathbb{R}^{10000}$. Run PCA on a database of thousands of face photos and the top principal directions—reshaped back into images—look like ghostly face templates, nicknamed **eigenfaces** [9]. Empirically, only 50–100 of them are needed to reconstruct a face recognizably:

$$\text{face} \approx \bar{\mathbf{y}} + x_1(\text{eigenface}_1) + x_2(\text{eigenface}_2) + \cdots + x_{50}(\text{eigenface}_{50}).$$

A face is then *stored* as just 50 numbers $x_1, \dots, x_{50}$ instead of 10,000 pixels—a 200× compression—and two photos are compared by comparing their 50-number codes. Modern face recognition uses deep networks, but the idea of learning a small set of directions that explain the data is PCA’s idea, and PCA is still the first tool any data scientist reaches for when meeting a new high-dimensional dataset.

6.6 End of Chapter Exercises**Week 6 Exercises**

6.1 For $A = \begin{bmatrix} 1 & -2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$, compute the 2×2 matrix $A^T A$.

6.2 Find the eigenvalues of the $A^T A$ from 6.1 and the non-zero singular values σ_1, σ_2 of A .

6.3 Find the right singular vectors $\mathbf{v}_1, \mathbf{v}_2$ (eigenvectors of $A^T A$, normalized, largest eigenvalue first).

6.4 Use the linking relation $\mathbf{u}_i = \frac{1}{\sigma_i} A \mathbf{v}_i$ to compute the left singular vectors $\mathbf{u}_1, \mathbf{u}_2$, and verify $\mathbf{u}_1^T \mathbf{u}_2 = 0$.

6.5 A matrix $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ -1 & 1 \end{bmatrix}$ has singular values $\sigma_1 = \sqrt{3}$, $\sigma_2 = \sqrt{2}$ with right singular vectors $\mathbf{v}_1 = [0, 1]^T$, $\mathbf{v}_2 = [1, 0]^T$. Compute $\mathbf{u}_1$ and $\mathbf{u}_2$.

6.6 (*Theoretical*) Explain why the data must be *centered* (mean subtracted) before PCA. What direction would the “top eigenvector” of $\frac{1}{N} Y Y^T$ tend to point toward if the data had a large nonzero mean?

Part II

Navigating the Loss Landscape (Unconstrained Optimization)

Chapter 7

Week 7: Derivatives, Gradients, and the Direction of Steepest Descent

7.1 The Intuition: Blindfolded on a Mountain

Part I taught us how data is represented. Part II asks the central question of *learning*: how does a machine improve?

In machine learning we define a **loss function** $f(\mathbf{x})$ that measures how badly the model is doing; the input $\mathbf{x}$ collects all the model's adjustable parameters (for a large neural network, billions of them). Training means finding the $\mathbf{x}$ that makes f as small as possible. Picture the graph of f as a vast mountain landscape, with the AI standing on it blindfolded. It cannot see the valley—it can only *feel the local slope under its feet*. The mathematical object that encodes that local slope, in every direction at once, is the **gradient**. This chapter builds it from the ground up.

7.2 Warm-up: Derivatives and Taylor Polynomials in One Variable

Recall the one-variable derivative,

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h},$$

the slope of the tangent line. The two rules we will use constantly:

$$\textbf{Product rule: } (fg)' = f'g + fg', \quad \textbf{Chain rule: } (f(g(x)))' = f'(g(x))g'(x).$$

For example, $\frac{d}{dx}e^{-x^2} = e^{-x^2} \cdot (-2x)$ (outer derivative times inner derivative).

The same rules give the derivatives of several activation functions used in neural networks.

The ReLU function

The rectified linear unit is

$$\text{ReLU}(x) = \max\{0, x\} = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

Its derivative is

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases}$$

At $x = 0$, the left and right derivatives are different, so ReLU is not differentiable there. Machine learning software usually assigns a convenient value, often 0, to the derivative at this single point.

For an affine input $z = wx + b$, the chain rule gives, whenever $z \neq 0$,

$$\frac{d}{dx} \text{ReLU}(wx + b) = \begin{cases} 0, & wx + b < 0, \\ w, & wx + b > 0. \end{cases}$$

The sigmoid function

The sigmoid function is

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Differentiating gives

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x)(1 - \sigma(x)).$$

Thus its derivative can be written directly in terms of the function itself. For an affine input $z = wx + b$, the chain rule gives

$$\frac{d}{dx} \sigma(wx + b) = w \sigma(wx + b)(1 - \sigma(wx + b)).$$

The hyperbolic tangent function

The hyperbolic tangent activation function is

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Its derivative is

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$

Consequently, for an affine input $z = wx + b$, the chain rule gives

$$\frac{d}{dx} \tanh(wx + b) = w(1 - \tanh^2(wx + b)).$$

Like the sigmoid, $\tanh$ has a derivative that can be expressed directly using the function's output.

The derivative's deeper job is *approximation*. Near a base point x_0 , a smooth function looks like a polynomial—its **Taylor polynomial**:

$$T_n(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2 + \cdots$$

with an error (remainder) controlled by the next derivative:

$$f(x) - T_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1} \quad \text{for some } \xi \text{ between } x_0 \text{ and } x.$$

Example. For $f(x) = \sin x$ at $x_0 = 0$: the degree-1 polynomial is $T_1(x) = x$, and because $f''(0) = -\sin 0 = 0$, the degree-2 polynomial is *still* $T_2(x) = x$ —the quadratic term vanishes. Applying the remainder formula to T_2 , with $|f^{(3)}(x)| = |-\cos x| \leq 1$, gives the bound

$$|\sin x - x| \leq \frac{|x|^3}{6}.$$

Near 0 the line $y = x$ is an excellent stand-in for the sine curve. Part of optimization is built on this idea: *replace a complicated function locally by its first- or second-order Taylor polynomial, and work with the polynomial.*

7.3 Partial Derivatives and the Gradient

Definition: Partial derivatives and the gradient

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **partial derivative** with respect to x_i differentiates while freezing all other variables:

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_n)}{h}.$$

The **gradient** collects all of them into a vector:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{bmatrix}.$$

Example. $f(x, y) = x^2y^3 + 2x - 3y^2$:

$$\frac{\partial f}{\partial x} = 2xy^3 + 2 \quad (\text{treat } y \text{ as a constant}), \quad \frac{\partial f}{\partial y} = 3x^2y^2 - 6y,$$

so $\nabla f = [2xy^3 + 2, \quad 3x^2y^2 - 6y]^T$.

7.4 The Multivariable Chain Rule and the Jacobian

If the inputs themselves depend on time, $x_1 = x_1(t), \dots, x_n = x_n(t)$, then

$$\frac{d}{dt} f(\mathbf{x}(t)) = \nabla f(\mathbf{x}(t))^T \mathbf{x}'(t) = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \frac{dx_i}{dt}.$$

Each route by which t influences f contributes one term.

Example 7.4: Chain rule along a curve

Let $f(x, y) = x^2 + 3xy$ with $x(t) = \cos t$, $y(t) = \sin t$. Compute $\frac{d}{dt}f(x(t), y(t))$ at $t = 0$.

Step 1: Gradient. $\nabla f = \begin{bmatrix} 2x + 3y \\ 3x \end{bmatrix}$. At $t = 0$: $(x, y) = (\mathbf{1}, \mathbf{0})$, so $\nabla f(1, 0) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$.

Step 2: Velocity of the curve. $\mathbf{x}'(t) = \begin{bmatrix} -\sin t \\ \cos t \end{bmatrix}$, so $\mathbf{x}'(0) = \begin{bmatrix} \mathbf{0} \\ \mathbf{1} \end{bmatrix}$.

Step 3: Dot product.

$$\left. \frac{d}{dt}f \right|_{t=0} = \nabla f(1, 0)^T \mathbf{x}'(0) = (2)(0) + (3)(1) = \mathbf{3}.$$

For a *vector-valued* function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (each of m outputs depends on n inputs), we stack the gradients of the outputs as rows:

Definition: Jacobian matrix

$$J = \frac{d\mathbf{f}}{d\mathbf{x}} = \begin{bmatrix} \partial f_1/\partial x_1 & \cdots & \partial f_1/\partial x_n \\ \vdots & & \vdots \\ \partial f_m/\partial x_1 & \cdots & \partial f_m/\partial x_n \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

The chain rule for compositions becomes a *matrix product*: if $\mathbf{h} = \mathbf{f} \circ \mathbf{g}$, then $J_{\mathbf{h}} = J_{\mathbf{f}} J_{\mathbf{g}}$.

Example 7.5: Computing a Jacobian

For $\mathbf{f}(x, y) = \begin{bmatrix} x^2 + y^2 \\ e^{x-y} + xy \\ y^2 \end{bmatrix}$,

$$J(x, y) = \begin{bmatrix} 2x & 2y \\ e^{x-y} + y & -e^{x-y} + x \\ 0 & 2y \end{bmatrix}, \quad J(1, 1) = \begin{bmatrix} 2 & 2 \\ 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

A particularly important case occurs when $m = 1$, so that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is scalar-valued. The gradient is conventionally written as a column vector,

$$\nabla f = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix},$$

whereas the Jacobian is a row vector. Therefore,

$$J_f = (\nabla f)^T.$$

Keeping track of this transpose is important when applying the chain rule. For $f : \mathbb{R}^m \rightarrow \mathbb{R}$ and $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^m$,

$$J_{f \circ \mathbf{g}} = J_f J_{\mathbf{g}} = (\nabla f)^T J_{\mathbf{g}}.$$

Transposing both sides gives the gradient form of the chain rule:

$$\nabla(f \circ \mathbf{g})(\mathbf{x}) = J_{\mathbf{g}}(\mathbf{x})^T \nabla f(\mathbf{g}(\mathbf{x})).$$

The chain rule and backpropagation. When the intermediate variables are vectors, the chain rule has the same basic meaning as in one variable: differentiate each stage and multiply the derivatives. The difference is that the derivatives are now matrices.

Suppose

$$\mathbf{x} \xrightarrow{\mathbf{g}} \mathbf{z} \xrightarrow{\mathbf{f}} \mathbf{y},$$

where

$$\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^p.$$

Then

$$J_{\mathbf{f} \circ \mathbf{g}}(\mathbf{x}) = J_{\mathbf{f}}(\mathbf{g}(\mathbf{x})) J_{\mathbf{g}}(\mathbf{x}).$$

The matrix dimensions show why the multiplication must occur in this order:

$$\underbrace{J_{\mathbf{f} \circ \mathbf{g}}}_{p \times n} = \underbrace{J_{\mathbf{f}}}_{p \times m} \underbrace{J_{\mathbf{g}}}_{m \times n}.$$

In machine learning, the final output is often a scalar loss $L : \mathbb{R}^m \rightarrow \mathbb{R}$. If $\mathbf{z} = \mathbf{g}(\mathbf{x})$, the gradient form of the chain rule is

$$\boxed{\nabla_{\mathbf{x}} L = J_{\mathbf{g}}(\mathbf{x})^T \nabla_{\mathbf{z}} L.}$$

Thus a gradient is passed backward through a function by multiplying it by the transpose of that function's Jacobian.

A neural network is a long composition of functions. If

$$\mathbf{z}_0 = \mathbf{x}, \quad \mathbf{z}_1 = \mathbf{f}_1(\mathbf{z}_0), \quad \mathbf{z}_2 = \mathbf{f}_2(\mathbf{z}_1), \quad \dots, \quad L = L(\mathbf{z}_k),$$

then the gradients can be propagated backward according to

$$\nabla_{\mathbf{z}_{j-1}} L = J_{\mathbf{f}_j}(\mathbf{z}_{j-1})^T \nabla_{\mathbf{z}_j} L.$$

Applying this rule repeatedly is called **backpropagation**. It is not a different differentiation rule: it is the multivariable chain rule organized so that the derivatives of a scalar loss can be computed efficiently from the last layer back to the first.

Example 7.6: A composition of two vector-valued mappings

Consider the two mappings

$$\begin{bmatrix} s \\ t \end{bmatrix} \xrightarrow{\mathbf{g}} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \xrightarrow{\mathbf{f}} \begin{bmatrix} u \\ v \end{bmatrix},$$

defined by

$$x = s + t, \quad y = s - t, \quad z = st,$$

and

$$u = x^2 + yz, \quad v = xy + z.$$

Thus

$$\mathbf{g} : \mathbb{R}^2 \rightarrow \mathbb{R}^3, \quad \mathbf{f} : \mathbb{R}^3 \rightarrow \mathbb{R}^2.$$

The Jacobian of the first mapping is

$$J_{\mathbf{g}}(s, t) = \frac{\partial(x, y, z)}{\partial(s, t)} = \begin{bmatrix} \frac{\partial x}{\partial s} & \frac{\partial x}{\partial t} \\ \frac{\partial y}{\partial s} & \frac{\partial y}{\partial t} \\ \frac{\partial z}{\partial s} & \frac{\partial z}{\partial t} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ t & s \end{bmatrix}.$$

The Jacobian of the second mapping is

$$J_{\mathbf{f}}(x, y, z) = \frac{\partial(u, v)}{\partial(x, y, z)} = \begin{bmatrix} 2x & z & y \\ y & x & 1 \end{bmatrix}.$$

The composition

$$\mathbf{h} = \mathbf{f} \circ \mathbf{g}$$

maps (s, t) directly to (u, v) . By the chain rule,

$$J_{\mathbf{h}}(s, t) = J_{\mathbf{f}}(\mathbf{g}(s, t)) J_{\mathbf{g}}(s, t).$$

Notice how the dimensions fit:

$$\underbrace{J_{\mathbf{h}}}_{2 \times 2} = \underbrace{J_{\mathbf{f}}}_{2 \times 3} \underbrace{J_{\mathbf{g}}}_{3 \times 2}.$$

Carrying out the multiplication gives

$$J_{\mathbf{h}} = \begin{bmatrix} 2x & z & y \\ y & x & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ t & s \end{bmatrix} = \begin{bmatrix} 2x + z + yt & 2x - z + ys \\ x + y + t & y - x + s \end{bmatrix},$$

where $x = s + t$, $y = s - t$, and $z = st$.

For example, at $(s, t) = (1, 2)$,

$$(x, y, z) = (3, -1, 2).$$

Therefore,

$$J_{\mathbf{g}}(1, 2) = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 2 & 1 \end{bmatrix}, \quad J_{\mathbf{f}}(3, -1, 2) = \begin{bmatrix} 6 & 2 & -1 \\ -1 & 3 & 1 \end{bmatrix},$$

and hence

$$J_{\mathbf{h}}(1, 2) = \begin{bmatrix} 6 & 2 & -1 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 3 \\ 4 & -3 \end{bmatrix}.$$

For instance, the upper-left entry comes from all three paths connecting s to u :

$$\frac{\partial u}{\partial s} = \frac{\partial u}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial u}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial u}{\partial z} \frac{\partial z}{\partial s}.$$

At $(s, t) = (1, 2)$, this becomes

$$\frac{\partial u}{\partial s} = 6(1) + 2(1) + (-1)(2) = 6.$$

Thus matrix multiplication automatically adds the contributions from all intermediate variables through which one quantity affects another.

Example 7.7: The multivariable chain rule in a small neural network

Consider a network with two inputs and two hidden units:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad \mathbf{a} = \text{ReLU}(\mathbf{z}), \quad \hat{y} = \mathbf{v}^T \mathbf{a} + c,$$

followed by the squared-error loss

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

The ReLU function is applied to each component of $\mathbf{z}$.

Take

$$\mathbf{x} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

and

$$\mathbf{v} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad c = 0, \quad y = 1.$$

We first perform the *forward pass*:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b} = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Both components are positive, so

$$\mathbf{a} = \text{ReLU}(\mathbf{z}) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Therefore,

$$\hat{y} = \mathbf{v}^T \mathbf{a} = \begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 0,$$

and

$$L = \frac{1}{2}(0 - 1)^2 = \frac{1}{2}.$$

We now propagate derivatives backward. First,

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -1.$$

Since $\hat{y} = \mathbf{v}^T \mathbf{a} + c$,

$$\nabla_{\mathbf{a}} L = \mathbf{v} \frac{\partial L}{\partial \hat{y}} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} (-1) = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

Because both entries of $\mathbf{z}$ are positive, the Jacobian of the componentwise ReLU function is

$$J_{\text{ReLU}}(\mathbf{z}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Hence

$$\nabla_{\mathbf{z}} L = J_{\text{ReLU}}(\mathbf{z})^T \nabla_{\mathbf{a}} L = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

Finally, because $\mathbf{z} = W\mathbf{x} + \mathbf{b}$,

$$J_{\mathbf{z}}(\mathbf{x}) = W,$$

and the gradient with respect to the input is

$$\nabla_{\mathbf{x}}L = W^T \nabla_{\mathbf{z}}L = \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

Backpropagation also gives the derivatives with respect to the trainable parameters:

$$\nabla_{\mathbf{v}}L = (\hat{y} - y)\mathbf{a} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}, \quad \frac{\partial L}{\partial c} = -1,$$

and

$$\nabla_W L = (\nabla_{\mathbf{z}}L)\mathbf{x}^T = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}, \quad \nabla_{\mathbf{b}}L = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

The calculation moves forward to evaluate the network, but backward to compute the gradients:

$$L \longrightarrow \hat{y} \longrightarrow \mathbf{a} \longrightarrow \mathbf{z} \longrightarrow \mathbf{x}.$$

At each step, the incoming gradient is multiplied by a local derivative or the transpose of a local Jacobian. This repeated use of the chain rule is the core of backpropagation.

7.4.1 A Toolbox of Gradient Identities

The following identities are used on nearly every page of machine learning. Each is proved by writing out the sums—do it once in your life, then use the table forever. Here $\mathbf{w}$ is a constant vector and A a constant matrix; ∇ is with respect to $\mathbf{x}$.

$f(\mathbf{x})$	$\nabla f(\mathbf{x})$	why
$\mathbf{w}^T \mathbf{x}$	$\mathbf{w}$	linear function; slope is constant
$\mathbf{x}^T \mathbf{x} = \ \mathbf{x}\ ^2$	$2\mathbf{x}$	like $\frac{d}{dx}x^2 = 2x$, per coordinate
$\mathbf{x}^T A \mathbf{x}$ (A symmetric)	$2A\mathbf{x}$	quadratic form; the matrix version of ax^2
$e^{\mathbf{w}^T \mathbf{x}}$	$e^{\mathbf{w}^T \mathbf{x}} \mathbf{w}$	chain rule with inner function $\mathbf{w}^T \mathbf{x}$
$\frac{1}{2}\ A\mathbf{x} - \mathbf{b}\ ^2$	$A^T(A\mathbf{x} - \mathbf{b})$	chain rule; derived in full in Week 9

Sample derivation (first row): $f = \sum_i w_i x_i$, so $\partial f / \partial x_j = w_j$; stacking gives $\nabla f = \mathbf{w}$. The third row generalizes it: $f = \sum_{i,j} a_{ij} x_i x_j$, and differentiating with respect to x_k picks up the terms where $i = k$ or $j = k$, giving $\sum_j a_{kj} x_j + \sum_i a_{ik} x_i = 2(A\mathbf{x})_k$ when A is symmetric.

7.5 Level Curves and the Geometry of the Gradient

A surface $z = f(x_1, x_2)$ can be drawn as a **contour map**: the *level curves* $f(x_1, x_2) = c$ for equally spaced values of c , exactly like a hiking map. For example, $f = x_1^2 + 3x_2^2$ has ellipses as level curves. Two facts to internalize from the picture:

- the gradient at a point is **perpendicular to the level curve** through that point (moving along the curve keeps f constant, so the rate of change along it is zero);
- densely packed contours mean steep terrain; widely spaced contours mean flat terrain.

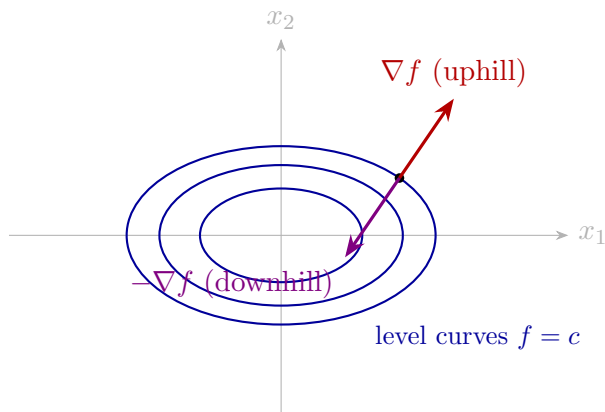


Figure 7.1: Contour map of $f(x_1, x_2) = x_1^2 + 3x_2^2$. The gradient is perpendicular to the level curve and points uphill; its negative is the direction gradient descent will walk.

7.6 The Directional Derivative

The partial derivatives measure the slope along the coordinate axes. What about an arbitrary direction?

Definition: Directional derivative

Let $\mathbf{u}$ be a *unit* vector. The directional derivative of f at $\mathbf{x}$ in the direction $\mathbf{u}$ is

$$D_{\mathbf{u}}f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{u}) - f(\mathbf{x})}{h} = \nabla f(\mathbf{x})^T \mathbf{u}.$$

(The second equality follows from the chain rule applied to $g(h) = f(\mathbf{x} + h\mathbf{u})$ at $h = 0$.) It is the slope you feel if you walk through $\mathbf{x}$ heading in direction $\mathbf{u}$.

Now the most important consequence in all of machine learning. By the angle formula of Week 2,

$$D_{\mathbf{u}}f = \nabla f^T \mathbf{u} = \|\nabla f\| \underbrace{\|\mathbf{u}\|}_{1} \cos \theta = \|\nabla f\| \cos \theta,$$

where θ is the angle between $\mathbf{u}$ and the gradient. As $\mathbf{u}$ swings around, $\cos \theta$ ranges over $[-1, 1]$, so:

Key Idea: Steepest ascent and descent

- the slope is *largest* ($= \|\nabla f\|$) when $\theta = 0$: **the gradient points in the direction of steepest ascent**;
- the slope is *most negative* ($= -\|\nabla f\|$) when $\theta = 180^\circ$: **$-\nabla f$ is the direction of steepest descent**;
- the slope is zero when $\theta = 90^\circ$: moving along a level curve.

To reduce its loss as fast as possible, a learning algorithm should step in the direction $-\nabla f$. That single sentence is the design principle of gradient descent.

Example 7.8: Calculating a directional derivative

Let $f(x_1, x_2) = x_1 e^{x_2}$. We stand at $(2, 0)$ and consider stepping in the direction of $\mathbf{v} = [3, 4]^T$.

Step 1: Normalize the direction. $\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = 5$, so $\mathbf{u} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}$. (Skipping this step is the single most common exam error—the formula requires a *unit* vector.)

Step 2: Gradient. $\nabla f = \begin{bmatrix} e^{x_2} \\ x_1 e^{x_2} \end{bmatrix}$, and at $(2, 0)$: $\nabla f(2, 0) = \begin{bmatrix} e^0 \\ 2e^0 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.

Step 3: Dot product.

$$D_{\mathbf{u}}f = \begin{bmatrix} 1 \\ 2 \end{bmatrix}^T \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = \frac{3}{5} + \frac{8}{5} = \frac{11}{5}.$$

The slope is positive: stepping this way *increases* the loss. The fastest decrease is instead along $-\nabla f / \|\nabla f\| = -\frac{1}{\sqrt{5}}[1, 2]^T$, with slope $-\|\nabla f\| = -\sqrt{5}$.

7.7 Machine Learning Application

ML Link: Gradient Descent and Backpropagation

Gradient descent is the core training loop of nearly every modern ML system. The parameter update at step k is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k),$$

where $\alpha > 0$ is the **learning rate** (step size). The minus sign implements this chapter’s key idea: walk against the gradient, downhill. Choosing α well is delicate—we devote Week 11 to it.

And how does a network with a billion parameters compute its gradient? By the **multivariable chain rule**, applied systematically layer by layer. A neural network is a composition of layers

$$\mathbf{h}_\ell = \phi_\ell(A_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell), \quad \mathbf{h}_0 = \mathbf{x}, \quad F_\ell(\mathbf{h}) = \phi_\ell(A_\ell \mathbf{h} + \mathbf{b}_\ell).$$

For a loss $\mathcal{L} = \ell(\mathbf{h}_L, \mathbf{y})$, **backpropagation** applies the chain rule backward through the layers, propagating

$$\delta_{\ell-1} = J_{F_\ell}(\mathbf{h}_{\ell-1})^T \delta_\ell,$$

while computing the gradients with respect to all weights and biases. When you hear “back-prop,” hear “Week 7 chain rule, organized efficiently.”

7.8 End of Chapter Exercises

Week 7 Exercises

7.1 Let $\mathbf{f}(x, y) = \begin{bmatrix} x^2 + y \\ \sin(xy) \end{bmatrix}$. Compute the Jacobian matrix $J(x, y)$.

7.2 Let $f(x, y, z) = x^2y + yz^2 - 2xz$. Compute the directional derivative of f at the point $(1, 2, -1)$ in the direction of $\mathbf{v} = [2, -1, 2]^T$. (Remember to normalize.)

7.3 For the function in 7.2, what unit direction $\mathbf{u}$ gives the fastest possible rate of decrease at $(1, 2, -1)$, and what is that rate?

7.4 Let $f(x, y, z) = x^2y + yz^2$. (a) Compute ∇f at $(1, 1, 1)$. (b) Find the directional derivative at $(1, 1, 1)$ in the direction $\mathbf{v} = [2, -1, 2]^T$.

7.5 Using the identity table, write down (no computation!) the gradients of: (a) $f(\mathbf{x}) = [1, -2, 5]\mathbf{x}$; (b) $f(\mathbf{x}) = \|\mathbf{x}\|^2$; (c) $f(\mathbf{x}) = \mathbf{x}^T \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \mathbf{x}$.

7.6 Sketch (by hand) three level curves of $f(x, y) = x^2 + 3y^2$ for $c = 1, 2, 3$, and draw the gradient vector at the point $(0, 1)$. Confirm visually that it is perpendicular to the level curve through that point.

Exam Practice

Problem 2 (20 points). Let $f(x, y) = x^2 + 3xy$. Suppose the variables x and y are parameterized by time t such that $x(t) = \cos t$ and $y(t) = \sin t$.

(a) Find the gradient vector $\nabla f(x, y)$.

(b) Use the multivariable chain rule to compute $\frac{d}{dt}f(x(t), y(t))$ evaluated exactly at $t = 0$.

(Worked in full as Example 7.1—try it closed-book first.)

Problem 4 (15 points). Consider the vector-valued function

$$\mathbf{f}(x, y) = \begin{bmatrix} x^2 + y^2 \\ e^{x-y} + xy \\ y^2 \end{bmatrix}.$$

Compute the Jacobian matrix $J(x, y)$ (also denoted $d\mathbf{f}/d\mathbf{x}$) at the base point $\mathbf{x}_0 = (1, 1)$. (Compare Example 7.2. Before differentiating, predict the dimensions of J .)

Chapter 8

Week 8: The Hessian, Curvature, and Taylor Approximations

8.1 The Intuition: Feeling the Curvature

The gradient tells us the slope, but it is blind to **curvature**. Two landscapes can have identical gradients at a point yet behave completely differently one step later: one curls up into a bowl, the other drops away into a saddle. To distinguish them we need second derivatives—all of them, organized into a matrix.

8.2 The Hessian Matrix

Definition: Hessian

The Hessian of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the $n \times n$ matrix of second partial derivatives,

$$H(\mathbf{x}) = \nabla^2 f(\mathbf{x}), \quad H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

For smooth functions the order of differentiation does not matter ($H_{ij} = H_{ji}$), so **the Hessian is always symmetric**—which means the entire spectral theory of Week 4 applies to it.

Example. $f(x, y) = x^2 - 2x^2y + y^3$:

$$\nabla f = \begin{bmatrix} 2x - 4xy \\ -2x^2 + 3y^2 \end{bmatrix}, \quad H = \begin{bmatrix} 2 - 4y & -4x \\ -4x & 6y \end{bmatrix}.$$

Note the equal off-diagonal entries, as promised.

What the Hessian measures. Apply the directional-derivative idea twice: the second-order rate of change of f along a unit direction $\mathbf{u}$ works out to be the quadratic form

$$D_{\mathbf{u}}^2 f = \mathbf{u}^T H \mathbf{u}.$$

So the Hessian is a curvature-measuring machine: feed it a direction, get back the curvature in that direction. Whether the landscape is a bowl, a dome, or a saddle is the question of the *sign* of $\mathbf{u}^T H \mathbf{u}$ across all directions:

Definition: Definiteness

A symmetric matrix H is

- **positive definite** if $\mathbf{x}^T H \mathbf{x} > 0$ for every $\mathbf{x} \neq \mathbf{0}$ (curves up in every direction: a bowl),
- **negative definite** if $\mathbf{x}^T H \mathbf{x} < 0$ for every $\mathbf{x} \neq \mathbf{0}$ (curves down in every direction: a dome),
- **indefinite** if $\mathbf{x}^T H \mathbf{x}$ takes both signs (up some ways, down others: a saddle).

8.2.1 Classifying via Eigenvalues

Definiteness sounds like it requires checking infinitely many directions. The spectral theorem reduces it to finitely many numbers. Write $H = \sum_i \lambda_i \mathbf{v}_i \mathbf{v}_i^T$ with orthonormal eigenvectors and expand any $\mathbf{x} = \sum_i c_i \mathbf{v}_i$. Then

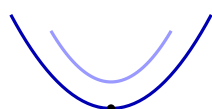
$$\mathbf{x}^T H \mathbf{x} = \sum_i \lambda_i c_i^2.$$

Every direction's curvature is a nonnegative mixture of the eigenvalues! Hence:

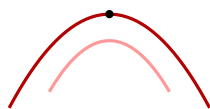
Key Idea: Eigenvalues decide everything

- | | | |
|---------------------|--------|------------------------------|
| all $\lambda_i > 0$ | $\iff$ | H positive definite (bowl) |
| all $\lambda_i < 0$ | $\iff$ | H negative definite (dome) |
| mixed signs | $\iff$ | H indefinite (saddle) |

Moreover, over unit vectors, $\mathbf{x}^T H \mathbf{x}$ is *maximized* at $\lambda_{\max}$ (achieved at the top eigenvector) and *minimized* at $\lambda_{\min}$: the eigenvectors are the directions of extreme curvature.



bowl: $\lambda_1, \lambda_2 > 0$
(positive definite)



dome: $\lambda_1, \lambda_2 < 0$
(negative definite)



saddle: mixed signs
(indefinite)

Figure 8.1: Cross-sections of the three local landscape shapes. Along each eigenvector the surface is a parabola whose curvature is the eigenvalue.

8.2.2 The 2×2 Shortcut: The Determinant Test

For a 2×2 Hessian we can skip the eigenvalue computation. Since $\det H = \lambda_1 \lambda_2$ and the diagonal entry h_{11} is the curvature along the x -axis:

- $\det H > 0$: the eigenvalues share a sign. Then $h_{11} > 0 \Rightarrow$ **positive definite**; $h_{11} < 0 \Rightarrow$ **negative definite**.
- $\det H < 0$: the eigenvalues have opposite signs: **indefinite (saddle)**.
- $\det H = 0$: borderline; the test is inconclusive.

Example 8.1: The determinant test in action

Classify $H_A = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ and $H_B = \begin{bmatrix} -3 & 1 \\ 1 & -2 \end{bmatrix}$.

Matrix H_A : $\det H_A = (4)(3) - (2)(2) = 8 > 0$, so the eigenvalues share a sign; $h_{11} = 4 > 0$ picks the sign: **positive definite** (a bowl).

Matrix H_B : $\det H_B = (-3)(-2) - (1)(1) = 5 > 0$, same sign again; now $h_{11} = -3 < 0$: **negative definite** (a dome).

8.3 Linear and Quadratic Approximations in Several Variables

We can now state the multivariable Taylor expansion to second order around a base point $\mathbf{x}_0$:

Key Idea: Multivariate Taylor approximations

Linear (tangent plane): $L(\mathbf{x}) = f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^T (\mathbf{x} - \mathbf{x}_0)$,

Quadratic (fitted bowl): $Q(\mathbf{x}) = L(\mathbf{x}) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^T H(\mathbf{x}_0) (\mathbf{x} - \mathbf{x}_0)$.

L matches f 's value and slope at $\mathbf{x}_0$; Q additionally matches the curvature, so it stays accurate over a larger neighborhood.

Example 8.2: Linear vs. quadratic approximation (from lecture)

Let $f(x, y) = x^2 + 2xy + y^3$, with base point $\mathbf{x}_0 = (1, 2)$.

Ingredients. $f(1, 2) = 1 + 4 + 8 = 13$.

$$\nabla f = \begin{bmatrix} 2x + 2y \\ 2x + 3y^2 \end{bmatrix} \Rightarrow \nabla f(1, 2) = \begin{bmatrix} 6 \\ 14 \end{bmatrix}, \quad H = \begin{bmatrix} 2 & 2 \\ 2 & 6y \end{bmatrix} \Rightarrow H(1, 2) = \begin{bmatrix} 2 & 2 \\ 2 & 12 \end{bmatrix}.$$

The two approximations.

$$L(x, y) = 13 + 6(x - 1) + 14(y - 2),$$

$$Q(x, y) = L(x, y) + \frac{1}{2} \left[2(x - 1)^2 + 4(x - 1)(y - 2) + 12(y - 2)^2 \right].$$

Accuracy check at the nearby point $(1.1, 1.9)$. Here $x - 1 = 0.1$ and $y - 2 = -0.1$. The exact value is

$$f(1.1, 1.9) = (1.1)^2 + 2(1.1)(1.9) + (1.9)^3 = 1.21 + 4.18 + 6.859 = 12.249.$$

The linear approximation gives

$$L(1.1, 1.9) = 13 + 6(0.1) + 14(-0.1) = 13 + 0.6 - 1.4 = 12.2,$$

and the quadratic approximation adds the curvature correction:

$$\begin{aligned} Q(1.1, 1.9) &= 12.2 + \frac{1}{2} \left[2(0.1)^2 + 4(0.1)(-0.1) + 12(-0.1)^2 \right] \\ &= 12.2 + \frac{1}{2}(0.02 - 0.04 + 0.12) = 12.25. \end{aligned}$$

The linear approximation is off by about 0.049, while the quadratic approximation is off by about 0.001. Matching curvature pays off enormously.

8.4 Machine Learning Application

ML Link: Why Deep Learning Cannot Afford the Hessian

If the Hessian knows the full local geometry, why don't neural networks use it directly?

Scale. A model with $n = 10^6$ parameters has a gradient with 10^6 entries—but a Hessian with $n^2 = 10^{12}$ (a trillion) entries. Merely *storing* it would take terabytes; inverting it (as Newton's method in Week 12 requires) is also expensive. This is why deep learning often runs on first-order methods (gradient descent, momentum, Adam), which only ever touch ∇f , while second-order methods are reserved for the smaller models of classical statistics and engineering.

Even so, the Hessian's *eigenvalues* secretly run the show: in Week 11 we prove that $\lambda_{\max}$ caps the safe learning rate and $\lambda_{\max}/\lambda_{\min}$ dictates how fast training can converge. Practitioners who never form the Hessian still spend their days fighting its spectrum.

8.5 End of Chapter Exercises

Week 8 Exercises

8.1 Compute the Hessian matrix of $f(x, y) = x^2 + 2xy + e^{x+y}$, and verify it is symmetric.

8.2 Classify each matrix as positive definite, negative definite, or indefinite:

$$H_B = \begin{bmatrix} -3 & 1 \\ 1 & -2 \end{bmatrix}, \quad H_C = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \quad H_D = \begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix}.$$

8.3 Compute the linear approximation $L(x, y)$ of $f(x, y) = x^3y - e^{x-y}$ at $\mathbf{x}_0 = (1, 1)$, and use it to estimate $f(1.05, 0.95)$.

8.4 Compute the quadratic approximation $Q(x, y)$ of $f(x, y) = x^2 \ln y$ at $\mathbf{x}_0 = (2, 1)$.

8.5 For $H = \begin{bmatrix} 5 & -3 \\ -3 & 5 \end{bmatrix}$, find both eigenvalues, identify the directions of maximum and minimum curvature, and classify H .

8.6 Write the linear approximation $L(x, y, z)$ of $f(x, y, z) = x^2y + yz^2$ near $\mathbf{x}_0 = (1, 1, 1)$.

8.7 (When the test is silent) Compute the Hessian at the critical point $(0, 0)$ for each of

$$f(x, y) = x^4 + y^2 \quad \text{and} \quad f(x, y) = x^3 + y^2,$$

and verify that the determinant test gives $\det H = 0$ (inconclusive) in *both* cases. Then decide by direct inspection of the functions that the first has a local minimum at $(0, 0)$ while the second has neither a minimum nor a maximum there. Moral: the Hessian test classifies many critical points, but not all.

Exam Practice

Consider the function $f(x, y, z) = x^2y + yz^2$.

- (a) Compute the gradient ∇f at the point $(1, 1, 1)$.
- (b) Find the directional derivative of f at the point $(1, 1, 1)$ in the direction of the vector $\mathbf{v} = \begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix}$. (Remember: the formula $D_{\mathbf{u}}f = \nabla f^T \mathbf{u}$ requires a *unit* vector—normalize first.)
- (c) Write the linear approximation $L(x, y, z)$ for the function near the point $\mathbf{x}_0 = (1, 1, 1)$.

(Parts (a)–(b) are Exercise 7.4 and part (c) is Exercise 8.6; on the exam all three were asked together, exactly as here.)

Chapter 9

Week 9: Critical Points, Convexity, and Least Squares

9.1 The Intuition: Finding the Flat Spots

We now have everything needed to actually *solve* an unconstrained optimization problem. In one-variable calculus the recipe was: set $f'(x) = 0$ to find the flat spots, then check f'' to see which are minima. The multivariable recipe is identical with gradient in place of f' and Hessian in place of f'' .

Key Idea: The two-step recipe for minimization

1. **Find critical points:** solve the system $\nabla f(\mathbf{x}) = \mathbf{0}$ (all partial derivatives vanish at once).
2. **Classify each one** with the Hessian evaluated there:

$H(\mathbf{x}_*)$ pos. definite $\Rightarrow$ local min; neg. definite $\Rightarrow$ local max; indefinite $\Rightarrow$ saddle.

Example 9.1: Finding and classifying critical points

Let $f(x, y) = x^4 + y^4 - 4xy$.

Step 1: Solve $\nabla f = \mathbf{0}$.

$$\nabla f = \begin{bmatrix} 4x^3 - 4y \\ 4y^3 - 4x \end{bmatrix} = \mathbf{0} \implies y = x^3 \text{ and } x = y^3.$$

Substituting the first into the second: $x = (x^3)^3 = x^9$, i.e. $x(x^8 - 1) = 0$, with real solutions $x = 0, 1, -1$. The critical points are $(0, 0)$, $(1, 1)$, $(-1, -1)$.

Step 2: Build the Hessian.

$$H(x, y) = \begin{bmatrix} 12x^2 & -4 \\ -4 & 12y^2 \end{bmatrix}.$$

Step 3: Classify. At $(0, 0)$: $H = \begin{bmatrix} \mathbf{0} & -4 \\ -4 & 0 \end{bmatrix}$, $\det H = -16 < 0$: **saddle point**. At $(1, 1)$

and $(-1, -1)$: $H = \begin{bmatrix} \mathbf{12} & -4 \\ -4 & 12 \end{bmatrix}$, $\det H = 144 - 16 = 128 > 0$ and $h_{11} = 12 > 0$: **local**

minima. (By symmetry both give $f = -2$, the global minimum value.)

Example 9.2: A critical point involving an exponential

Let $f(x, y) = (x^2 + y^2)e^{-x}$. The gradient is

$$\nabla f = \begin{bmatrix} (2x - x^2 - y^2)e^{-x} \\ 2ye^{-x} \end{bmatrix}.$$

Since $e^{-x} > 0$ always, the second component forces $y = 0$, and the first then requires $2x - x^2 = 0$, so $x = 0$ or $x = 2$. At $(0, 0)$ the Hessian is $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$ (positive definite): a **local minimum**—indeed $f \geq 0$ with equality only there. At $(2, 0)$ the Hessian works out to $\begin{bmatrix} -2e^{-2} & 0 \\ 0 & 2e^{-2} \end{bmatrix}$ (mixed signs): a **saddle point**.

9.2 Convex Functions: When Local Means Global

A nagging worry: a local minimum might not be the *global* one. There is a clean class of functions for which the worry vanishes.

Definition: Convex function

f is **convex** if the line segment between any two points of its graph lies on or above the graph:

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}) \quad \text{for all } \mathbf{x}, \mathbf{y} \text{ and } t \in [0, 1].$$

Equivalent characterizations for smooth f : (i) the tangent plane always sits below the graph, $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x})$; (ii) the Hessian is positive semidefinite *everywhere* (all eigenvalues ≥ 0 at every point).

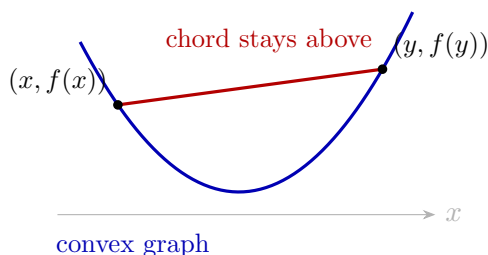


Figure 9.1: Convexity: every chord lies above the graph. Such a landscape has a single basin—any local minimum of a convex function is global, although the global minimizer need not be unique.

For a convex function, every local minimum is automatically global, although there may be more than one global minimizer. If f is *strictly* convex, then it has at most one minimizer. A positive-definite Hessian everywhere is a sufficient, but not necessary, condition for strict convexity; for example, $f(x) = x^4$ is strictly convex even though $f''(0) = 0$. Examples of convex functions include $\|\mathbf{x}\|^2$, e^x , $-\log x$ (on $x > 0$), and—crucially for what comes next—every least squares loss.

9.3 The Grand Crossover: Least Squares as Optimization

We now reach the moment Part I and Part II fuse together. (This section is the hinge of the course: it is the first place where the linear algebra of Part I reappears *inside* an optimization problem, and in class it is where the two halves of the syllabus stopped feeling like separate subjects.) The **least squares problem** asks: given a tall matrix $A \in \mathbb{R}^{m \times n}$ ($m > n$: more equations than unknowns) and a target $\mathbf{b}$, the system $A\mathbf{x} = \mathbf{b}$ generally has *no* exact solution. So find the $\mathbf{x}$ whose error is smallest:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \frac{1}{2} \|A\mathbf{x} - \mathbf{b}\|^2.$$

Deriving the gradient in slow motion. Expand the squared norm using $\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w}$:

$$\begin{aligned} f(\mathbf{x}) &= \frac{1}{2} (A\mathbf{x} - \mathbf{b})^T (A\mathbf{x} - \mathbf{b}) \\ &= \frac{1}{2} [\mathbf{x}^T A^T A \mathbf{x} - \mathbf{x}^T A^T \mathbf{b} - \mathbf{b}^T A \mathbf{x} + \mathbf{b}^T \mathbf{b}] \\ &= \frac{1}{2} \mathbf{x}^T (A^T A) \mathbf{x} - (A^T \mathbf{b})^T \mathbf{x} + \frac{1}{2} \|\mathbf{b}\|^2. \end{aligned}$$

(The two middle terms are equal—each is a number, and a number equals its own transpose.) Now apply the Week 7 identity table, with symmetric matrix $A^T A$ and constant vector $A^T \mathbf{b}$:

$$\nabla f(\mathbf{x}) = A^T A \mathbf{x} - A^T \mathbf{b}.$$

Setting the gradient to zero produces the celebrated

Key Idea: The Normal Equations

$$A^T A \mathbf{x} = A^T \mathbf{b}$$

Moreover, the Hessian is $H = A^T A$, which is positive definite whenever A has independent columns (because $\mathbf{x}^T A^T A \mathbf{x} = \|A\mathbf{x}\|^2 > 0$ for $\mathbf{x} \neq \mathbf{0}$). So the loss is a perfect convex bowl: the normal equations' solution is the unique *global* minimum, guaranteed.

The geometric meaning. The set of all possible predictions $A\mathbf{x}$ is the span of the columns of A (Week 1's column view)—a subspace W of $\mathbb{R}^m$. Minimizing $\|A\mathbf{x} - \mathbf{b}\|$ means finding the point of W closest to $\mathbf{b}$. By Week 3, the closest point is the *orthogonal projection* of $\mathbf{b}$ onto W , and the residual $\mathbf{e} = \mathbf{b} - A\mathbf{x}_*$ must be perpendicular to every column of A :

$$A^T \mathbf{e} = \mathbf{0} \iff A^T (\mathbf{b} - A\mathbf{x}_*) = \mathbf{0} \iff A^T A \mathbf{x}_* = A^T \mathbf{b}.$$

The normal equations are nothing but Week 3's perpendicularity condition. Calculus and geometry give the same answer—a reassuring consistency check, and the true meaning of “least squares is a projection.”

Example 9.3: Fitting the best line through data

Fit the line $y = c_1 + c_2 x$ to the four data points $(-1, 1)$, $(0, 0)$, $(1, 0)$, $(2, 2)$.

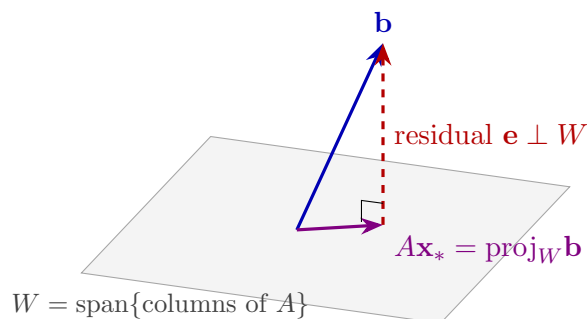


Figure 9.2: Least squares geometrically: the best prediction $A\mathbf{x}_*$ is the orthogonal projection of the data $\mathbf{b}$ onto the model subspace W ; the unavoidable error is perpendicular to everything the model can express.

Step 1: Set up $A\mathbf{c} \approx \mathbf{b}$. Each data point demands $c_1 + c_2x_i = y_i$:

$$A = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 2 \end{bmatrix}.$$

Four equations, two unknowns: no exact solution—perfect for least squares.

Step 2: Normal equations.

$$A^T A = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}, \quad A^T \mathbf{b} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

Step 3: Solve. $\det(A^T A) = 24 - 4 = \mathbf{20}$, so

$$\mathbf{c} = (A^T A)^{-1} A^T \mathbf{b} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 12 \\ 6 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.3 \end{bmatrix}.$$

The best-fit line is $y = 0.6 + 0.3x$. No other line achieves a smaller sum of squared vertical errors—the convex bowl guarantees it.

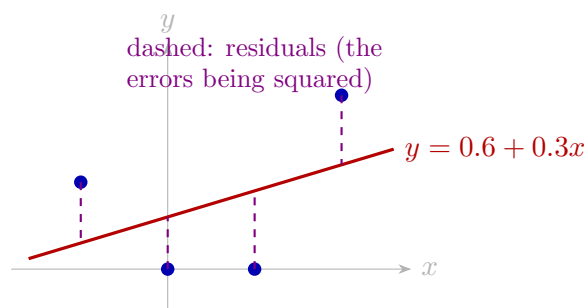


Figure 9.3: Example 9.3: least squares chooses the line minimizing the sum of the squared dashed lengths.

9.3.1 Polynomial Regression: Same Machinery, Richer Models

Nothing in the derivation required the model to be a straight line—only that it be *linear in the unknown coefficients*. To fit a parabola $p(x) = c_1 + c_2x + c_3x^2$ through the same data, simply add a column of x_i^2 values to A :

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \end{bmatrix},$$

and solve the same normal equations. For our four data points this yields

$$c_1 = -\frac{3}{20} = -0.15, \quad c_2 = -\frac{9}{20} = -0.45, \quad c_3 = \frac{3}{4} = 0.75,$$

i.e. $p(x) = -0.15 - 0.45x + 0.75x^2$, whose sum of squared residuals is 0.05—far below the line’s 2.30 (Figure 9.4). A richer model fits the *training* data more closely. The same trick fits any combination of basis functions: polynomials, sines and cosines, splines. This single template, “linear in the parameters + squared loss $\implies$ normal equations,” is the foundation of regression in statistics and ML.

Common Pitfall: A smaller training error is not the goal

The parabola’s training error (0.05) crushes the line’s (2.30), and a degree-3 polynomial through these four points would achieve error *exactly zero*. That does not make it a better model. A model rich enough to thread every data point also threads the *noise* in the data, and its predictions at new x -values can be far worse—a failure called **overfitting**. In machine learning practice one therefore evaluates a model on data *held out from the fit* (a validation or test set): as the model grows richer, the training error may decrease, but the held-out error eventually turns around and increases. Choosing the model complexity at that turning point is one of the central crafts of the field [6].

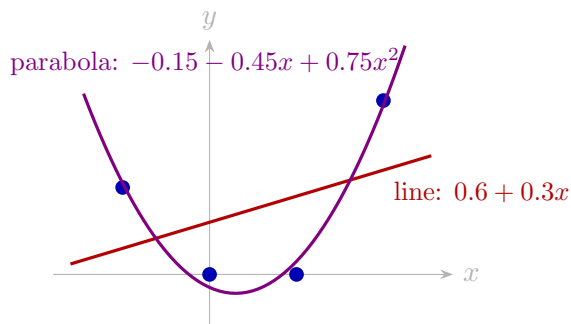


Figure 9.4: The same four data points fitted by the best line (squared error 2.30) and the best parabola (squared error 0.05). The richer model hugs the training data far more closely—which is a virtue only if the wiggles it introduces reflect signal rather than noise.

9.3.2 How to Solve the Normal Equations Well

In practice, forming $A^T A$ explicitly can amplify rounding errors. Two better routes, both using Part I’s factorizations:

- **Via QR.** If $A = QR$ is the *reduced* QR factorization of Week 4 (so $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns, $Q^T Q = I_n$, and $R \in \mathbb{R}^{n \times n}$ is upper triangular), then $A^T A = R^T Q^T Q R = R^T R$, and the normal equations collapse to the triangular system $R\mathbf{x} = Q^T \mathbf{b}$, solved by simple back-substitution.
- **Via SVD.** If $A = U\Sigma V^T$ (Week 5), the solution is $\mathbf{x} = V\Sigma^+ U^T \mathbf{b}$, where Σ^+ inverts the nonzero singular values. This route works even when the columns of A are dependent.

The lesson: Part I was not a separate subject. Its factorizations are precisely the tools that make Part II's optimization problems solvable.

9.4 Machine Learning Application

ML Link: Linear Regression and the MSE Loss

The least squares objective $\frac{1}{2}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$ is proportional to the **Mean Squared Error (MSE)** loss in machine learning, and minimizing it is **linear regression**—the first model in every ML course and still a workhorse for predicting continuous quantities (housing prices, demand forecasts, lab measurements). In ML notation, the rows of A are training examples' *features*, $\mathbf{b}$ holds their *labels*, and $\mathbf{x}$ is the model's *weight vector*.

Linear regression is also the rare ML model with a closed-form solution (the normal equations). The moment the model becomes nonlinear in its parameters—as every neural network is—no formula exists, and we must *iterate* our way downhill instead. That is exactly where Part III picks up the story.

ML Link: From Regression to Classification—Logistic Regression

Least squares predicts a *number*. The next-simplest ML task predicts a *yes/no label*: is this email spam? Does this image contain a cat? **Logistic regression** adapts the linear model by squashing its output through the **sigmoid function**,

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1),$$

so the model outputs a *probability* rather than a raw number. Instead of the squared error, one minimizes the **cross-entropy (negative log-likelihood) loss**

$$\ell(\mathbf{w}, b) = - \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right], \quad p_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b),$$

which heavily penalizes confident wrong answers. Two facts make logistic regression the perfect bridge from this chapter to Part III. First, the loss is *convex* in $(\mathbf{w}, b)$, so—as for least squares—any local minimum is global. Second, unlike least squares it has *no closed-form solution*: setting the gradient to zero produces a nonlinear system, so the model must be trained *iteratively*, by exactly the gradient methods of Weeks 10–12. Exercise 9.7 derives its remarkably tidy gradient, $(\sigma(\mathbf{w}^T \mathbf{x}) - y) \mathbf{x}$: “prediction error times input,” the same pattern as least squares.

9.5 End of Chapter Exercises

Week 9 Exercises

9.1 Find the best linear fit $y = c_1 + c_2x$ for the three data points $(-1, 0)$, $(0, 1)$, $(1, 3)$. Set up A and $\mathbf{b}$, form the normal equations, and solve for c_1, c_2 .

9.2 For the least squares problem with $A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$: apply Gram-Schmidt to the columns of A to obtain an orthogonal basis, project $\mathbf{b}$ onto each basis vector, and recover the least squares coefficients. Verify against the normal equations.

9.3 Find all critical points of $f(x, y) = x^3 - 3xy + y^3$, evaluate the Hessian at each, and classify them as local minimum, local maximum, or saddle point.

9.4 Show that $f(\mathbf{x}) = \frac{1}{2}\|A\mathbf{x} - \mathbf{b}\|^2$ is convex by showing its Hessian $A^T A$ satisfies $\mathbf{v}^T (A^T A) \mathbf{v} \geq 0$ for every $\mathbf{v}$.

9.5 Set up (but do not solve) the matrix A for fitting the parabola $p(x) = c_1 + c_2x + c_3x^2$ to the data of Example 9.3. What are its dimensions?

9.6 (Projection matrix) For a tall matrix A with independent columns, the best prediction is $A\mathbf{x}_* = A(A^T A)^{-1}A^T \mathbf{b}$, so

$$P = A(A^T A)^{-1}A^T$$

is the matrix that projects any $\mathbf{b}$ onto the column space of A . Verify algebraically the two signature properties of Week 3: $P^2 = P$ and $P^T = P$. (Use $(BC)^T = C^T B^T$ and $((A^T A)^{-1})^T = (A^T A)^{-1}$, which holds because $A^T A$ is symmetric.)

9.7 (Logistic-regression gradient) For a single data point $(\mathbf{x}, y)$ with $y \in \{0, 1\}$, the logistic loss can be written

$$\ell(\mathbf{w}) = \log(1 + e^{\mathbf{w}^T \mathbf{x}}) - y \mathbf{w}^T \mathbf{x}.$$

Using the chain rule and the Week 7 identity $\nabla_{\mathbf{w}}(\mathbf{w}^T \mathbf{x}) = \mathbf{x}$, show that

$$\nabla \ell(\mathbf{w}) = (\sigma(\mathbf{w}^T \mathbf{x}) - y) \mathbf{x}, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

(Hint: $\frac{d}{dz} \log(1 + e^z) = \frac{e^z}{1 + e^z} = \sigma(z)$.)

Exam Practice

Consider the function $f(x, y) = x^3 - 3xy + y^3$.

- Find all critical (stationary) points of $f(x, y)$.
- Evaluate the Hessian at each critical point and classify them as a local minimum, local

maximum, or saddle point.

(This is Exercise 9.3; the two-step recipe of Section 9.1 applies verbatim. Expect a polynomial system in step (a)—substitute one equation into the other.)

Part III

The Algorithmic Journey (Iterative and Constrained Optimization)

Chapter 10

Week 10: Norms, Iterations, and the Science of Convergence

10.1 The Intuition: Why We Must Walk, Not Teleport

In Week 9 we solved least squares *exactly*: set the gradient to zero, solve a linear system, done. But suppose the loss function belongs to a neural network with a billion parameters. Setting a billion-dimensional *nonlinear* gradient to zero produces a system no algebra can solve. Since we cannot teleport to the minimum, we must **walk** there: start from a guess $\mathbf{x}_0$ and repeatedly apply an update rule to generate $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots$, hoping the sequence settles down at the answer. This is an **iterative method**, and essentially all of modern machine learning training is one.

Two examples in one dimension show the idea (and a preview of how good iterations can be):

- **Computing $\sqrt{2}$.** The rule $x_{k+1} = \frac{1}{2}(x_k + \frac{2}{x_k})$ starting from $x_0 = 1$ produces

$$x_1 = 1.5, \quad x_2 = 1.41\overline{6}, \quad x_3 = 1.4142157\dots$$

Three steps already give $\sqrt{2} = 1.4142135\dots$ to four decimal places. (This is Newton’s method; we will see why it is so fast in Week 12.)

- **The golden ratio.** The rule $x_{k+1} = \sqrt{x_k + 1}$ from $x_0 = 0$ creeps steadily toward $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$, gaining roughly a constant factor of accuracy per step—reliable but slower.

The central questions of Part III: *When does an iteration converge? How fast? And how do we design fast ones?* Answering them requires a way to measure the *size* of errors—norms.

10.2 Vector Norms and Matrix Norms

Definition: Vector norms

A norm assigns a length $\|\mathbf{x}\| \geq 0$ to each vector, with $\|\mathbf{x}\| = 0$ only for $\mathbf{x} = \mathbf{0}$, $\|c\mathbf{x}\| = |c| \|\mathbf{x}\|$, and the triangle inequality $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. The three norms used in practice:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} \text{ (the usual length)}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|.$$

For example, $\mathbf{x} = [3, -4]^T$ has $\|\mathbf{x}\|_1 = 7$, $\|\mathbf{x}\|_2 = 5$, $\|\mathbf{x}\|_\infty = 4$. Saying “ $\mathbf{x}_k \rightarrow \mathbf{x}_*$ ” means $\|\mathbf{x}_k - \mathbf{x}_*\| \rightarrow 0$ (in any of these norms—they all agree about convergence).

How much can a matrix *amplify* a vector? That is the induced matrix norm:

Definition: Induced matrix norm

$$\|A\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} \quad (\text{the worst-case amplification factor}),$$

so that $\|A\mathbf{x}\| \leq \|A\| \|\mathbf{x}\|$ for every $\mathbf{x}$. Each vector norm induces its own matrix norm, and the three above have convenient formulas:

$$\|A\|_1 = \max_j \sum_i |a_{ij}| \quad (\text{max column sum}), \quad \|A\|_\infty = \max_i \sum_j |a_{ij}| \quad (\text{max row sum}),$$

$$\|A\|_2 = \sigma_{\max}(A),$$

the largest *singular value*—Week 5 strikes again. A related quantity is the **spectral radius** $\rho(A) = \max_i |\lambda_i(A)|$, the size of the largest eigenvalue. For every induced norm, $\rho(A) \leq \|A\|$.

Example 10.1: Computing the norms of one matrix

Let $A = \begin{bmatrix} 0 & 4 \\ 0 & 3 \end{bmatrix}$.

1-norm (max column sum): columns sum (in absolute value) to 0 and $4 + 3 = 7$, so $\|A\|_1 = 7$.

∞ -norm (max row sum): rows sum to 4 and 3, so $\|A\|_\infty = 4$.

2-norm: $A^T A = \begin{bmatrix} 0 & 0 \\ 0 & 25 \end{bmatrix}$ has eigenvalues 25 and 0, so $\|A\|_2 = \sqrt{25} = 5$.

Spectral radius: A is triangular, so its eigenvalues sit on the diagonal: 0 and 3. Thus $\rho(A) = 3$ —indeed smaller than all three norms, as the theory predicts.

10.3 Linear Iterations and the Convergence Theorem

The model iteration for analysis—and a surprisingly faithful model of gradient descent, as Week 11 will show—is the **linear vector iteration**

$$\mathbf{x}_{k+1} = G\mathbf{x}_k + \mathbf{b}.$$

If the iteration settles at a **fixed point** $\mathbf{x}_*$, that point satisfies $\mathbf{x}_* = G\mathbf{x}_* + \mathbf{b}$. Now subtract the two equations. Defining the error $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}_*$:

$$\mathbf{e}_{k+1} = \mathbf{x}_{k+1} - \mathbf{x}_* = (G\mathbf{x}_k + \mathbf{b}) - (G\mathbf{x}_* + \mathbf{b}) = G\mathbf{e}_k.$$

The constant $\mathbf{b}$ cancels completely: *the error obeys the pure matrix recursion* $\mathbf{e}_{k+1} = G\mathbf{e}_k$, hence $\mathbf{e}_k = G^k \mathbf{e}_0$. Taking norms,

$$\|\mathbf{e}_k\| = \|G^k \mathbf{e}_0\| \leq \|G\|^k \|\mathbf{e}_0\|.$$

Key Idea: The convergence test

- If $\|G\| < 1$ in *some* induced norm, then $\|\mathbf{e}_k\| \leq \|G\|^k \|\mathbf{e}_0\| \rightarrow 0$: the iteration **converges from every starting point**, and the next error is at most $\|G\|$ times the current error.

$$G^k \rightarrow 0 \iff \rho(G) < 1.$$

- If $\|G\| > 1$ in one norm, do not despair—another norm may still certify convergence.
- The sharp criterion uses the spectral radius:

$$\text{convergence for every } \mathbf{x}_0 \Leftarrow \rho(G) < 1.$$

Example 10.2: One norm fails, another succeeds

Let $G = \begin{bmatrix} 0.8 & 0.5 \\ 0.1 & 0.2 \end{bmatrix}$.

Try the 1-norm: column sums are 0.9 and 0.7, so $\|G\|_1 = \mathbf{0.9} < 1$. **Convergence guaranteed.**

Try the ∞ -norm: row sums are 1.3 and 0.3, so $\|G\|_\infty = \mathbf{1.3} > 1$ —inconclusive on its own. This shows why the theorem says “*some* norm”: one successful norm is enough, and here the 1-norm already settles the matter. The error shrinks by at least 10% per step (measured in the 1-norm).

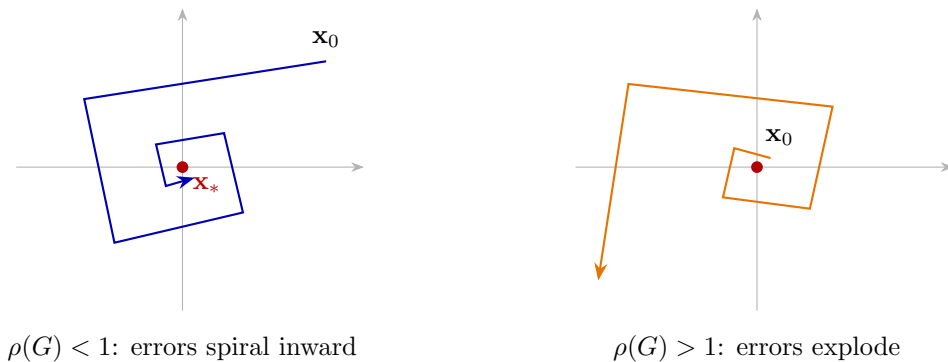


Figure 10.1: The fate of an iteration from a generic initial point is decided by the spectral radius of its error-propagation matrix G .

10.4 The Vocabulary of Optimization Problems

Before unleashing algorithms, we fix terminology. The generic problem is

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}),$$

and a point $\mathbf{x}_*$ with $f(\mathbf{x}_*) \leq f(\mathbf{x})$ for all $\mathbf{x}$ is a **minimizer**, written $\mathbf{x}_* = \arg \min_{\mathbf{x}} f(\mathbf{x})$. (Note the distinction: $\min f$ is the lowest *value*; $\arg \min f$ is the *location* where it occurs.) Two simplifications cost nothing:

$$\arg \min_{\mathbf{x}} (f(\mathbf{x}) + c) = \arg \min_{\mathbf{x}} f(\mathbf{x}), \quad \arg \min_{\mathbf{x}} (cf(\mathbf{x})) = \arg \min_{\mathbf{x}} f(\mathbf{x}) \quad (c > 0) :$$

adding a constant or rescaling by a positive number moves the landscape up or stretches it, but the bottom stays where it was. This is why ML practitioners freely use $\frac{1}{2}\|\cdot\|^2$ instead of $\|\cdot\|^2$ (the $\frac{1}{2}$ cleans up the gradient) and ignore additive constants in losses.

10.5 Machine Learning Application

ML Link: Training Is an Iteration; Norms Are the Dashboard

A progress bar you often see on a model diagnostic run is a plot of an error norm versus iteration count k . The theory of this chapter is the reason those plots are read on a *logarithmic* axis: when $\|\mathbf{e}_k\| \approx \|G\|^k \|\mathbf{e}_0\|$, the log of the error falls on a straight line whose slope is $\log \|G\|$. A steeper line means a smaller $\|G\|$ —a faster algorithm. In Week 11 we compute exactly what that slope is for gradient descent, and discover which single number of the problem controls it.

10.6 End of Chapter Exercises

Week 10 Exercises

10.1 For the iteration $\mathbf{x}_{k+1} = G\mathbf{x}_k + \mathbf{b}$ with $G = \begin{bmatrix} 0.3 & 0.6 \\ 0.3 & 0.5 \end{bmatrix}$, compute $\|G\|_\infty$ (maximum absolute row sum). Will the iteration converge for every starting point? Justify.

10.2 For $A = \begin{bmatrix} 0 & 4 \\ 0 & 3 \end{bmatrix}$, verify the values computed in Example 10.1: $\|A\|_1$, $\|A\|_\infty$, $\|A\|_2$, and $\rho(A)$.

10.3 For the nonlinear iteration $\mathbf{x}_{k+1} = F(\mathbf{x}_k)$ with $F(x, y) = \begin{bmatrix} \frac{1}{3} \arctan x + \frac{1}{4} y \\ \frac{1}{2} x - \frac{1}{4} \arctan y \end{bmatrix}$, compute the Jacobian $J(x, y)$. (In Week 12 we will use $\|J\|$ to decide convergence of such nonlinear iterations.)

10.4 Compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$, and $\|\mathbf{x}\|_\infty$ for $\mathbf{x} = [1, -2, 2]^T$. Which is largest? (It always orders this way: $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$.)

10.5 (*Theoretical*) Show that the fixed point of $\mathbf{x}_{k+1} = G\mathbf{x}_k + \mathbf{b}$ is $\mathbf{x}_* = (I - G)^{-1}\mathbf{b}$ whenever $I - G$ is invertible, by solving $\mathbf{x}_* = G\mathbf{x}_* + \mathbf{b}$.

Chapter 11

Week 11: Gradient Descent and the Tyranny of the Condition Number

11.1 The Intuition: How Big a Step Dare We Take?

Gradient descent walks downhill:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k),$$

with learning rate $\alpha > 0$. If α is tiny, training takes forever. If α is too large, each step overshoots the valley floor, landing higher on the opposite wall—and the iteration explodes. Somewhere in between is a safe range and an optimal choice. This chapter computes both *exactly*, for the cleanest possible landscape: a quadratic bowl. The answer will hand control to two familiar numbers—the extreme eigenvalues of the Hessian.

11.2 Gradient Descent on a Quadratic Bowl: The Full Analysis

Take the model loss

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}, \quad A \text{ symmetric positive definite,}$$

whose gradient (Week 7 table) is $\nabla f = A\mathbf{x} - \mathbf{b}$ and whose unique minimizer satisfies $A\mathbf{x}_* = \mathbf{b}$. (Least squares has exactly this form with $A \rightarrow A^T A$; and by Taylor, *every* smooth loss looks like this near its minimum—which is why this analysis matters far beyond quadratics.)

Step 1: Gradient descent is a linear iteration.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha(A\mathbf{x}_k - \mathbf{b}) = \underbrace{(I - \alpha A)}_G \mathbf{x}_k + \alpha \mathbf{b}.$$

This is exactly Week 10's setup with $G = I - \alpha A$! Therefore the error obeys $\mathbf{e}_{k+1} = (I - \alpha A) \mathbf{e}_k$, and convergence is decided by $\rho(I - \alpha A)$.

Step 2: Eigenvalues of G from eigenvalues of A . If $A\mathbf{v} = \lambda\mathbf{v}$, then $(I - \alpha A)\mathbf{v} = (1 - \alpha\lambda)\mathbf{v}$: same eigenvectors, shifted eigenvalues. So

$$\rho(G) = \max_{\lambda \in \{\lambda_1, \dots, \lambda_n\}} |1 - \alpha\lambda|,$$

and we need every $|1 - \alpha\lambda_i| < 1$, i.e. $-1 < 1 - \alpha\lambda_i < 1$, i.e. $0 < \alpha\lambda_i < 2$ for each eigenvalue. The binding constraint comes from the largest one:

Key Idea: The learning-rate speed limit

Gradient descent on a quadratic with Hessian A converges if and only if

$$0 < \alpha < \frac{2}{\lambda_{\max}(A)}$$

Violate it and the component of the error along the top eigenvector is *multiplied* by $|1 - \alpha\lambda_{\max}| > 1$ every step: guaranteed divergence. Two sub-regimes are worth knowing: $\alpha < 1/\lambda_{\max}$ gives smooth, monotone decay (all $1 - \alpha\lambda_i \geq 0$), while $1/\lambda_{\max} < \alpha < 2/\lambda_{\max}$ still converges but the iterates *oscillate* across the valley as they descend.

Example 11.1: Finding the safe learning rate

For $f(x, y) = \frac{5}{2}x^2 - 3xy + \frac{5}{2}y^2$, what learning rates are safe?

Step 1: Hessian. $\nabla f = \begin{bmatrix} 5x - 3y \\ -3x + 5y \end{bmatrix}$, so $H = \begin{bmatrix} 5 & -3 \\ -3 & 5 \end{bmatrix}$.

Step 2: Eigenvalues. $\det(H - \lambda I) = (5 - \lambda)^2 - 9 = 0 \implies 5 - \lambda = \pm 3 \implies \lambda = \mathbf{2, 8}$.

Step 3: Speed limit.

$$0 < \alpha < \frac{2}{\lambda_{\max}} = \frac{2}{8} = \frac{1}{4}.$$

A practitioner who sets $\alpha = 0.3$ will watch the loss blow up; one who sets $\alpha = 0.01$ will converge but waste compute. The sweet spot is computed next.

11.3 The Optimal Rate and the Condition Number

Within the safe range, which α shrinks the error fastest? We must minimize the worst factor $\max_i |1 - \alpha\lambda_i|$, and the extremes are attained at $\lambda_{\min}$ and $\lambda_{\max}$. The best α balances the two ($1 - \alpha\lambda_{\min} = -(1 - \alpha\lambda_{\max})$), giving

$$\alpha_{\text{opt}} = \frac{2}{\lambda_{\min} + \lambda_{\max}}, \quad \rho(G)|_{\text{opt}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}, \quad \kappa := \frac{\lambda_{\max}}{\lambda_{\min}}.$$

The single number $\kappa \geq 1$ is the **condition number** of the Hessian, and it tyrannizes gradient descent:

$$\|\mathbf{e}_{k+1}\| \leq \frac{\kappa - 1}{\kappa + 1} \|\mathbf{e}_k\|.$$

- $\kappa = 1$ (perfectly round bowl): factor 0—one step lands exactly at the minimum.
- $\kappa = 5$: factor $4/6 \approx 0.67$; about 69 steps to shrink the error by 10^{12} .
- $\kappa = 50$: factor $49/51 \approx 0.96$; about 691 steps for the same accuracy.
- $\kappa = 200$: factor $199/201 \approx 0.99$; about 2764 steps.

(These step counts come from solving $(\frac{\kappa-1}{\kappa+1})^k < 10^{-12}$ for k with logarithms: $k > 12 \ln 10 / \ln \frac{\kappa+1}{\kappa-1}$. Exercise 11.4 runs the same computation for $\kappa = 100$.) Geometrically, large κ means the level curves are long thin ellipses—a narrow canyon. The gradient points nearly perpendicular to the canyon walls rather than along its floor, so the iterates zigzag wastefully (Figure 11.1).

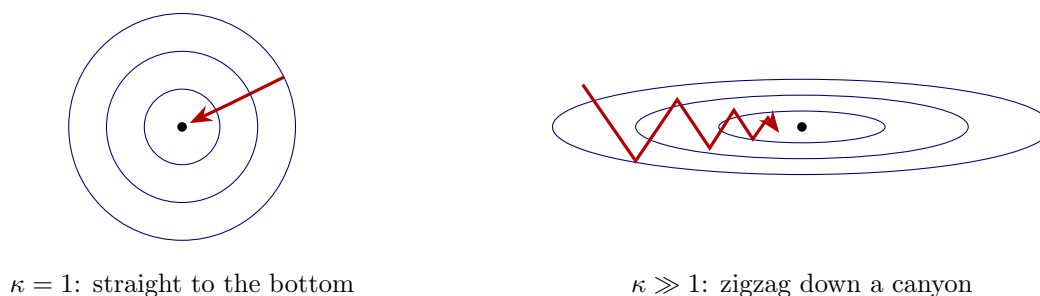


Figure 11.1: Gradient descent on a round bowl versus an ill-conditioned canyon. The gradient is perpendicular to the level curves, so in a canyon it points across the valley, not along it.

11.4 Convergence of the Loss Values

Practitioners watch the *loss*, not the parameter error. The two are linked. Writing f exactly (no approximation—the function is quadratic) around the minimizer:

$$f(\mathbf{x}_k) - f(\mathbf{x}_*) = \frac{1}{2} \mathbf{e}_k^T A \mathbf{e}_k \leq \frac{\lambda_{\max}}{2} \|\mathbf{e}_k\|^2.$$

Since $\|\mathbf{e}_k\|$ shrinks geometrically with factor $\rho = \frac{\kappa-1}{\kappa+1}$, the loss gap shrinks with factor ρ^2 —loss curves fall twice as fast (on a log scale) as parameter errors, a useful fact when reading training plots.

11.5 Machine Learning Application

ML Link: Why Data Normalization Is Helpful?

Where do huge condition numbers come from in practice? From *mismatched feature scales*. Suppose a model uses “age in years” (range ~ 0 –100) and “income in dollars” (range ~ 0 –100,000) as inputs. In the least squares Hessian $A^T A$, the income feature contributes entries roughly 10^6 times larger than the age feature—producing κ in the millions and a hopeless canyon.

The standard mitigation, **feature normalization** (rescale every feature to mean 0, standard deviation 1, before training), is nothing mysterious: it is preconditioning—reshaping the bowl to push κ toward 1 so that the $\frac{\kappa-1}{\kappa+1}$ contraction factor becomes small. Batch normalization layers inside deep networks pursue the same goal at every layer. One inequality from this chapter explains an entire family of ML engineering practices.

ML Link: Ridge Regression—Buying a Better Condition Number

There is a second, purely algebraic way to tame κ : **regularization**. **Ridge regression** adds a penalty on the size of the weights to the least squares loss,

$$\min_{\mathbf{x}} \frac{1}{2} \|A\mathbf{x} - \mathbf{b}\|^2 + \frac{\lambda}{2} \|\mathbf{x}\|^2, \quad \lambda > 0,$$

whose gradient is $A^T(A\mathbf{x} - \mathbf{b}) + \lambda\mathbf{x}$ (Exercise 11.6), giving the modified normal equations

$$(A^T A + \lambda I) \mathbf{x} = A^T \mathbf{b}.$$

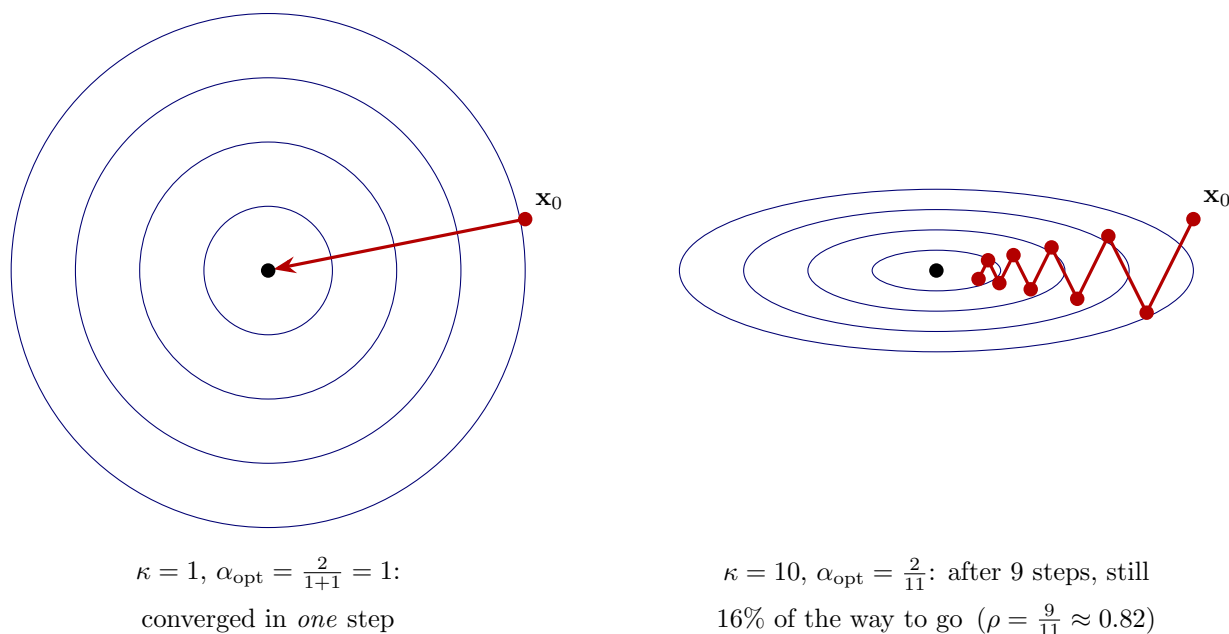


Figure 11.2: Actual gradient-descent iterates (not a sketch). Left: for $f = \frac{1}{2}(x^2 + y^2)$, whose Hessian has $\kappa = 1$, the optimal step $\alpha = 1$ reaches the minimum exactly in one move. Right: for $f = \frac{1}{2}(x^2 + 10y^2)$ with $\kappa = 10$ and the optimal $\alpha = 2/11$, both error components shrink by exactly $\rho = \frac{\kappa-1}{\kappa+1} = \frac{9}{11}$ per step, but the steep component flips sign each time—producing the zigzag down the canyon’s long axis. The nine plotted steps reduce the error by the predicted factor $(9/11)^9 \approx 0.16$. Reproduce this plot in NumPy: iterate $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \text{diag}(1, 10) \mathbf{x}_k$ from $\mathbf{x}_0 = (10, 2)$.

Adding λI shifts *every* eigenvalue of the Hessian up by λ : the condition number drops from $\sigma_{\max}^2/\sigma_{\min}^2$ to $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda)$, a dramatic improvement when $\sigma_{\min}$ is tiny—and the system becomes solvable even when the columns of A are dependent ($\sigma_{\min} = 0$). The price is a bias: the penalty pulls the solution slightly toward $\mathbf{0}$. Ridge regression thus connects four threads of this course at once—least squares (Week 9), eigenvalue shifting (Week 4), conditioning (this week), and the bias toward simpler models that combats overfitting (Week 9’s pitfall box).

11.6 End of Chapter Exercises

Week 11 Exercises

11.1 A quadratic cost has Hessian $H = \begin{bmatrix} 8 & 4 \\ 4 & 8 \end{bmatrix}$. Find both eigenvalues, the safe learning-rate range, the optimal α , and the condition number κ .

11.2 For $f(x) = (x - 4)^2 \cdot \frac{1}{2}$ (so $f'(x) = x - 4$), run gradient descent with $\alpha = 0.1$ from $x_0 = 0$ for two steps by hand. Then determine: for which α does the iteration $x_{k+1} = x_k - \alpha(x_k - 4)$ converge?

11.3 For the least squares loss $f(\mathbf{x}) = \frac{1}{2}\|\mathbf{Ax} - \mathbf{b}\|^2$, gradient descent reads $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha A^T(\mathbf{Ax}_k - \mathbf{b})$. Show this has the form $\mathbf{x}_{k+1} = G\mathbf{x}_k + \mathbf{c}$ with $G = I - \alpha A^T A$, and state the convergence condition in terms of $\sigma_{\max}(A)$. (Answer to aim for: $0 < \alpha < 2/\sigma_{\max}(A)^2$, since the eigenvalues of $A^T A$ are the squared singular values of A . If A has full column rank, the iteration then converges to the unique least squares minimizer; if A is rank-deficient, the loss still decreases under this condition, but the minimizer is no longer unique.)

11.4 Using the contraction factor $\frac{\kappa-1}{\kappa+1}$, estimate how many iterations are needed to reduce the error by a factor of 10^6 when $\kappa = 100$. (Hint: solve $\rho^k = 10^{-6}$ with logarithms.)

11.5 Explain in one paragraph, using the contour picture, why normalizing features before training a regression model speeds up gradient descent.

11.6 (Ridge regression) (a) Using the Week 7 identity table, show that the gradient of $f(\mathbf{x}) = \frac{1}{2}\|\mathbf{Ax} - \mathbf{b}\|^2 + \frac{\lambda}{2}\|\mathbf{x}\|^2$ is $A^T(\mathbf{Ax} - \mathbf{b}) + \lambda\mathbf{x}$, and derive the modified normal equations $(A^T A + \lambda I)\mathbf{x} = A^T \mathbf{b}$. (b) If $A^T A$ has eigenvalues $\mu_1 \geq \dots \geq \mu_n > 0$, show that the Hessian $A^T A + \lambda I$ has eigenvalues $\mu_i + \lambda$ (same eigenvectors), and conclude that its condition number $\frac{\mu_1 + \lambda}{\mu_n + \lambda}$ is smaller than $\frac{\mu_1}{\mu_n}$ whenever $\mu_n < \mu_1$.

Chapter 12

Week 12: Nonlinear Iterations, Newton's Method, and Momentum

12.1 The Intuition: Beyond Plain Gradient Descent

Plain gradient descent is reliable but, as we just saw, painfully slow in a canyon. This chapter builds two upgrades. **Newton's method** uses curvature (the Hessian) to jump nearly straight to the bottom—blazingly fast, but expensive per step. **Momentum** costs almost nothing extra per step yet provably tames the canyon, replacing the dependence on κ by one on $\sqrt{\kappa}$. Before studying these faster methods, we begin with bisection, a slower one-dimensional root-finding method whose reliability is almost impossible to beat. We then develop a general theory of nonlinear iterations to measure “fast” against.

12.2 Root Finding: Bisection versus Newton

Many optimization problems lead to a root-finding problem. For example, a one-variable minimizer often satisfies $f'(x) = 0$. The **bisection method** finds a root without using derivatives.

Suppose g is continuous on $[a_0, b_0]$ and

$$g(a_0)g(b_0) < 0.$$

Because the endpoint values have opposite signs, the Intermediate Value Theorem guarantees that g has a root x_* inside the interval.

Key Idea: The bisection method

Given an interval $[a_k, b_k]$ containing a sign change, compute its midpoint

$$m_k = \frac{a_k + b_k}{2}.$$

If $g(m_k) = 0$, we have found the root. Otherwise retain the half containing the sign change:

$$[a_{k+1}, b_{k+1}] = \begin{cases} [a_k, m_k], & g(a_k)g(m_k) < 0, \\ [m_k, b_k], & g(m_k)g(b_k) < 0. \end{cases}$$

Each step cuts the search interval in half. After k interval reductions,

$$b_k - a_k = \frac{b_0 - a_0}{2^k}.$$

Using the midpoint $x_k = (a_k + b_k)/2$ as the approximation gives

$$|x_k - x_*| \leq \frac{b_k - a_k}{2} = \frac{b_0 - a_0}{2^{k+1}}.$$

Thus, to guarantee $|x_k - x_*| \leq \epsilon$, it is enough to take

$$k \geq \left\lceil \log_2 \left(\frac{b_0 - a_0}{2\epsilon} \right) \right\rceil.$$

Example 12.1: Finding $\sqrt{2}$ by bisection

To solve $x^2 = 2$, let

$$g(x) = x^2 - 2.$$

Since $g(1) = -1$ and $g(2) = 2$, the root $\sqrt{2}$ lies in $[1, 2]$. Successive midpoint tests give

k	$[a_k, b_k]$	m_k	$\text{sign}(g(m_k))$
0	$[1, 2]$	1.5	+
1	$[1, 1.5]$	1.25	−
2	$[1.25, 1.5]$	1.375	−
3	$[1.375, 1.5]$	1.4375	+

Therefore,

$$\sqrt{2} \in [1.375, 1.4375].$$

The midpoint of this final interval is 1.40625, and the guaranteed error is at most

$$\frac{1.4375 - 1.375}{2} = 0.03125.$$

Further steps continue to halve this bound.

Bisection is slow but robust: continuity and an initial sign-changing interval are enough to guarantee convergence. Newton's root-finding update,

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)},$$

is usually much faster near a simple root and is quadratically convergent under suitable smoothness assumptions. However, it requires derivatives and a sufficiently good starting point; a poor initial guess can cause it to move away from the desired root. Bisection provides certainty, whereas Newton provides speed when its assumptions and starting-point requirements hold.

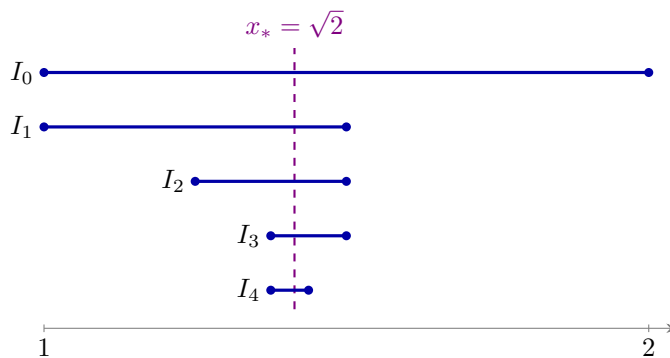


Figure 12.1: Bisection repeatedly retains the half interval containing the sign change. For $g(x) = x^2 - 2$, the nested intervals remain guaranteed to contain $x_* = \sqrt{2}$, while their widths decrease by a factor of two at every step.

Short coding exercise: Bisection

Write a function that takes g , a , b , and a tolerance ϵ as inputs and applies bisection until $(b - a)/2 \leq \epsilon$. The function should return the midpoint and the number of iterations. Test it on

$$g(x) = x^2 - 2, \quad [a, b] = [1, 2], \quad \epsilon = 10^{-6},$$

and compare the computed error with the theoretical bound.

12.3 Fixed Points, Contractions, and Rates of Convergence

A general iteration has the form $\mathbf{x}_{k+1} = F(\mathbf{x}_k)$ for some map F . If it settles at $\mathbf{x}_*$, then $\mathbf{x}_* = F(\mathbf{x}_*)$: a **fixed point**. How does the error evolve? Linearize F around $\mathbf{x}_*$ (Week 8's machinery, applied to the map itself):

$$\mathbf{e}_{k+1} = F(\mathbf{x}_k) - F(\mathbf{x}_*) \approx J_F(\mathbf{x}_*) \mathbf{e}_k,$$

where J_F is the Jacobian. Near the fixed point, a nonlinear iteration behaves like the linear iteration $\mathbf{e}_{k+1} = J_F(\mathbf{x}_*)\mathbf{e}_k$, so Week 10's test applies verbatim:

$$\rho(J_F(\mathbf{x}_*)) < 1 \implies \text{local convergence.}$$

A map with $\|J_F\| \leq \rho < 1$ everywhere is called a **contraction**: it shrinks all distances, and the iteration converges from any start, with $\|\mathbf{e}_k\| \leq \rho^k \|\mathbf{e}_0\|$.

Definition: Linear vs. quadratic convergence

- **Linear convergence:** $\|\mathbf{e}_{k+1}\| \approx \rho \|\mathbf{e}_k\|$ with $0 < \rho < 1$. Each step adds a fixed number of correct digits. (This happens when $J_F(\mathbf{x}_*) \neq 0$ with norm less than 1.)
- **Quadratic convergence:** $\|\mathbf{e}_{k+1}\| \approx C \|\mathbf{e}_k\|^2$. Each step *doubles* the number of correct digits: errors go $10^{-1} \rightarrow 10^{-2} \rightarrow 10^{-4} \rightarrow 10^{-8}$. This is the jackpot, and it happens when $J_F(\mathbf{x}_*) = 0$ —when the linear term of the error recursion vanishes, leaving only the second-order term of the Taylor expansion.

Example 12.2: Tuning an iteration for quadratic convergence

Consider solving $x^2 = 9$ via the iteration $x_{k+1} = F(x_k) = x_k - a(x_k^2 - 9)$, where a is a design parameter.

Fixed point: $x_* = F(x_*)$ requires $a(x_*^2 - 9) = 0$, so the positive fixed point is $x_* = \mathbf{3}$, for any $a \neq 0$.

Convergence rate: $F'(x) = 1 - 2ax$, so $F'(3) = 1 - 6a$. Convergence near 3 needs $|1 - 6a| < 1$, i.e. $0 < a < \frac{1}{3}$ — and the magic choice

$$F'(3) = 0 \iff a = \frac{1}{6}$$

upgrades the iteration to **quadratic** convergence. With $a = \frac{1}{6}$ the rule becomes $x_{k+1} = x_k - \frac{x_k^2 - 9}{6}$, and starting from $x_0 = 2$: $x_1 = 2.8\bar{3}$, $x_2 = 2.9954$, $x_3 = 2.999996$. Digits double each step. This tuned fixed-point iteration has the same local feature that makes Newton's method fast: the linear term of the error recursion vanishes at the root ($F'(x_*) = 0$). It is not exactly Newton's method, however. Newton's method for $g(x) = x^2 - 9 = 0$ divides by the *current* derivative $g'(x_k) = 2x_k$ rather than by the constant 6:

$$x_{k+1} = x_k - \frac{x_k^2 - 9}{2x_k}.$$

Our iteration “freezes” that denominator at its value $2x_* = 6$ at the root—which is enough for quadratic convergence *near* the root, but requires knowing where the root is to tune a . Newton's method, explained next in general, achieves the same effect with no tuning at all.

12.4 Newton's Method for Optimization

To minimize f , Week 8 taught us to trust the local quadratic model. At the current iterate $\mathbf{x}_k$,

$$f(\mathbf{x}) \approx Q(\mathbf{x}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T(\mathbf{x} - \mathbf{x}_k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^T H(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k).$$

Rather than stepping a cautious α downhill, jump straight to the *minimum of the model*: setting $\nabla Q = \mathbf{0}$ gives $H(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k) = -\nabla f(\mathbf{x}_k)$, i.e.

Key Idea: Newton's method

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k).$$

Compare with gradient descent $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$: Newton replaces the scalar α by the matrix H^{-1} , automatically taking long steps along flat directions and short steps along steep ones—a built-in cure for bad conditioning. Near a minimum with positive definite Hessian, convergence is *quadratic*. The price: computing, storing, and inverting H , which costs $O(n^2)$ memory and $O(n^3)$ work—fine for n in the thousands, impossible for n in the billions.

Example 12.3: One Newton step by hand

Minimize $f(x, y) = \frac{1}{3}x^3 + xy + y^2$, starting from $\mathbf{x}_0 = [1, 0]^T$.

Step 1: Gradient and Hessian.

$$\nabla f = \begin{bmatrix} x^2 + y \\ x + 2y \end{bmatrix}, \quad H = \begin{bmatrix} 2x & 1 \\ 1 & 2 \end{bmatrix}.$$

At $\mathbf{x}_0 = [1, 0]^T$: $\nabla f(\mathbf{x}_0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $H(\mathbf{x}_0) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

Step 2: Invert the Hessian. $\det H = 4 - 1 = 3$, so $H^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$.

Step 3: The Newton update.

$$\mathbf{x}_1 = \mathbf{x}_0 - H^{-1} \nabla f(\mathbf{x}_0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2/3 \\ -1/3 \end{bmatrix}.$$

(The point $[\frac{1}{2}, -\frac{1}{4}]^T$ is the local minimizer near the starting point—not a global one, since $f \rightarrow -\infty$ as $x \rightarrow -\infty$. The next three errors are approximately 3.73×10^{-2} , 2.19×10^{-3} , and 8.53×10^{-6} . Thus two additional steps bring the iterate within 10^{-2} , and one further step brings it within 10^{-4} , illustrating the eventual quadratic convergence.)

12.5 The Momentum Method: A Heavy Ball Rolling Downhill

Newton is too expensive for deep learning; plain gradient descent is too slow. **Momentum** (Polyak's heavy-ball method) is the budget upgrade: remember the previous step and keep some of it,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) + \beta (\mathbf{x}_k - \mathbf{x}_{k-1}), \quad 0 \leq \beta < 1.$$

The physical picture: a heavy ball rolling down the loss surface with friction $1 - \beta$. In a long flat corridor it accumulates speed (successive gradients pointing the same way reinforce); across a narrow canyon it damps the zigzag (alternating gradient components cancel against the remembered velocity). The intuition is exactly right, and the analysis makes it quantitative: applying the linearization machinery to the pair $(\mathbf{x}_k, \mathbf{x}_{k-1})$ on a quadratic with condition number κ , the optimal tuning works out to

$$\alpha = \left(\frac{2}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}} \right)^2, \quad \beta = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^2,$$

with contraction factor per step

$$\rho_{\text{momentum}} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \approx 1 - \frac{2}{\sqrt{\kappa}} \quad \text{versus} \quad \rho_{\text{GD}} = \frac{\kappa - 1}{\kappa + 1} \approx 1 - \frac{2}{\kappa}.$$

The headline: momentum replaces κ by $\sqrt{\kappa}$. On a canyon with $\kappa = 10^4$, gradient descent needs on the order of 10^4 steps per digit of accuracy; momentum needs on the order of 10^2 —a hundredfold speedup for two extra lines of code.

Example 12.4: Calculating a momentum step

Minimize $f(x, y) = 2x^2 + y^2$ with $\alpha = 0.1$, $\beta = 0.5$, given the previous position $\mathbf{x}_0 = [2, 2]^T$ and current position $\mathbf{x}_1 = [1, 1]^T$. Find $\mathbf{x}_2$.

Step 1: Gradient at the current point. $\nabla f = \begin{bmatrix} 4x \\ 2y \end{bmatrix}$, so $\nabla f(1, 1) = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$.

Step 2: Velocity (the remembered step).

$$\mathbf{x}_1 - \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}.$$

Step 3: Combine.

$$\begin{aligned} \mathbf{x}_2 &= \mathbf{x}_1 - \alpha \nabla f(\mathbf{x}_1) + \beta(\mathbf{x}_1 - \mathbf{x}_0) \\ &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} 4 \\ 2 \end{bmatrix} + 0.5 \begin{bmatrix} -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 - 0.4 - 0.5 \\ 1 - 0.2 - 0.5 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.3 \end{bmatrix}. \end{aligned}$$

The remembered velocity carried the iterate substantially closer to the minimum at the origin than the gradient alone would have.

12.6 Machine Learning Application

ML Link: The Optimizers Menu in Deep Learning

Open any deep learning framework and you will choose an “optimizer” from a menu: SGD, SGD+momentum, Adam, ... You now know what the menu means.

- SGD is gradient descent; the “S” stands for *stochastic*. The full loss is an average over N training examples, so the exact gradient is itself an average,

$$\nabla f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\mathbf{x}),$$

which is expensive when N is millions. SGD replaces it each step by the average over a small random **mini-batch** B_k of examples,

$$\mathbf{g}_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla \ell_i(\mathbf{x}_k),$$

and updates $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{g}_k$. Because each example is equally likely to be sampled, $\mathbf{g}_k$ is an *unbiased* estimate of the true gradient: correct on average, noisy in any one step. The result is a cheaper, noisier walk downhill whose average behavior follows the mathematics of Week 11.

- **momentum** is this chapter’s heavy ball [12], with a typical default $\beta = 0.9$. A word of caution about interpreting that default: if the tuning rule $\beta = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$ were being applied, $\beta = 0.9$ would correspond to $\kappa \approx 1440$ (whereas a *contraction factor* of $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 0.9$ corresponds to $\kappa \approx 361$ —two different statements that are easy to conflate). In practice $\beta = 0.9$ is simply a robust default rather than a statement about any particular κ .
- **Adam** [11] combines momentum with adaptive per-coordinate scaling based on running averages of past gradients and squared gradients. The effect resembles a *diagonal*

preconditioner—each coordinate gets its own effective learning rate—although it is not a true Hessian inverse and captures no cross-coordinate curvature.

Full Newton remains absent from the menu for the reason computed in Week 8: a billion-parameter Hessian has 10^{18} entries. Each optimizer on the menu is a different compromise between gradient descent's cheapness and Newton's geometry.

12.7 End of Chapter Exercises

Week 12 Exercises

12.1 Consider the fixed-point iteration

$$x_{k+1} = x_k - a(e^{x_k} - 2).$$

Find the fixed point $x_* > 0$, and determine the value of a for which the linear error term vanishes, so that the iteration has quadratic convergence near x_* .

12.2 For

$$f(x, y) = \frac{1}{4}x^4 + \frac{1}{2}(x - y)^2 + y^2,$$

compute the gradient and Hessian, and perform one Newton step starting from $\mathbf{x}_0 = [1, 0]^T$.

12.3 Verify that the iteration $x_{k+1} = \frac{1}{2}(x_k + \frac{2}{x_k})$ for computing $\sqrt{2}$ is Newton's method applied to solving $g(x) = x^2 - 2 = 0$ via $x_{k+1} = x_k - g(x_k)/g'(x_k)$.

12.4 With $\kappa = 100$: compute the contraction factors $\frac{\kappa-1}{\kappa+1}$ (gradient descent) and $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ (momentum), and the approximate ratio of iteration counts the two methods need for the same accuracy.

12.5 Continue Example 12.4 one more step: using $\mathbf{x}_1 = [1, 1]^T$ and $\mathbf{x}_2 = [0.1, 0.3]^T$, compute $\mathbf{x}_3$ with the same α, β .

Chapter 13

Week 13: Equality Constraints and Lagrange Multipliers

13.1 The Intuition: Optimizing While Obeying Rules

Until now our algorithms roamed freely over the whole landscape. Real problems come with rules: minimize cost *while producing exactly 1,000 units*; maximize a quadratic form *while keeping the vector unit length* (PCA!); choose probabilities *that sum exactly to one*. The template is

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) = 0,$$

where the **constraint** $g(\mathbf{x}) = 0$ carves out a curve or surface we must stay on.

The geometric key. Suppose that f and g are differentiable and that $\mathbf{x}_*$ is a local optimum subject to $g(\mathbf{x}) = 0$. We also require

$$\nabla g(\mathbf{x}_*) \neq \mathbf{0}.$$

This **equality-constraint qualification** ensures that the constraint forms a smooth curve or surface near $\mathbf{x}_*$. Walk along that surface and watch the level curves of f go by. At an optimum, the level curve of f must be tangent to the constraint, so their normal vectors are parallel.

Key Idea: The Lagrange condition

If $g(\mathbf{x}_*) = 0$ and $\nabla g(\mathbf{x}_*) \neq \mathbf{0}$, then a constrained local optimum satisfies

$$\nabla f(\mathbf{x}_*) = \lambda \nabla g(\mathbf{x}_*)$$

for some scalar λ , called a **Lagrange multiplier**. Equivalently, $\mathbf{x}_*$ and λ are stationary points of the **Lagrangian**

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda g(\mathbf{x}) : \quad \nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}, \quad \partial_{\lambda} \mathcal{L} = 0.$$

For several equality constraints

$$g_1(\mathbf{x}) = \cdots = g_m(\mathbf{x}) = 0,$$

the corresponding qualification is that $\nabla g_1(\mathbf{x}_*), \dots, \nabla g_m(\mathbf{x}_*)$ are linearly independent. Then

$$\nabla f(\mathbf{x}_*) = \sum_{j=1}^m \lambda_j \nabla g_j(\mathbf{x}_*).$$

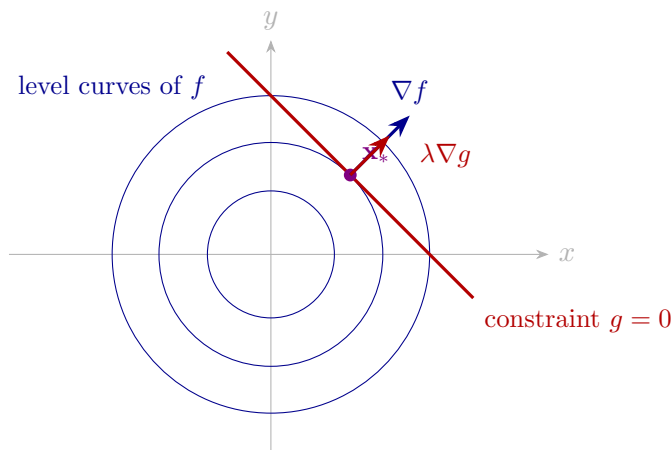


Figure 13.1: At the constrained optimum, the level curve of f is tangent to the constraint curve, so their gradients line up: $\nabla f = \lambda \nabla g$. Anywhere else the level curve crosses the constraint, and sliding along the constraint still improves f .

Example 13.1: Warm-up on the unit circle

Minimize and maximize $f(x, y) = xy$ subject to $x^2 + y^2 = 1$.

Step 1: Lagrangian and stationarity. $\mathcal{L} = xy - \lambda(x^2 + y^2 - 1)$.

$$\mathcal{L}_x = y - 2\lambda x = 0, \quad \mathcal{L}_y = x - 2\lambda y = 0, \quad x^2 + y^2 = 1.$$

Step 2: Solve. From the first two equations, $y = 2\lambda x$ and $x = 2\lambda y = 4\lambda^2 x$. Since $x = 0$ would force $y = 0$ (violating the constraint), we need $4\lambda^2 = 1$, i.e. $\lambda = \pm \frac{1}{2}$, and hence $y = \pm x$. The constraint then gives $x = \pm \frac{1}{\sqrt{2}}$.

Step 3: Compare values. At $y = x$: $f = x^2 = \frac{1}{2}$ (maximum). At $y = -x$: $f = -\frac{1}{2}$ (minimum). Four tangency points, two values—the candidates the Lagrange condition finds always include both maxima and minima; the final comparison of f -values sorts them.

Example 13.2: Closest point on the intersection of two planes

Find the point on the intersection of $x + y + z = 1$ and $x + 2y + 3z = 6$ closest to the origin.

Step 1: Setup. Minimize $f = x^2 + y^2 + z^2$ (squared distance, same minimizer as distance and far friendlier derivatives), subject to two constraints $g_1 = x + y + z - 1 = 0$ and $g_2 = x + 2y + 3z - 6 = 0$. With two constraints come two multipliers:

$$\mathcal{L} = x^2 + y^2 + z^2 - \lambda(x + y + z - 1) - \mu(x + 2y + 3z - 6).$$

Step 2: Stationarity.

$$\mathcal{L}_x = 2x - \lambda - \mu = 0 \Rightarrow x = \frac{\lambda + \mu}{2}, \quad y = \frac{\lambda + 2\mu}{2}, \quad z = \frac{\lambda + 3\mu}{2}.$$

Step 3: Enforce the constraints. Substituting into $g_1 = 0$ and $g_2 = 0$:

$$\begin{aligned} \frac{3\lambda + 6\mu}{2} &= 1 \Rightarrow 3\lambda + 6\mu = 2, \\ \frac{6\lambda + 14\mu}{2} &= 6 \Rightarrow 6\lambda + 14\mu = 12. \end{aligned}$$

Doubling the first and subtracting: $2\mu = 8 \Rightarrow \mu = 4$, then $3\lambda = 2 - 24 \Rightarrow \lambda = -22/3$.
Step 4: Recover the point.

$$\mathbf{x}_* = \left(\frac{\lambda+\mu}{2}, \frac{\lambda+2\mu}{2}, \frac{\lambda+3\mu}{2} \right) = \left(-\frac{5}{3}, \frac{1}{3}, \frac{7}{3} \right).$$

13.2 Two Constrained Problems That Run Machine Learning

13.2.1 Quadratic Forms on the Unit Sphere: PCA, Properly This Time

In Week 6 we asserted that maximizing variance leads to eigenvectors. Now we can *prove* it in two lines. The problem is

$$\max_{\mathbf{w}} \mathbf{w}^T C \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^T \mathbf{w} = 1.$$

Lagrangian: $\mathcal{L} = \mathbf{w}^T C \mathbf{w} - \lambda(\mathbf{w}^T \mathbf{w} - 1)$. By the Week 7 identity table,

$$\nabla_{\mathbf{w}} \mathcal{L} = 2C\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0} \iff \boxed{C\mathbf{w} = \lambda\mathbf{w}}$$

The constrained critical points are exactly the *eigenvectors* of C , and at an eigenvector the objective equals $\mathbf{w}^T C \mathbf{w} = \lambda \mathbf{w}^T \mathbf{w} = \lambda$. So the best direction is the top eigenvector and the achieved variance is the top eigenvalue—the Lagrange multiplier *is* the eigenvalue. (Adding a second constraint $\mathbf{w}^T \mathbf{v}_1 = 0$ and repeating the argument produces the second eigenvector, and so on: all of PCA falls out of constrained optimization.)

13.2.2 Maximum Entropy: Why Uniform Is the Fairest Guess

Choose a discrete probability distribution $p_1, \dots, p_n$ (so $\sum_i p_i = 1$, $p_i \geq 0$) that maximizes the **entropy** $-\sum_i p_i \ln p_i$, a measure of spread-out-ness. Equivalently, minimize $f(\mathbf{p}) = \sum_i p_i \ln p_i$ subject to $g = \sum_i p_i - 1 = 0$:

$$\mathcal{L} = \sum_i p_i \ln p_i - \lambda \left(\sum_i p_i - 1 \right), \quad \frac{\partial \mathcal{L}}{\partial p_i} = \ln p_i + 1 - \lambda = 0 \Rightarrow p_i = e^{\lambda-1}.$$

Every p_i equals the *same* constant; the constraint then forces $p_i = \frac{1}{n}$: **the uniform distribution**. With no other information, spreading belief evenly is mathematically optimal. (A note on the inequality constraints $p_i \geq 0$, which we quietly ignored: the calculation is legitimate because the optimum is *interior*—each $p_i = \frac{1}{n} > 0$, so none of the nonnegativity fences is touched and, in the language of the next chapter, their multipliers are all zero. Conveniently, $p \ln p \rightarrow 0$ as $p \rightarrow 0^+$, and the derivative $\ln p + 1 \rightarrow -\infty$ there, which is exactly what pushes the optimum away from the boundary.) The same calculation with $f = \sum_i p_i \ln(p_i/q_i)$ (the **KL divergence** from a target distribution q , where we assume $q_i > 0$ and $\sum_i q_i = 1$) yields $p_i = q_i$: the distribution closest to q in the information sense is q itself. This innocuous-looking exercise underlies how generative models are trained to imitate data distributions.

13.3 Algorithms for Constraints: Penalties and Projections

Lagrange gives *conditions*; large problems still need *iterations*. Two practical schemes:

- **Penalty method.** Replace the constrained problem by the unconstrained

$$F_\mu(\mathbf{x}) = f(\mathbf{x}) + \frac{1}{2\mu} g(\mathbf{x})^2,$$

and minimize with the tools of Weeks 11–12. The quadratic term fines any violation of $g = 0$; as the penalty weight grows ($\mu \rightarrow 0$), the minimizer of F_μ slides onto the constraint. For instance, minimizing $x^2 + y^2$ subject to $x + y = 2$ with penalty weight μ gives the minimizer $(\frac{1}{1+\mu}, \frac{1}{1+\mu}) \rightarrow (1, 1)$, the exact constrained answer. The trade-off: small μ makes the landscape ill-conditioned (a Week 11 canyon!), demanding a smaller learning rate.

- **Projected gradient descent (PGD).** Alternate a plain gradient step with a projection back onto the constraint set:

$$\mathbf{y}_k = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = (\text{closest point to } \mathbf{y}_k \text{ satisfying } g = 0).$$

Take a free step, then snap back to legality. When the projection is cheap (spheres, boxes, affine subspaces—and Week 3 taught us exactly how to project onto the last), PGD is the method of choice.

13.4 Machine Learning Application

ML Link: Constraints Hidden Inside Everyday ML

PCA maximizes variance subject to unit norm—Section 13.2.1 is its honest derivation. **Softmax layers** handle the probability constraints of classification by construction: the map

$$p_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

turns any raw score vector $\mathbf{z}$ into numbers that are automatically positive and sum to one, so the network can optimize freely over $\mathbf{z}$ with no constraint bookkeeping at all—the simplex constraint is baked into the parametrization. The companion **cross-entropy loss**, $L = -\sum_j y_j \log p_j$, is the negative log-likelihood of the observed label under the model; equivalently, it equals the KL divergence from the empirical label distribution to the model’s distribution, up to an additive constant that does not affect the minimizer (Week 10’s “free simplifications”). That is the precise sense in which this chapter’s maximum-entropy/KL calculation explains the loss function used to train essentially every neural classifier. Exercise 13.6 computes the famous gradient $\nabla_{\mathbf{z}} L = \mathbf{p} - \mathbf{y}$. **Adversarial robustness** research uses projected gradient descent directly: to find the worst small perturbation of an image, one ascends the loss but projects back into the set of perturbations of bounded norm after every step. The Lagrangian is not exotic theory; it is the standard bookkeeping device of the field.

13.5 End of Chapter Exercises

Week 13 Exercises

13.1 Find the point on the parabola $y = \frac{1}{2}x^2$ closest to $(0, 3)$: minimize $f(x, y) = x^2 + (y - 3)^2$ subject to $g(x, y) = y - \frac{1}{2}x^2 = 0$. Write the Lagrangian, derive the stationarity equations, find all stationary points, and identify the true minimizers. (*Answer:* $(\pm 2, 2)$.)

13.2 Maximize $f(x, y) = xy$ subject to $x^2 + y^2 = 2$ using a Lagrange multiplier.

13.3 Re-derive the PCA result: show that the stationary points of $\mathbf{w}^T C \mathbf{w}$ subject to $\|\mathbf{w}\|^2 = 1$ satisfy $C\mathbf{w} = \lambda\mathbf{w}$, and explain why the constrained maximum value equals $\lambda_{\max}(C)$.

13.4 Using a Lagrange multiplier, show that minimizing $\sum_i p_i \ln(p_i/q_i)$ subject to $\sum_i p_i = 1$ yields $p_i = q_i$.

13.5 For the penalty method applied to: minimize x^2 subject to $x = 1$, write $F_\mu(x) = x^2 + \frac{1}{2\mu}(x - 1)^2$, find its minimizer as a function of μ , and verify it tends to the constrained solution as $\mu \rightarrow 0$.

13.6 (Softmax and cross-entropy) Let $p_j = e^{z_j} / \sum_k e^{z_k}$ (softmax) and $L = -\sum_j y_j \log p_j$ with one-hot label ($y_j \in \{0, 1\}$, $\sum_j y_j = 1$). (a) Show $\partial p_j / \partial z_i = p_j(\delta_{ij} - p_i)$, where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise. (b) Combine with the chain rule to show the remarkably clean result

$$\frac{\partial L}{\partial z_i} = p_i - y_i, \quad \text{i.e.} \quad \nabla_{\mathbf{z}} L = \mathbf{p} - \mathbf{y}.$$

(“Prediction minus truth”—the same error-times-nothing pattern that made least squares and logistic regression tidy. This is the single most used gradient in deep learning.)

Chapter 14

Week 14: Inequality Constraints and the KKT Conditions

14.1 The Intuition: The Mathematical Fence

Equality constraints force us *onto* a curve. Real rules are usually one-sided: spend *at most* the budget, keep the temperature *below* the limit, classify with a margin of *at least* one. The template becomes

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) \leq 0,$$

and the constraint now bounds a *region* (the **feasible set**) with the curve $g = 0$ as its fence.

Picture a ball rolling downhill inside a fenced field. At a constrained minimum, there are two possibilities:

1. **Inactive constraint.** The ball settles strictly inside the field, so the fence plays no role. Thus

$$g < 0, \quad \lambda = 0,$$

and the ordinary condition $\nabla f = \mathbf{0}$ holds.

2. **Active constraint.** The ball stops on the fence, so

$$g = 0.$$

At a regular boundary point for a constraint $g \leq 0$, the gradient ∇g points outward from the feasible region. If the ball presses against the fence, the fence supplies an inward reaction $-\lambda \nabla g$, with $\lambda > 0$. Force balance gives

$$-\nabla f - \lambda \nabla g = \mathbf{0},$$

or equivalently,

$$\nabla f + \lambda \nabla g = \mathbf{0}.$$

It is also possible for the ball to stop on the boundary without pressing against the fence; then $g = 0$ but $\lambda = 0$. Thus an active constraint may have $\lambda = 0$ or $\lambda > 0$. A fence can push but never pull, which explains the condition $\lambda \geq 0$.

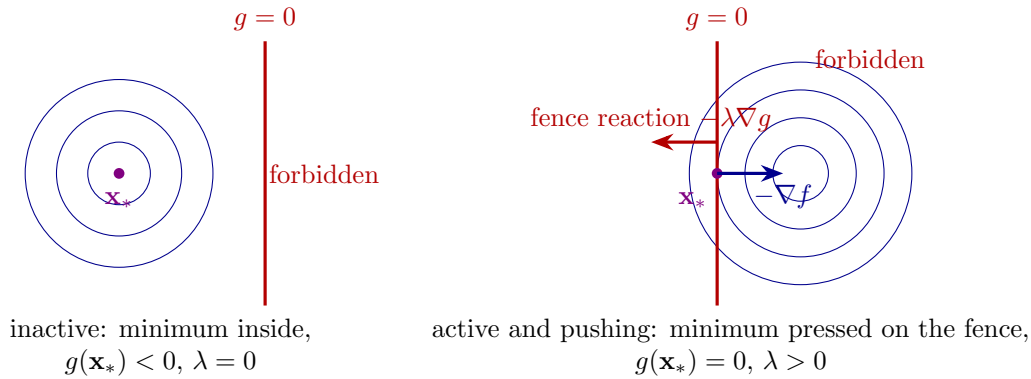


Figure 14.1: The two principal regimes of an inequality constraint. In the inactive case, the unconstrained minimum is feasible and $\lambda = 0$. In the active case, the downhill force $-\nabla f$ is balanced by the inward fence reaction $-\lambda \nabla g$. Equivalently, $\nabla f + \lambda \nabla g = \mathbf{0}$. In both cases, $\lambda g(\mathbf{x}_*) = 0$.

14.2 The KKT Conditions

The two regimes share one elegant algebraic summary. Build the Lagrangian with a *plus* sign (the convention for $g \leq 0$ constraints), $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$.

Key Idea: The Karush–Kuhn–Tucker (KKT) conditions

At a minimizer $\mathbf{x}_*$ of f subject to $g(\mathbf{x}) \leq 0$, there exists $\lambda \geq 0$ such that:

(stationarity)	$\nabla f(\mathbf{x}_*) + \lambda \nabla g(\mathbf{x}_*) = \mathbf{0}$
(primal feasibility)	$g(\mathbf{x}_*) \leq 0$
(dual feasibility)	$\lambda \geq 0$
(complementary slackness)	$\lambda \cdot g(\mathbf{x}_*) = 0$

The last line is the binary switch: *either* the constraint is inactive and its multiplier is dead ($g \leq 0 \implies \lambda = 0$), *or* the multiplier is alive and the constraint is tight ($\lambda > 0 \implies g = 0$). With several constraints $g_1, \dots, g_m \leq 0$, each gets its own multiplier $\lambda_i \geq 0$ and its own slackness equation $\lambda_i g_i(\mathbf{x}_*) = 0$.

Fine print, stated once. For general nonlinear problems the KKT conditions are *necessary* under mild regularity assumptions on the constraints, called **constraint qualifications** (every example in this book satisfies them; trouble can arise, for instance, when constraint gradients vanish or line up degenerately at the solution). They become *sufficient* for optimality when the problem is convex—a convex objective minimized over a convex feasible set, as in every ML application of Weeks 14–15.

How to solve KKT systems by hand: guess which constraints are active, solve the resulting equality-constrained problem, then *check* the guess—inactive constraints must come out satisfied ($g < 0$) and active multipliers must come out nonnegative. Wrong guesses fail one of the checks; revise and repeat.

Example 14.1: One fence, both regimes

(a) **Inactive.** Minimize $f = x^2 + y^2$ subject to $g = x - 4 \leq 0$ (i.e. $x \leq 4$). Try $\lambda = 0$: then stationarity is $\nabla f = \mathbf{0}$, giving $(0, 0)$, and the check $g(0, 0) = -4 \leq 0$ passes. **Solution** $(0, 0)$, **fence irrelevant**.

(b) **Active.** Minimize $f = x^2 + y^2$ subject to $2 - x \leq 0$ (i.e. $x \geq 2$). Try $\lambda = 0$ first: the unconstrained minimum $(0, 0)$ violates $x \geq 2$ — guess rejected. So the constraint must be active: $x = 2$. The Lagrangian is $\mathcal{L} = x^2 + y^2 + \lambda(2 - x)$, with stationarity

$$\mathcal{L}_x = 2x - \lambda = 0 \Rightarrow \lambda = 2x = 4, \quad \mathcal{L}_y = 2y = 0 \Rightarrow y = 0.$$

The check $\lambda = 4 \geq 0$ passes: the fence is genuinely pushing back. **Solution** $(2, 0)$ **with multiplier** $\lambda = 4$.

Example 14.2: Two constraints, mixed status (from lecture)

Minimize $f(x, y) = (x - 3)^2 + (y - 3)^2$ subject to $g_1 = x + y - 4 \leq 0$ and $g_2 = x - 1 \leq 0$.

Geometry first. The unconstrained minimum $(3, 3)$ violates both constraints, so at least one is active.

Guess: only g_1 active ($\lambda > 0, \mu = 0$). Then we minimize on the line $x + y = 4$. Stationarity of $\mathcal{L} = f + \lambda g_1$:

$$2(x - 3) + \lambda = 0, \quad 2(y - 3) + \lambda = 0 \Rightarrow x = y.$$

With $x + y = 4$: the candidate is $(2, 2)$ with $\lambda = 2(3 - 2) = 2 \geq 0 \checkmark$. But check g_2 : $g_2(2, 2) = 2 - 1 = 1 > 0$ — infeasible! Guess rejected.

Guess: both active. Then $x = 1$ (from g_2) and $y = 4 - x = 3$ (from g_1). Solve for the multipliers from stationarity of $\mathcal{L} = f + \lambda g_1 + \mu g_2$:

$$2(x - 3) + \lambda + \mu = 0 \Rightarrow \lambda + \mu = 4, \quad 2(y - 3) + \lambda = 0 \Rightarrow \lambda = 0.$$

So $\lambda = 0, \mu = 4$. Both multipliers are $\geq 0 \checkmark$, both constraints hold $\checkmark$ — and the solution is $\mathbf{x}_* = (1, 3)$ with $\mu = 4$. The value $\lambda = 0$ carries a precise geometric message: at $(1, 3)$ the constraint g_1 is *binding* ($g_1(1, 3) = 1 + 3 - 4 = 0$, the point sits exactly on that fence too), yet it exerts *no force*—remove it and the solution would not move. Only g_2 has a positive multiplier and genuinely pushes back. (Re-solving with only g_2 active indeed gives $(1, 3)$ directly.) The lesson: “touching the fence” and “leaning on the fence” are different things, and complementary slackness lets the *algebra* distinguish them even when your geometric guess is imperfect.

14.3 Machine Learning Application

ML Link: ReLU Is a KKT Solution

Here is the simplest constrained problem in machine learning: given a number y , find the closest *nonnegative* number,

$$\min_p \frac{1}{2}(p - y)^2 \quad \text{subject to} \quad -p \leq 0.$$

KKT: stationarity $(p - y) - \lambda = 0$; slackness $\lambda p = 0$. If $y \geq 0$: take $\lambda = 0$, $p = y$ (interior). If $y \leq 0$: the constraint activates, $p = 0$, $\lambda = -y \geq 0$. Compactly:

$$p_*(y) = \max(0, y) = \text{ReLU}(y),$$

the **Rectified Linear Unit**—the most-used activation function in deep learning, evaluated trillions of times a day. ReLU admits this projection/KKT interpretation.

14.4 End of Chapter Exercises

Week 14 Exercises

14.1 Minimize $f(x, y) = (x - 3)^2 + (y - 3)^2$ subject to the single constraint $x + y \leq 4$. Write the Lagrangian, assume the constraint is active, solve for (x^*, y^*) and λ , and verify all four KKT conditions.

14.2 Minimize $f(x, y) = x^2 + y^2$ subject to $x + 2y \geq 4$ (rewrite as $4 - x - 2y \leq 0$ first). Determine whether the constraint is active and find the solution and multiplier.

14.3 Minimize $f(x, y) = (x - 2)^2 + y^2$ subject to $x \leq 4$ and $y \geq 0$. Determine the status (active/inactive) of each constraint at the solution.

14.4 Carry out the ReLU derivation: solve $\min_p \frac{1}{2}(p - y)^2$ s.t. $p \geq 0$ via KKT for the two cases $y > 0$ and $y \leq 0$, and confirm $p_* = \max(0, y)$.

14.5 (Conceptual) A problem has 10,000 inequality constraints. Explain why “guess the active set” is hopeless as an algorithm (2^{10000} guesses), and why this motivates the dual approach of Week 15.

14.6 (Debug the statement) A student says: “If $g(\mathbf{x}_*) = 0$ at the solution, then the inequality constraint must have a positive multiplier.” Is this correct? Give a concrete counterexample. (*Hint: Example 14.2 contains one; alternatively, minimize x^2 subject to $x \leq 0$.*)

Chapter 15

Week 15: Duality, Soft Penalties, and Machine Learning Capstones

15.1 The Intuition: Bypassing the Binary Guesswork

The KKT conditions are correct but combinatorial: with m fences, which subset is active? Checking all 2^m patterns is impossible at scale. The **dual problem** sidesteps the guessing entirely. Instead of hunting for $\mathbf{x}$ while juggling constraint statuses, we flip the script: for each fixed penalty level $\lambda \geq 0$, let the $\mathbf{x}$ -minimization happen freely and in closed form; what remains is a clean maximization over λ alone, in which the active/inactive decision resolves itself automatically (the sign of the optimal λ is the answer).

Definition: The dual function and dual problem

For the problem $\min f(\mathbf{x})$ s.t. $g(\mathbf{x}) \leq 0$ with Lagrangian $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$, define the **dual function**

$$q(\lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda) \quad (\text{an unconstrained minimization, often solvable by formula}).$$

The **dual problem** is then $\max_{\lambda \geq 0} q(\lambda)$. For every $\lambda \geq 0$, $q(\lambda)$ is a *lower bound* on the true optimal value (weak duality). For convex problems satisfying a mild regularity condition—the standard one, *Slater's condition*, requires a point that satisfies the inequality constraints strictly, $g(\mathbf{x}) < 0$ —the highest lower bound matches the optimal value exactly (strong duality), and the maximizing λ is the KKT multiplier. Every dual we compute in this book is of this well-behaved kind; see [3] for the general theory.

Example 15.1: The dual finds the multiplier automatically

Revisit Example 14.1(b): minimize $x^2 + y^2$ subject to $2 - x \leq 0$, with $\mathcal{L} = x^2 + y^2 + \lambda(2 - x)$.

Step 1: Minimize out $\mathbf{x}$ for fixed λ . $\partial_x \mathcal{L} = 2x - \lambda = 0 \Rightarrow x = \frac{\lambda}{2}$; $\partial_y \mathcal{L} = 2y = 0 \Rightarrow y = 0$.

Step 2: Build $q(\lambda)$. Substitute back:

$$q(\lambda) = \left(\frac{\lambda}{2}\right)^2 + 0 + \lambda\left(2 - \frac{\lambda}{2}\right) = \frac{\lambda^2}{4} + 2\lambda - \frac{\lambda^2}{2} = 2\lambda - \frac{\lambda^2}{4}.$$

A downward parabola in λ —always concave, whatever the original problem, which is what makes the dual so pleasant to maximize.

Step 3: Maximize. $q'(\lambda) = 2 - \frac{\lambda}{2} = 0 \Rightarrow \lambda = 4 \geq 0$. The dual found $\lambda = 4$ with no case analysis; plugging back gives $x = 2, y = 0$, matching Week 14. And $q(4) = 8 - 4 = 4 = f(2, 0)$: the bound is tight (strong duality).

Example 15.2: A two-constraint dual (from lecture)

Revisit Example 14.2: minimize $f = (x - 3)^2 + (y - 3)^2$ subject to $x + y \leq 4$ and $x \leq 1$, now *without* guessing active sets. The Lagrangian, with one multiplier per fence, is

$$\mathcal{L} = (x - 3)^2 + (y - 3)^2 + \lambda(x + y - 4) + \mu(x - 1), \quad \lambda, \mu \geq 0.$$

Step 1: Minimize out (x, y) for fixed (λ, μ) .

$$\partial_x \mathcal{L} = 2(x - 3) + \lambda + \mu = 0 \Rightarrow x = 3 - \frac{\lambda + \mu}{2}, \quad \partial_y \mathcal{L} = 2(y - 3) + \lambda = 0 \Rightarrow y = 3 - \frac{\lambda}{2}.$$

Step 2: Build $q(\lambda, \mu)$. Substituting back (using $x - 3 = -\frac{\lambda + \mu}{2}$, $y - 3 = -\frac{\lambda}{2}$, $x + y - 4 = 2 - \lambda - \frac{\mu}{2}$, and $x - 1 = 2 - \frac{\lambda + \mu}{2}$) and collecting terms:

$$q(\lambda, \mu) = 2\lambda + 2\mu - \frac{\lambda^2}{2} - \frac{\lambda\mu}{2} - \frac{\mu^2}{4}.$$

As always, the dual is concave—a downward-dome in (λ, μ) .

Step 3: Maximize over $\lambda, \mu \geq 0$.

$$\frac{\partial q}{\partial \lambda} = 2 - \lambda - \frac{\mu}{2} = 0, \quad \frac{\partial q}{\partial \mu} = 2 - \frac{\lambda}{2} - \frac{\mu}{2} = 0.$$

Subtracting the second equation from the first gives $-\frac{\lambda}{2} = 0$, so $\lambda = 0$, and then $\mu = 4$. Both values already satisfy the sign constraints, so no clamping is even needed: the unconstrained maximizer of q is the dual solution.

Step 4: Read off everything. Plugging back:

$$x = 3 - \frac{0 + 4}{2} = 1, \quad y = 3 - \frac{0}{2} = 3,$$

matching Example 14.2 exactly—including the subtle part. The dual discovered *by itself* that the budget fence $x + y \leq 4$, although touched by the solution $(1, 3)$, exerts no force ($\lambda = 0$), while the fence $x \leq 1$ pushes with strength $\mu = 4$. And the bound is tight: $q(0, 4) = 8 - 4 = 4 = f(1, 3)$ — strong duality, verified by hand. No case analysis, no 2^m guesses: the active set fell out of a smooth maximization.

15.2 Soft Fences: The Exterior Penalty Method

Sometimes the cleanest algorithm replaces the hard fence by a rubber band. The **exterior penalty method** drops multipliers entirely:

$$F(\mathbf{x}) = f(\mathbf{x}) + \rho \max(0, g(\mathbf{x}))^2.$$

Inside the feasible region ($g \leq 0$) the max returns 0: no penalty. The moment the iterate crosses the fence, a quadratic fine—with stiffness ρ —pulls it back. As $\rho \rightarrow \infty$, the minimizer of F converges to the true constrained solution (Exercise 15.2 walks through the 1D case). The penalty function $\max(0, \cdot)$ should look familiar: it is the ReLU again, now serving as a constraint enforcer.

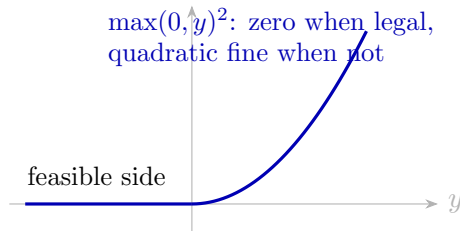


Figure 15.1: The squared-ReLU penalty: flat inside the feasible region, a steepening wall outside it. Increasing ρ stiffens the wall toward a hard constraint.

15.3 Capstone 1: Allocating a Data Budget Optimally

A genuinely practical ML problem from lecture, solved start to finish with this course’s tools. You can buy training samples from two data sources. Source i has noise level V_i and per-sample cost C_i ; buying $N_i > 0$ samples reduces your model’s error variance by averaging, contributing V_i/N_i . Your budget is B . How should you split it?

$$\min_{N_1, N_2} \frac{V_1}{N_1} + \frac{V_2}{N_2} \quad \text{subject to} \quad C_1 N_1 + C_2 N_2 \leq B.$$

The budget constraint is clearly active (more data always helps), so by KKT with multiplier λ :

$$-\frac{V_i}{N_i^2} + \lambda C_i = 0 \quad \implies \quad N_i = \frac{1}{\sqrt{\lambda}} \sqrt{\frac{V_i}{C_i}}.$$

Each allocation is proportional to $\sqrt{V_i/C_i}$: noisier sources deserve more samples, pricier sources fewer—but by *square-root* laws, not linear ones (a source $4\times$ noisier merits only $2\times$ the samples). Enforcing the budget pins down λ and yields

$$N_1 = B \frac{\sqrt{V_1/C_1}}{\sqrt{V_1 C_1} + \sqrt{V_2 C_2}},$$

and symmetrically for N_2 . A data-purchasing strategy used in practice, derived in five lines of KKT.

15.4 Capstone 2: Support Vector Machines

The closing example draws on every part of the course. We want to classify data points $\mathbf{x}_i \in \mathbb{R}^d$ with labels $y_i \in \{+1, -1\}$ (cats vs. dogs) by a separating hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$. Among all separating hyperplanes, the safest maximizes its **margin**—the buffer zone to the nearest points. The scale of $(\mathbf{w}, b)$ is fixed by requiring

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1.$$

The two margin boundaries are therefore

$$\mathbf{w}^T \mathbf{x} + b = 1 \quad \text{and} \quad \mathbf{w}^T \mathbf{x} + b = -1,$$

and the distance between them is

$$\frac{2}{\|\mathbf{w}\|}.$$

Thus maximizing the margin is equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$ subject to the classification constraints.

A point whose constraint is strict lies outside the margin and has $\lambda_i = 0$ by complementary slackness. Points with $\lambda_i > 0$ satisfy

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$$

and lie on one of the margin boundaries. These are the *support vectors*: they are the training points that determine the maximum-margin separator.

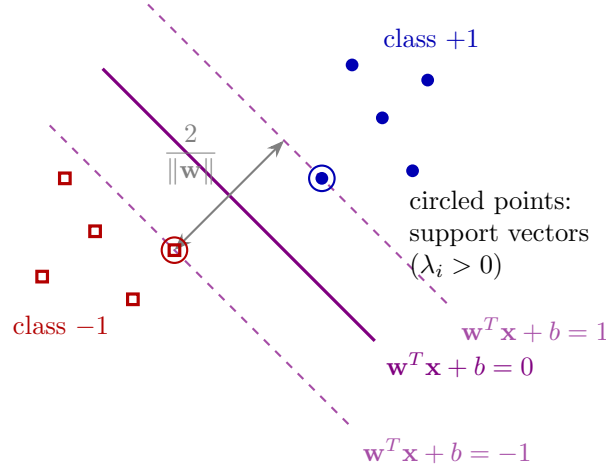


Figure 15.2: A hard-margin support vector machine selects the separating hyperplane with the largest margin. Under the normalization $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$, the margin boundaries are $\mathbf{w}^T \mathbf{x} + b = \pm 1$ and their separation is $2/\|\mathbf{w}\|$. The circled points lie on these boundaries and have nonzero KKT multipliers; they are the support vectors that determine the separator.

What KKT reveals. Attach a multiplier $\lambda_i \geq 0$ to each data point's constraint. Complementary slackness says

$$\lambda_i \cdot (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) = 0 :$$

points safely beyond the margin have *inactive* constraints and $\lambda_i = 0$; only points lying exactly on the margin keep $\lambda_i > 0$. Stationarity of the Lagrangian in $\mathbf{w}$ gives

$$\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i,$$

and stationarity in the bias b gives a second, often-forgotten condition,

$$\sum_i \lambda_i y_i = 0$$

(the support vectors’ “pushes” from the two classes must balance). The learned classifier is built *only* from the points with $\lambda_i > 0$. These few critical points are the **support vectors**, and they give the method its name. A dataset of a million images may yield a classifier supported on a few hundred of them: KKT performs automatic data compression.

The dual, explicitly. Substituting $\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$ back into the Lagrangian (Exercise 15.6 carries this out) eliminates $\mathbf{w}$ and b entirely and leaves

$$\max_{\lambda_i \geq 0} \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{subject to} \quad \sum_i \lambda_i y_i = 0.$$

This is the problem SVM software actually solves: a concave maximization over the multipliers—exactly the structure of this chapter—in which the data enter *only* through the inner products $\mathbf{x}_i^T \mathbf{x}_j$ of Week 2. (That last observation is the doorway to the famous “kernel trick”: replace $\mathbf{x}_i^T \mathbf{x}_j$ by a more exotic similarity measure and the same dual learns curved boundaries. We leave that door ajar for your next ML course.)

15.4.1 Soft Margins: When the Data Refuse to Separate

One mislabeled photo may make the hard-margin feasible set empty. The practical fix gives every point i a **slack variable** $\xi_i \geq 0$ measuring how badly it violates its margin, and charges for the total slack in the objective:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

This is the **soft-margin SVM** [10]—the version used in practice. The parameter $C > 0$ sets the exchange rate between a wide margin and a few violations: large C approaches the hard margin (violations nearly forbidden), small C tolerates outliers in exchange for a wider, calmer boundary. Note the family resemblance to Week 9’s overfitting discussion: C is a model-complexity dial, tuned on held-out data. The KKT analysis goes through with one elegant change: the multipliers become *capped*, $0 \leq \lambda_i \leq C$, so no single point—however badly mislabeled—can exert unbounded influence on the classifier.

The soft-margin SVM has the same dual objective and equality constraint as the hard-margin problem, but its multipliers also satisfy $0 \leq \lambda_i \leq C$ for any i .

15.5 Machine Learning Application

ML Link: One Course, One Story

Step back and watch the threads tie together. Data became vectors and matrices (Weeks 1–2); geometry gave us projections and the SVD (Weeks 3–6); calculus turned learning into descending a landscape (Weeks 7–9); iteration theory told us how fast we can descend and how to accelerate (Weeks 10–12); and constrained optimization handled the rules—ending with ReLU as a KKT solution, cross-entropy from maximum entropy, optimal data budgeting, and SVMs from duality (Weeks 13–15). Many core machine learning models can be understood through exactly these three tools: linear algebra for *representation*, calculus for *sensitivity*, and optimization for *learning*. Your next ML courses will add statistical reasoning and architectural ideas on top—but when they compute, they will compute with the operations of this book. That was the purpose of MATH 225.

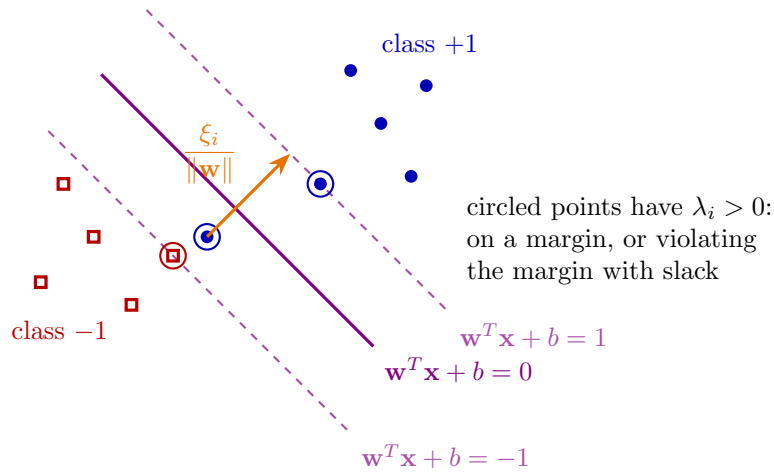


Figure 15.3: A soft-margin SVM permits violations of the constraints $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ by introducing slack variables: $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$, with $\xi_i \geq 0$. The orange arrow shows the geometric distance $\xi_i / \|\mathbf{w}\|$ from the misclassified blue point to its correct-class margin, while the objective penalizes $C\xi_i$. Points strictly beyond their correct margin have $\xi_i = 0$ and $\lambda_i = 0$.

15.6 End of Chapter Exercises

Week 15 Capstone Exercises

15.1 For the problem: minimize $f(x, y) = (x - 3)^2 + (y - 3)^2$ subject to $x + y \leq 4$, derive the dual function $q(\lambda)$ by minimizing the Lagrangian over (x, y) , show $q(\lambda) = -\frac{\lambda^2}{2} + 2\lambda + \text{const}$, maximize it, and recover (x^*, y^*) .

15.2 Consider: minimize $f(x) = x^2$ subject to $x \geq 1$. (a) Write the exterior penalty function $F(x, \rho) = x^2 + \rho \max(0, 1 - x)^2$. (b) For $x < 1$, set $F'(x) = 0$ and solve for the minimizer x_ρ as a function of ρ . (c) Show $x_\rho \rightarrow 1$ as $\rho \rightarrow \infty$.

15.3 In the data-budget problem, suppose $V_1 = 4V_2$ and $C_1 = C_2$. Using $N_i \propto \sqrt{V_i/C_i}$, what fraction of the budget goes to source 1?

15.4 Solve via KKT: minimize $\frac{1}{2}(p - y)^2$ subject to $p \geq 0$, and explain in one sentence why the answer is the ReLU function.

15.5 Explain in your own words how complementary slackness allows a support vector machine to discard most of its training data after training, and why the remaining points are called support vectors.

15.6 (Deriving the SVM dual) Starting from the Lagrangian

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_i \lambda_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)), \quad \lambda_i \geq 0,$$

(a) set $\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{0}$ and $\partial \mathcal{L} / \partial b = 0$ to obtain $\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$ and $\sum_i \lambda_i y_i = 0$; (b) substitute both back into $\mathcal{L}$ to derive the dual objective $\sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$.

15.7 (A tiny SVM by hand) Take the four training points

$$\mathbf{x}_1 = (1, 0), y_1 = +1;$$

$$\mathbf{x}_2 = (2, 1), y_2 = +1;$$

$$\mathbf{x}_3 = (-1, 0), y_3 = -1;$$

$$\mathbf{x}_4 = (-2, -1), y_4 = -1.$$

(a) Sketch the points and argue from the picture that the maximum-margin boundary is the vertical line $x = 0$, with margin boundaries $x = \pm 1$, i.e. $\mathbf{w} = (1, 0)$ and $b = 0$. (b) Which points are the support vectors? (c) Verify the KKT conditions: find multipliers $\lambda_1, \dots, \lambda_4 \geq 0$ satisfying $\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$, $\sum_i \lambda_i y_i = 0$, and complementary slackness. (*Answer:* $\lambda_1 = \lambda_3 = \frac{1}{2}$, $\lambda_2 = \lambda_4 = 0$.)

References and Further Reading

This book has emphasized the mathematical techniques that connect many machine learning methods: linear algebra for representing data, calculus for measuring sensitivity and carrying out back-propagation, and optimization for fitting models. It is not intended to be a comprehensive survey of machine learning. From here, readers may follow either of two complementary paths.

Students who would like a deeper mathematical and algorithmic treatment can continue with Deisenroth, Faisal, and Ong [13], which is closest in spirit to this book and develops the mathematical foundations of several additional learning methods. Murphy [14] gives a broad modern treatment organized around probability and statistical modeling. Shalev-Shwartz and Ben-David [15] is a more theoretical next step for readers interested in questions such as why learning algorithms generalize, how much data they require, and how their computational costs are analyzed. The books by Bishop [5], Hastie, Tibshirani, and Friedman [6], and Goodfellow, Bengio, and Courville [7] provide further depth in probabilistic modeling, statistical learning, and deep learning, respectively.

Readers who prefer to encounter more machine learning models through examples and computation may begin with James, Witten, Hastie, Tibshirani, and Taylor [16]. It gives a less technical introduction to regression, classification, tree-based methods, support-vector machines, clustering, and deep learning, together with Python laboratories. Géron [17] takes an even more implementation-oriented route, progressing from complete machine-learning projects with Scikit-Learn to neural networks and modern deep learning with PyTorch.

These two paths reinforce one another. A useful progression is to derive a small model by hand, implement its essential steps with NumPy, and then repeat the experiment using a machine learning library. Comparing the training, validation, and test behavior while changing the data, model, and hyperparameters is often the best way to turn the mathematics into working intuition.

Bibliography

- [1] G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley–Cambridge Press, 2023. (The classic source for Part I.)
- [2] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*, SIAM, 1997. (QR, SVD, and conditioning, one level deeper.)
- [3] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. (Duality, KKT, and Slater’s condition in full rigor.)
- [4] J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. (Newton-type and constrained methods.)
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009. (Regression, regularization, and the train/test trade-off.)
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [8] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: bringing order to the web,” Stanford InfoLab Technical Report, 1999.
- [9] M. Turk and A. Pentland, “Eigenfaces for recognition,” *Journal of Cognitive Neuroscience* 3(1):71–86, 1991.
- [10] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning* 20:273–297, 1995.
- [11] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” *Proceedings of ICLR*, 2015.
- [12] B. T. Polyak, “Some methods of speeding up the convergence of iteration methods,” *USSR Computational Mathematics and Mathematical Physics* 4(5):1–17, 1964. (The heavy-ball method of Week 12.)
- [13] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. (A natural continuation of the mathematical foundations developed here.)
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. (A broad modern treatment connecting classical and deep learning through probabilistic modeling.)
- [15] S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*, Cambridge University Press, 2014. (Learning theory, generalization, and algorithms at an advanced undergraduate level.)

- [16] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. (An accessible, example-driven introduction with Python laboratories.)
- [17] A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, O'Reilly Media, 2025. (A coding-first guide to complete machine learning projects and modern deep learning.)